

C语言笔记

零基础小白也能看懂的C语言学习笔记

作者: [HQ_herold]

适合人群: 编程零基础、大学生、自学爱好者

目录

【基础篇】

- 第一章: C语言概述与环境搭建
- 第二章: 数据类型与变量
- 第三章: 运算符与表达式
- 第四章: 流程控制 (if-else、switch)
- 第五章: 循环 (for、while)

【进阶篇】

- 第六章: 数组 (一维、二维、多维)
- 第七章: 函数
- 第八章: 指针 (C语言的灵魂)
- 第九章: 指针与函数 (函数指针、指针函数)
- 第十章: 链表 (单向链表、循环链表)
- 附录: 常见问题与练习题

【框架与设计模式篇】

- 第十一章: C语言常用框架
- 第十二章: C语言常用设计模式

【单片机外设篇】

- 第十三章: GPIO (通用输入输出)
- 第十四章: 中断系统
- 第十五章: 定时器
- 第十六章: 串行通信 (UART/I2C/SPI)
- 第十七章: ADC模数转换
- 第十八章: 其他常用外设

【FreeRTOS实时操作系统篇】

- 第十九章: FreeRTOS简介与核心概念
- 第二十章: 任务管理
- 第二十一章: 调度机制

- 第二十二章：同步与通信机制
- 第二十三章：内存管理
- 第二十四章：中断管理与定时器

第一章：C语言概述与环境搭建

1.1 什么是C语言？

一句话理解

C语言是一种人和计算机交流的语言。就像我们用中文交流一样，我们用C语言告诉计算机要做什么。

为什么学C语言？

优点	说明
基础扎实	学好C语言，学其他语言更容易
运行效率高	适合对性能要求高的场景
应用广泛	操作系统、嵌入式、游戏开发都用它
面试常考	很多公司面试必问C语言

C语言能做什么？

- 操作系统（Windows、Linux的核心都是C写的）
- 嵌入式开发（智能家电、汽车电子）
- 游戏引擎
- 数据库（MySQL、Redis）
- 驱动程序

1.2 第一个C程序

经典的"Hello World"

```
#include <stdio.h>

int main()
{
    printf("Hello, world!\n");
    return 0;
}
```

程序逐行解释

代码	解释
<code>#include <stdio.h></code>	引入标准输入输出库，让我们能用printf
<code>int main()</code>	主函数，程序的入口，每个C程序必须有且只有一个main函数
<code>{ }</code>	花括号，表示函数的开始和结束
<code>printf("Hello, world!\n");</code>	输出一句话到屏幕上，\n表示换行
<code>return 0;</code>	程序正常结束，返回0给系统
<code>;</code>	分号，每条语句必须以分号结尾， 新手最容易忘

新手常见错误

✖ 忘记分号

`printf("Hello")` // 错误！缺少分号

✔ 正确写法

`printf("Hello");` // 正确

✖ 拼写错误

`print("Hello");` // 错误！C语言没有print函数

✔ 正确写法

`printf("Hello");` // 正确

1.3 环境搭建（推荐新手）

方案一：在线编译（最简单，推荐新手）

不用安装任何软件，直接在网页写代码。

推荐网站：

- 菜鸟教程在线工具：<https://c.runoob.com/compile/11/>
- OnlineGDB：https://www.onlinegdb.com/online_c_compiler

优点：不用安装，打开即用

缺点：必须联网

方案二：Dev-C++（推荐初学者）

下载地址：<https://sourceforge.net/projects/orwelldevcpp/>

安装步骤：

1. 下载安装包
2. 双击安装，一路"下一步"
3. 安装完成，桌面会出现Dev-C++图标

使用方法：

1. 打开Dev-C++
2. 点击"文件" → "新建" → "源代码"
3. 输入代码
4. 按 F11 编译运行

方案三：Visual Studio Code（进阶推荐）

适合有一定基础后使用，功能更强大。

需要安装：

1. VS Code编辑器
2. C/C++扩展插件
3. MinGW编译器

1.4 C程序的编写流程

编写代码 → 编译 → 链接 → 运行

通俗理解：

- └─ 编写代码：你写的C语言代码（人能看懂）
- └─ 编译：把C代码翻译成机器能理解的中间代码
- └─ 链接：把中间代码组合成可执行程序
- └─ 运行：执行程序，看到结果

第一章小结

本章要点

1. C语言是一种编程语言，适合入门学习
2. 每个C程序必须有main函数
3. 每条语句以分号结尾
4. 新手推荐用在线编译器或Dev-C++

动手练习

练习1：修改Hello World程序，输出你的名字。

```
#include <stdio.h>

int main()
{
    printf("我的名字是小明\n");
    return 0;
}
```

练习2：输出多行文字。

```
#include <stdio.h>

int main()
{
    printf("第一行\n");
    printf("第二行\n");
    printf("第三行\n");
    return 0;
}
```

第二章：数据类型与变量

2.1 什么是变量？

一句话理解

变量就是一个**装数据的盒子**，给盒子起个名字，方便以后使用。

生活类比

```
变量 = 盒子
变量名 = 盒子的标签
变量值 = 盒子里装的东西
```

例如：

```
int age = 18;
```

- `int`：盒子类型（只能装整数）
 - `age`：盒子标签（变量名）
 - `18`：盒子里装的东西（变量值）
-

2.2 基本数据类型

C语言的基本数据类型：

类型	关键字	占用空间	能存什么	举例
整型	int	4字节	整数	1, 100, -5
短整型	short	2字节	小范围整数	1, 100
长整型	long	4/8字节	大整数	100000
字符型	char	1字节	单个字符	'A', '0'
单精度浮点	float	4字节	小数	3.14
双精度浮点	double	8字节	更精确的小数	3.14159265

新手重点掌握

初学者只需要记住三个：

- int - 存整数
- float - 存小数
- char - 存字符

2.3 变量的定义与使用

定义变量的语法

数据类型 变量名 = 初始值；

示例代码

```
#include <stdio.h>

int main()
{
    // 定义整数变量
    int age = 18;

    // 定义小数变量
    float height = 1.75;

    // 定义字符变量（注意用单引号）
    char grade = 'A';

    // 输出变量的值
    printf("年龄: %d岁\n", age);
```

```
printf("身高: %.2f米\n", height); // .2f表示保留2位小数
printf("等级: %c\n", grade);

return 0;
}
```

输出格式符（重点记忆）

格式符	对应类型	说明
%d	int	输出整数
%f	float/double	输出小数
%.2f	float/double	保留2位小数
%c	char	输出单个字符
%s	字符串	输出字符串

2.4 变量命名规则

必须遵守的规则

规则	正确示例	错误示例
只能由字母、数字、下划线组成	age, user_name, age1	user-name, @age
必须以字母或下划线开头	age, _name	1age, 2name
不能用关键字	my_int	int, float
区分大小写	Age 和 age 是两个变量	-

推荐的命名习惯

✓ 好的命名（一看就知道是什么）

```
int studentAge = 18; // 学生年龄
float totalPrice = 99.5; // 总价
char userGrade = 'A'; // 用户等级
```

✗ 不好的命名

```
int a = 18; // 谁知道a是什么？
float x = 99.5; // 完全看不懂
```

2.5 变量的赋值

先定义，后赋值

```
int age;           // 先定义
age = 18;          // 后赋值
```

定义时直接赋值（推荐）

```
int age = 18;      // 定义并赋值
```

修改变量的值

```
int age = 18;
printf("年龄: %d\n", age); // 输出18

age = 20;                // 修改值
printf("年龄: %d\n", age); // 输出20
```

2.6 常量

什么是常量？

常量是不能改变的值，定义后不能修改。

定义常量的两种方式

方式一：#define（推荐）

```
#include <stdio.h>

#define PI 3.14159    // 定义常量PI

int main()
{
    printf("圆周率 = %f\n", PI);
    return 0;
}
```


方式二：const关键字

```
#include <stdio.h>

int main()
{
    const float PI = 3.14159;    // 定义常量PI
    printf("圆周率 = %f\n", PI);

    // PI = 3.14;  // 错误！常量不能修改

    return 0;
}
```

两者的区别

方式	语法	区别
#define	#define 常量名 值	没有类型，在编译前替换
const	const 类型 常量名 = 值	有类型，更安全

第二章小结

本章要点

- 1. 变量是装数据的盒子
- 2. 基本数据类型：int、float、char
- 3. 变量命名要规范
- 4. 常量定义后不能修改

动手练习

练习1：定义三个变量，分别存储你的年龄、身高、成绩等级，并输出。

```
#include <stdio.h>

int main()
{
    int age = 20;
    float height = 1.75;
    char grade = 'B';

    printf("年龄: %d岁\n", age);
    printf("身高: %.2f米\n", height);
    printf("成绩等级: %c\n", grade);

    return 0;
}
```

```
}

```

练习2：计算圆的面积。

```
#include <stdio.h>

#define PI 3.14159

int main()
{
    float radius = 5.0;           // 半径
    float area = PI * radius * radius; // 面积

    printf("半径 = %.2f\n", radius);
    printf("面积 = %.2f\n", area);

    return 0;
}
```

第三章：运算符与表达式

3.1 什么是运算符？

一句话理解

运算符就是进行运算的符号，比如加减乘除。

什么是表达式？

表达式是由变量、常量和运算符组成的式子，能计算出一个结果。

```
int a = 5 + 3;    // 5 + 3 就是表达式，结果是8

```

3.2 算术运算符

运算符	名称	示例	结果
+	加法	5 + 3	8
-	减法	5 - 3	2
*	乘法	5 * 3	15
/	除法	5 / 3	1（整数除法）
%	取余	5 % 3	2

重点注意：整数除法

```
#include <stdio.h>

int main()
{
    int a = 5;
    int b = 3;

    printf("5 / 3 = %d\n", a / b);    // 结果是1, 不是1.67
    printf("5 / 3 = %f\n", 5.0 / 3); // 结果是1.67

    return 0;
}
```

规律：

- 整数 ÷ 整数 = 整数 (舍去小数)
- 想得到小数结果，至少有一个数要是小数

取余运算 (%)

```
#include <stdio.h>

int main()
{
    printf("10 %% 3 = %d\n", 10 % 3); // 结果是1
    printf("10 %% 2 = %d\n", 10 % 2); // 结果是0

    // 常见用途：判断奇偶
    int num = 7;
    if (num % 2 == 0) {
        printf("%d是偶数\n", num);
    } else {
        printf("%d是奇数\n", num);    // 输出这个
    }

    return 0;
}
```

3.3 赋值运算符

基本赋值运算符

```
int a = 10;    // 把10赋值给a
```

复合赋值运算符

运算符	等价于	示例
<code>+=</code>	<code>a = a + b</code>	<code>a += 3</code> → <code>a = a + 3</code>
<code>-=</code>	<code>a = a - b</code>	<code>a -= 3</code> → <code>a = a - 3</code>
<code>*=</code>	<code>a = a * b</code>	<code>a *= 3</code> → <code>a = a * 3</code>
<code>/=</code>	<code>a = a / b</code>	<code>a /= 3</code> → <code>a = a / 3</code>
<code>%=</code>	<code>a = a % b</code>	<code>a %= 3</code> → <code>a = a % 3</code>

示例代码

```
#include <stdio.h>

int main()
{
    int a = 10;

    a += 5;    // a = a + 5 = 15
    printf("a += 5: %d\n", a);

    a -= 3;    // a = a - 3 = 12
    printf("a -= 3: %d\n", a);

    a *= 2;    // a = a * 2 = 24
    printf("a *= 2: %d\n", a);

    return 0;
}
```

3.4 自增自减运算符

基本用法

运算符	名称	说明
<code>++</code>	自增	变量值加1
<code>--</code>	自减	变量值减1

前置与后置的区别（重点）

```
#include <stdio.h>

int main()
{
    int a = 5;
    int b;

    // 后置++：先用后加
    b = a++;
    printf("a = %d, b = %d\n", a, b); // a=6, b=5

    // 重置
    a = 5;

    // 前置++：先加后用
    b = ++a;
    printf("a = %d, b = %d\n", a, b); // a=6, b=6

    return 0;
}
```

记忆口诀

后置++：先用再加
前置++：先加再用

3.5 关系运算符

运算符	名称	示例	结果
==	等于	5 == 3	0（假）
!=	不等于	5 != 3	1（真）
>	大于	5 > 3	1（真）
<	小于	5 < 3	0（假）
>=	大于等于	5 >= 5	1（真）
<=	小于等于	5 <= 3	0（假）

注意区分

✖

判断相等用 ==

if (a == 5)

// 判断a是否等于5

✖

赋值用 =

a = 5;

// 把5赋值给a

⚠

新手常见错误

if (a = 5)

// 错误！这是赋值，不是判断

3.6 逻辑运算符

运算符	名称	说明
&&	与 (AND)	两个都真才为真
	或 (OR)	有一个真就为真
!	非 (NOT)	真变假，假变真

真值表

与运算 (&&)

A	B	A && B
真	真	真
真	假	假
假	真	假
假	假	假

或运算 (||)

A	B	A B
真	真	真
真	假	真
假	真	真
假	假	假

示例代码

```
#include <stdio.h>

int main()
{
    int age = 20;
    int score = 85;

    // 判断：年龄>=18 并且 分数>=60
    if (age >= 18 && score >= 60) {
        printf("符合条件\n");
    }

    // 判断：年龄<18 或者 分数<60
    if (age < 18 || score < 60) {
        printf("不符合条件\n");
    } else {
        printf("符合条件\n");
    }

    return 0;
}
```

3.7 运算符优先级

优先级从高到低

优先级	运算符
最高	() 小括号
高	! ++ --
中	* / %
中	+ -
低	< > <= >=
低	== !=
更低	&&
更低	\ \
最低	= += -= 等

建议

不确定优先级时，用小括号！

```
// 不确定
int result = a + b * c > d && e;

// 清晰明了
int result = ((a + (b * c)) > d) && e;
```

第三章小结

本章要点

1. 算术运算符：+、-、*、/、%
2. 整数除法会舍去小数
3. 自增自减：前置先加后用，后置先用后加
4. 关系运算符结果：真（1）或假（0）
5. 不确定优先级就用小括号

动手练习

练习1：计算两个数的和、差、积、商。

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 3;

    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d - %d = %d\n", a, b, a - b);
    printf("%d * %d = %d\n", a, b, a * b);
    printf("%d / %d = %d\n", a, b, a / b);
    printf("%d %% %d = %d\n", a, b, a % b);

    return 0;
}
```

练习2：判断一个数是奇数还是偶数。

```
#include <stdio.h>

int main()
{
    int num = 7;
```



```
if (num % 2 == 0) {
    printf("%d 是偶数\n", num);
} else {
    printf("%d 是奇数\n", num);
}

return 0;
}
```

第四章：流程控制（if-else、switch）

4.1 什么是流程控制？

一句话理解

流程控制就是**控制程序执行的顺序**，让程序能根据不同情况做不同的事情。

生活类比

如果下雨 → 带伞
如果不下雨 → 不带伞

这就是流程控制！

4.2 if语句

基本语法

```
if (条件) {
    // 条件为真时执行的代码
}
```

示例代码

```
#include <stdio.h>

int main()
{
    int age = 18;

    if (age >= 18) {
        printf("你已经成年了! \n");
    }

    return 0;
}
```

注意事项

```
// 花括号可以省略（只有一行代码时）
if (age >= 18)
    printf("成年\n");

// 但建议始终使用花括号，更清晰
if (age >= 18) {
    printf("成年\n");
}
```

4.3 if-else语句

基本语法

```
if (条件) {
    // 条件为真时执行
} else {
    // 条件为假时执行
}
```

示例代码

```
#include <stdio.h>

int main()
{
    int score = 55;

    if (score >= 60) {
        printf("及格了! \n");
    } else {
        printf("不及格，继续努力! \n");
    }

    return 0;
}
```

4.4 if-else if-else语句

基本语法

```
if (条件1) {
    // 条件1为真时执行
} else if (条件2) {
    // 条件2为真时执行
} else if (条件3) {
```

```
    // 条件3为真时执行
} else {
    // 所有条件都不满足时执行
}
```

示例：成绩等级判断

```
#include <stdio.h>

int main()
{
    int score = 85;

    if (score >= 90) {
        printf("等级: A 优秀\n");
    } else if (score >= 80) {
        printf("等级: B 良好\n");
    } else if (score >= 70) {
        printf("等级: C 中等\n");
    } else if (score >= 60) {
        printf("等级: D 及格\n");
    } else {
        printf("等级: E 不及格\n");
    }

    return 0;
}
```

执行流程

```
score = 85

score >= 90? → 否
    ↓
score >= 80? → 是 → 输出"等级: B 良好" → 结束
```

4.5 switch语句

基本语法

```
switch (变量) {
    case 值1:
        // 执行代码1
        break;
    case 值2:
        // 执行代码2
        break;
    default:
        // 以上都不匹配时执行
}
```

```
        break;
    }
}
```

示例：星期几

```
#include <stdio.h>

int main()
{
    int day = 3;

    switch (day) {
        case 1:
            printf("星期一\n");
            break;
        case 2:
            printf("星期二\n");
            break;
        case 3:
            printf("星期三\n");
            break;
        case 4:
            printf("星期四\n");
            break;
        case 5:
            printf("星期五\n");
            break;
        case 6:
            printf("星期六\n");
            break;
        case 7:
            printf("星期日\n");
            break;
        default:
            printf("输入错误\n");
            break;
    }

    return 0;
}
```

重要：break的作用

```
// 没有break的情况（会继续执行下去）
int day = 1;
switch (day) {
    case 1:
        printf("星期一\n");
        // 没有break! 会继续执行下面的case
    case 2:
        printf("星期二\n");
}
```

```
        break;
    }
    // 输出：星期一  星期二

    // 有break的情况（正确）
    int day = 1;
    switch (day) {
        case 1:
            printf("星期一\n");
            break; // 跳出switch
        case 2:
            printf("星期二\n");
            break;
    }
    // 输出：星期一
```

switch vs if-else

对比项	switch	if-else
判断条件	只能是整数或字符	任意条件
判断方式	精确匹配	范围判断
可读性	多分支时更清晰	灵活
适用场景	固定值的判断	范围、复杂条件

4.6 嵌套if语句

基本语法

```
if (条件1) {
    if (条件2) {
        // 条件1和条件2都为真时执行
    }
}
```

示例代码

```
#include <stdio.h>

int main()
{
    int age = 20;
    int score = 85;

    if (age >= 18) {
        if (score >= 60) {
            printf("成年人且及格\n");
        }
    }
}
```

```
        } else {
            printf("成年人但不及格\n");
        }
    } else {
        printf("未成年\n");
    }

    return 0;
}
```

第四章小结

本章要点

1. if语句：条件为真才执行
2. if-else：二选一
3. if-else if-else：多选一
4. switch：精确匹配多个值
5. break很重要，别忘了写

动手练习

练习1：判断一个数是正数、负数还是零。

```
#include <stdio.h>

int main()
{
    int num = -5;

    if (num > 0) {
        printf("%d 是正数\n", num);
    } else if (num < 0) {
        printf("%d 是负数\n", num);
    } else {
        printf("%d 是零\n", num);
    }

    return 0;
}
```

练习2：用switch实现简单计算器。

```
#include <stdio.h>

int main()
{
    int a = 10, b = 3;
    char op = '+';
```

```
switch (op) {
    case '+':
        printf("%d + %d = %d\n", a, b, a + b);
        break;
    case '-':
        printf("%d - %d = %d\n", a, b, a - b);
        break;
    case '*':
        printf("%d * %d = %d\n", a, b, a * b);
        break;
    case '/':
        printf("%d / %d = %d\n", a, b, a / b);
        break;
    default:
        printf("不支持的操作\n");
        break;
}

return 0;
}
```

第五章：循环（for、while）

5.1 什么是循环？

一句话理解

循环就是**重复执行某段代码**，直到满足退出条件。

生活类比

跑步：跑10圈
├─ 第1圈
├─ 第2圈
├─ ...
└─ 第10圈 → 结束

这就是循环！

为什么需要循环？

```
// 没有循环，重复写5次
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");

// 有循环，3行代码搞定
for (int i = 0; i < 5; i++) {
    printf("Hello\n");
}
```

5.2 for循环

基本语法

```
for (初始化; 条件; 更新) {
    // 循环体（重复执行的代码）
}
```

执行流程

```
初始化 → 判断条件 → 执行循环体 → 更新 → 判断条件 → ...
                ↑                               ↓
                ←-----
```

示例代码

```
#include <stdio.h>

int main()
{
    // 输出1到5
    for (int i = 1; i <= 5; i++) {
        printf("i = %d\n", i);
    }

    return 0;
}

// 输出：
// i = 1
// i = 2
// i = 3
// i = 4
// i = 5
```


逐行解释

```
for (int i = 1; i <= 5; i++)
```

↑	↑	↑
初始化	条件	更新

执行过程：

第1次: $i=1 \rightarrow i \leq 5?$ 是 $\rightarrow$ 输出1 $\rightarrow i++ \rightarrow i=2$

第2次: $i=2 \rightarrow i \leq 5?$ 是 $\rightarrow$ 输出2 $\rightarrow i++ \rightarrow i=3$

第3次: $i=3 \rightarrow i \leq 5?$ 是 $\rightarrow$ 输出3 $\rightarrow i++ \rightarrow i=4$

第4次: $i=4 \rightarrow i \leq 5?$ 是 $\rightarrow$ 输出4 $\rightarrow i++ \rightarrow i=5$

第5次: $i=5 \rightarrow i \leq 5?$ 是 $\rightarrow$ 输出5 $\rightarrow i++ \rightarrow i=6$

第6次: $i=6 \rightarrow i \leq 5?$ 否 $\rightarrow$ 退出循环

常见用法

// 1. 输出1到n

```
for (int i = 1; i <= n; i++) {  
    printf("%d ", i);  
}
```

// 2. 输出n到1

```
for (int i = n; i >= 1; i--) {  
    printf("%d ", i);  
}
```

// 3. 计算1到n的和

```
int sum = 0;  
for (int i = 1; i <= n; i++) {  
    sum += i;  
}
```

// 4. 输出偶数

```
for (int i = 0; i <= 10; i += 2) {  
    printf("%d ", i); // 输出: 0 2 4 6 8 10  
}
```

5.3 while循环

基本语法

```
while (条件) {  
    // 循环体  
}
```

执行流程



示例代码

```
#include <stdio.h>

int main()
{
    int i = 1;

    while (i <= 5) {
        printf("i = %d\n", i);
        i++;
    }

    return 0;
}
```

for vs while

对比项	for	while
适用场景	循环次数确定	循环次数不确定
初始化位置	在for语句中	在while之前
更新位置	在for语句中	在循环体中

什么时候用while?

```
// 当循环次数不确定时，用while更合适

// 示例：猜数字游戏
#include <stdio.h>

int main()
{
    int answer = 7; // 正确答案
    int guess;

    printf("猜一个1-10的数字：\n");

    while (1) { // 无限循环
        scanf("%d", &guess);

        if (guess == answer) {
```

```
        printf("猜对了! \n");
        break; // 退出循环
    } else if (guess < answer) {
        printf("太小了, 再猜: \n");
    } else {
        printf("太大了, 再猜: \n");
    }
}

return 0;
}
```

5.4 do-while循环

基本语法

```
do {
    // 循环体
} while (条件);
```

与while的区别

while: 先判断, 后执行 (可能一次都不执行)
do-while: 先执行, 后判断 (至少执行一次)

示例代码

```
#include <stdio.h>

int main()
{
    int i = 1;

    do {
        printf("i = %d\n", i);
        i++;
    } while (i <= 5);

    return 0;
}
```

典型应用：输入验证

```
#include <stdio.h>

int main()
{
    int num;
```

```

do {
    printf("请输入一个正数: ");
    scanf("%d", &num);

    if (num <= 0) {
        printf("输入错误, 请重新输入! \n");
    }
} while (num <= 0);

printf("你输入的正数是: %d\n", num);

return 0;
}

```

5.5 break和continue

break: 跳出循环

```

#include <stdio.h>

int main()
{
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            break; // 当i=5时, 跳出循环
        }
        printf("%d ", i);
    }
    // 输出: 1 2 3 4

    return 0;
}

```

continue: 跳过本次循环

```

#include <stdio.h>

int main()
{
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            continue; // 跳过i=3这次循环
        }
        printf("%d ", i);
    }
    // 输出: 1 2 4 5

    return 0;
}

```

对比

关键字	作用
<code>break</code>	跳出整个循环
<code>continue</code>	跳过本次循环，继续下一次

5.6 嵌套循环

基本概念

循环里面再套循环。

示例：乘法口诀表

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 9; i++) {        // 外层循环：行
        for (int j = 1; j <= i; j++) {    // 内层循环：列
            printf("%d×%d=%d\t", j, i, i*j);
        }
        printf("\n"); // 每行结束后换行
    }

    return 0;
}
```

输出：

```
1×1=1
1×2=2    2×2=4
1×3=3    2×3=6    3×3=9
...
```

示例：打印图形

```
#include <stdio.h>

int main()
{
    // 打印5行5列的星号
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            printf("* ");
        }
        printf("\n");
    }
}
```

```
}

return 0;

}
```

输出：

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

第五章小结

本章要点

1. for循环：次数确定时用
2. while循环：次数不确定时用
3. do-while：至少执行一次
4. break：跳出循环
5. continue：跳过本次
6. 嵌套循环：循环套循环

动手练习

练习1：计算1到100的和。

```
#include <stdio.h>

int main()
{
    int sum = 0;

    for (int i = 1; i <= 100; i++) {
        sum += i;
    }

    printf("1到100的和 = %d\n", sum); // 5050

    return 0;
}
```

练习2：输出九九乘法表。

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 9; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d\t", j, i, i*j);
        }
        printf("\n");
    }

    return 0;
}
```

练习3：找出1-100中所有的偶数。

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 100; i++) {
        if (i % 2 == 0) {
            printf("%d ", i);
        }
    }

    return 0;
}
```

第六章：数组（一维、二维、多维）

6.1 什么是数组？

一句话理解

数组是一组相同类型的数据，用同一个名字管理，通过下标访问每个元素。

生活类比

一维数组 = 一排储物柜

0	1	2	3	4	← 下标（从0开始）
10	20	30	40	50	← 数据

scores[0] = 10

scores[2] = 30

为什么需要数组？

```
// 没有数组：存储5个成绩，要定义5个变量
int score1 = 90;
int score2 = 85;
int score3 = 78;
int score4 = 92;
int score5 = 88;

// 有数组：一行搞定
int scores[5] = {90, 85, 78, 92, 88};
```

6.2 一维数组

定义语法

数据类型 数组名[数组大小];

几种定义方式

```
// 方式1：先定义，后赋值
int scores[5];
scores[0] = 90;
scores[1] = 85;

// 方式2：定义时初始化
int scores[5] = {90, 85, 78, 92, 88};

// 方式3：省略大小（自动计算）
int scores[] = {90, 85, 78, 92, 88}; // 大小为5

// 方式4：部分初始化（其余为0）
int scores[5] = {90, 85}; // 等价于 {90, 85, 0, 0, 0}

// 方式5：全部初始化为0
int scores[5] = {0}; // {0, 0, 0, 0, 0}
```

⚠ 重要：下标从0开始

数组下标范围：0 ~ n-1

int arr[5] 的下标是 0, 1, 2, 3, 4

✗ 错误：arr[5] 越界！

6.3 二维数组

生活类比：电影院座位

二维数组 = 电影院座位表

	列0	列1	列2	列3
行0	[A1]	[A2]	[A3]	[A4]
行1	[B1]	[B2]	[B3]	[B4]
行2	[C1]	[C2]	[C3]	[C4]

`seats[1][2] = B3座位`
`seats[2][0] = C1座位`

定义和初始化

```
// 方式1: 完全初始化
int arr[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

// 方式2: 省略内层花括号
int arr[2][3] = {1, 2, 3, 4, 5, 6};

// 方式3: 部分初始化
int arr[2][3] = {
    {1, 2},           // 第一行: {1, 2, 0}
    {4}              // 第二行: {4, 0, 0}
};
```

实际应用：学生成绩表

```
#include <stdio.h>

int main()
{
    // 3个学生, 4门课程的成绩
    int scores[3][4] = {
        {90, 85, 78, 92}, // 学生1
        {80, 75, 88, 85}, // 学生2
        {95, 90, 92, 98}  // 学生3
    };

    // 输出每个学生的成绩
    for (int i = 0; i < 3; i++) {
        printf("学生%d的成绩: ", i + 1);
        for (int j = 0; j < 4; j++) {
            printf("%d ", scores[i][j]);
        }
    }
}
```

```

        printf("\n");
    }

    // 计算每个学生的平均分
    for (int i = 0; i < 3; i++) {
        int sum = 0;
        for (int j = 0; j < 4; j++) {
            sum += scores[i][j];
        }
        printf("学生%d的平均分: %.1f\n", i + 1, (float)sum / 4);
    }

    return 0;
}

```

实际应用：图像处理

一张图片可以看作二维数组

每个元素存储一个像素点的颜色值

	列0	列1	列2	列3	列4
行0	[255]	[128]	[0]	[0]	[0]
行1	[255]	[255]	[128]	[0]	[0]
行2	[255]	[255]	[255]	[128]	[0]

这在图像处理、游戏开发中经常用到！

6.4 多维数组

生活类比：教学楼

三维数组 = 教学楼的教室

教学楼[楼层][走廊][教室]

teaching_building[2][3][5] = 2楼、3号走廊、5号教室

定义和使用

```

#include <stdio.h>

int main()
{
    // 2层楼，每层3个走廊，每个走廊4个教室
    int building[2][3][4];

    // 初始化
    int room_number = 101;
}

```

```
for (int floor = 0; floor < 2; floor++) {
    for (int corridor = 0; corridor < 3; corridor++) {
        for (int room = 0; room < 4; room++) {
            building[floor][corridor][room] = room_number++;
        }
    }
}

// 输出
for (int floor = 0; floor < 2; floor++) {
    printf("=== 第%d层 ===\n", floor + 1);
    for (int corridor = 0; corridor < 3; corridor++) {
        printf("走廊%d: ", corridor + 1);
        for (int room = 0; room < 4; room++) {
            printf("%d ", building[floor][corridor][room]);
        }
        printf("\n");
    }
}

return 0;
}
```

实际应用场景

维度	应用场景
一维数组	存储一系列数据（成绩、温度记录）
二维数组	表格数据、图像、棋盘、地图
三维数组	RGB图像、视频帧、3D游戏地图
更高维度	科学计算、机器学习数据

6.5 数组常见操作

求和与平均值

```
#include <stdio.h>

int main()
{
    int scores[] = {90, 85, 78, 92, 88};
    int len = sizeof(scores) / sizeof(scores[0]);

    int sum = 0;
    for (int i = 0; i < len; i++) {
        sum += scores[i];
    }
}
```

```

float avg = (float)sum / len;

printf("总分: %d\n", sum);
printf("平均分: %.2f\n", avg);

return 0;
}

```

找最大值和最小值

```

#include <stdio.h>

int main()
{
    int scores[] = {90, 85, 78, 92, 88};
    int len = sizeof(scores) / sizeof(scores[0]);

    int max = scores[0];
    int min = scores[0];

    for (int i = 1; i < len; i++) {
        if (scores[i] > max) max = scores[i];
        if (scores[i] < min) min = scores[i];
    }

    printf("最高分: %d\n", max);
    printf("最低分: %d\n", min);

    return 0;
}

```

6.6 数组在项目中的应用

应用1：通讯录

```

#include <stdio.h>
#include <string.h>

int main()
{
    // 用二维数组存储多个名字
    char names[5][20] = {
        "张三",
        "李四",
        "王五",
        "赵六",
        "钱七"
    };

    // 用一维数组存储对应的电话
}

```

```

char phones[5][12] = {
    "13800000001",
    "13800000002",
    "13800000003",
    "13800000004",
    "13800000005"
};

// 搜索功能
char search[20];
printf("请输入要查找的名字: ");
scanf("%s", search);

for (int i = 0; i < 5; i++) {
    if (strcmp(names[i], search) == 0) {
        printf("找到! %s 的电话是 %s\n", names[i], phones[i]);
        break;
    }
}

return 0;
}

```

应用2：简单的游戏地图

```

#include <stdio.h>

int main()
{
    // 0表示空地, 1表示墙, 2表示玩家, 3表示宝藏
    int map[5][5] = {
        {1, 1, 1, 1, 1},
        {1, 2, 0, 0, 1},
        {1, 0, 1, 0, 1},
        {1, 0, 0, 3, 1},
        {1, 1, 1, 1, 1}
    };

    // 绘制地图
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            switch (map[i][j]) {
                case 0: printf(" "); break; // 空地
                case 1: printf("■ "); break; // 墙
                case 2: printf("♀ "); break; // 玩家
                case 3: printf("★ "); break; // 宝藏
            }
        }
        printf("\n");
    }

    return 0;
}

```

```
}
```

第六章小结

本章要点

- 一维数组：存储一系列数据
- 二维数组：存储表格数据（行和列）
- 多维数组：存储更复杂的数据结构
- 下标从0开始，注意越界问题
- 数组在项目中应用广泛

项目中的实际应用

项目类型	数组应用
游戏开发	地图、角色属性、物品栏
图像处理	像素数据、滤镜
数据分析	统计数据、图表
嵌入式	传感器数据缓冲、波形存储

动手练习

练习1：用二维数组实现井字棋棋盘。

```
#include <stdio.h>

int main()
{
    char board[3][3] = {
        {' ', ' ', ' '},
        {' ', ' ', ' '},
        {' ', ' ', ' '}
    };

    // 玩家1下棋
    board[0][0] = 'X';
    // 玩家2下棋
    board[1][1] = 'O';

    // 打印棋盘
    for (int i = 0; i < 3; i++) {
        printf(" %c | %c | %c \n", board[i][0], board[i][1], board[i][2]);
        if (i < 2) printf("---|---|---\n");
    }
}
```

```
    return 0;
}
```

练习2：用三维数组存储一个3x3的RGB图像数据。

第七章：函数

7.1 什么是函数？

一句话理解

函数是一段可重复使用的代码块，用来完成特定任务。

生活类比

函数 = 洗衣机

- └─ 你只需要：把衣服放进去，按开关
- └─ 洗衣机：完成洗衣、脱水等一系列操作
- └─ 你得到：干净的衣服

你不需要知道洗衣机内部怎么工作，
只需要知道：输入什么，输出什么。

为什么需要函数？

```
// 没有函数：重复写相同的代码
printf("1 + 2 = %d\n", 1 + 2);
printf("3 + 4 = %d\n", 3 + 4);
printf("5 + 6 = %d\n", 5 + 6);

// 有函数：定义一次，多次使用
int add(int a, int b) {
    return a + b;
}

printf("%d + %d = %d\n", 1, 2, add(1, 2));
printf("%d + %d = %d\n", 3, 4, add(3, 4));
printf("%d + %d = %d\n", 5, 6, add(5, 6));
```

7.2 函数的定义

基本语法

```
返回类型 函数名(参数列表) {  
    // 函数体  
    return 返回值;  
}
```

各部分解释

部分	说明
返回类型	函数返回值的数据类型 (int、float、void等)
函数名	函数的名称，调用时使用
参数列表	函数需要的输入，可以为空
函数体	函数要执行的代码
return	返回结果，void类型不需要

示例：加法函数

```
// 定义一个加法函数  
int add(int a, int b) {  
    return a + b;  
}
```

7.3 函数的调用

基本用法

```
#include <stdio.h>  
  
// 函数定义  
int add(int a, int b) {  
    return a + b;  
}  
  
int main()  
{  
    // 函数调用  
    int result = add(3, 5);  
    printf("3 + 5 = %d\n", result);  
  
    // 也可以直接在表达式中使用  
    printf("10 + 20 = %d\n", add(10, 20));  
}
```



```
    return 0;
}
```

调用流程

1. 程序从main开始执行
2. 遇到add(3, 5)，跳转到add函数
3. 将3赋值给a，5赋值给b
4. 执行a + b，得到8
5. return 8，返回调用处
6. result = 8
7. 继续执行后续代码

7.4 函数的类型

无参数无返回值

```
#include <stdio.h>

// 打印分隔线
void printLine() {
    printf("-----\n");
}

int main()
{
    printLine();
    printf("Hello, world!\n");
    printLine();

    return 0;
}
```

有参数无返回值

```
#include <stdio.h>

// 打印n个星号
void printStars(int n) {
    for (int i = 0; i < n; i++) {
        printf("* ");
    }
    printf("\n");
}

int main()
{
    printStars(5);    // * * * * *
```

```
    printStars(10); // * * * * * * * * * *  
  
    return 0;  
}
```

无参数有返回值

```
#include <stdio.h>  
  
// 获取用户输入的数字  
int getInput() {  
    int num;  
    printf("请输入一个数字: ");  
    scanf("%d", &num);  
    return num;  
}  
  
int main()  
{  
    int n = getInput();  
    printf("你输入的是: %d\n", n);  
  
    return 0;  
}
```

有参数有返回值

```
#include <stdio.h>  
  
// 计算两数中的最大值  
int max(int a, int b) {  
    if (a > b) {  
        return a;  
    } else {  
        return b;  
    }  
}  
  
int main()  
{  
    printf("max(3, 5) = %d\n", max(3, 5)); // 5  
    printf("max(10, 8) = %d\n", max(10, 8)); // 10  
  
    return 0;  
}
```

7.5 函数声明

问题：函数定义在main后面

```
#include <stdio.h>

int main()
{
    int result = add(3, 5); // 错误！编译器不知道add是什么
    printf("%d\n", result);
    return 0;
}

int add(int a, int b) {
    return a + b;
}
```

解决方案：函数声明

```
#include <stdio.h>

// 函数声明（告诉编译器有这个函数）
int add(int a, int b);

int main()
{
    int result = add(3, 5); // 正确！
    printf("%d\n", result);
    return 0;
}

// 函数定义
int add(int a, int b) {
    return a + b;
}
```

声明 vs 定义

概念	说明
函数声明	告诉编译器函数的存在，格式： <code>返回类型 函数名(参数列表);</code>
函数定义	函数的具体实现

7.6 函数示例

示例1：判断素数

```
#include <stdio.h>

// 判断是否为素数
int isPrime(int n) {
    if (n <= 1) return 0;

    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0; // 不是素数
        }
    }
    return 1; // 是素数
}

int main()
{
    int num = 17;

    if (isPrime(num)) {
        printf("%d 是素数\n", num);
    } else {
        printf("%d 不是素数\n", num);
    }

    return 0;
}
```

示例2：计算阶乘

```
#include <stdio.h>

// 计算n的阶乘
long factorial(int n) {
    if (n <= 1) return 1;

    long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main()
{
    printf("5! = %ld\n", factorial(5)); // 120
    printf("10! = %ld\n", factorial(10)); // 3628800
}
```

```
    return 0;
}
```

示例3：交换两个数

```
#include <stdio.h>

// 错误示范：值传递，不会改变原值
void swapWrong(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

// 正确示范：指针传递
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    int x = 10, y = 20;

    printf("交换前: x=%d, y=%d\n", x, y);

    swap(&x, &y); // 传递地址

    printf("交换后: x=%d, y=%d\n", x, y);

    return 0;
}
```

7.7 变量的作用域

局部变量

```
#include <stdio.h>

void test() {
    int a = 10; // 局部变量，只在test函数内有效
    printf("test: a = %d\n", a);
}

int main()
{
    test();
    // printf("%d", a); // 错误! a在这里不存在
}
```

```
    return 0;
}
```

全局变量

```
#include <stdio.h>

int global = 100; // 全局变量，所有函数都能访问

void test() {
    printf("test: global = %d\n", global);
    global = 200; // 修改全局变量
}

int main()
{
    printf("main: global = %d\n", global); // 100
    test();
    printf("main: global = %d\n", global); // 200

    return 0;
}
```

对比

类型	作用范围	生命周期
局部变量	定义它的函数内	函数调用时创建，函数结束时销毁
全局变量	整个程序	程序开始时创建，程序结束时销毁

第七章小结

本章要点

- 1. 函数是可重复使用的代码块
- 2. 函数有参数和返回值
- 3. 函数声明告诉编译器函数存在
- 4. 局部变量只在函数内有效
- 5. 好的函数设计让代码更清晰

动手练习

练习1：写一个函数，计算圆的面积。

```

#include <stdio.h>

float circleArea(float radius) {
    return 3.14159 * radius * radius;
}

int main()
{
    float r = 5.0;
    printf("半径 %.2f 的圆面积 = %.2f\n", r, circleArea(r));

    return 0;
}

```

练习2: 写一个函数，判断年份是否为闰年。

```

#include <stdio.h>

// 闰年规则：
// 1. 能被4整除但不能被100整除，或者
// 2. 能被400整除
int isLeapYear(int year) {
    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
        return 1;
    }
    return 0;
}

int main()
{
    int year = 2024;

    if (isLeapYear(year)) {
        printf("%d 是闰年\n", year);
    } else {
        printf("%d 不是闰年\n", year);
    }

    return 0;
}

```

第八章：指针（C语言的灵魂）

8.1 什么是指针？

生活类比：快递地址

变量 = 一个包裹
变量的值 = 包裹里的东西
指针 = 包裹的存放地址

知道地址 → 就能找到包裹 → 就能拿到东西

指针就是"地址"，通过地址可以找到变量

一句话理解

指针是一个**存储地址的变量**，它指向另一个变量在内存中的位置。

内存示意图：

地址	变量名	值
0x1000	a	10
0x1004	p	0x1000 ← p存储的是a的地址

我们说：p指向a

8.2 指针的定义和使用

基本语法

```
int *p;           // 定义一个指向int的指针
int a = 10;
p = &a;           // p指向a（&是取地址符）
```

两个关键符号

符号	名称	作用
&	取地址符	获取变量的地址
*	解引用符	获取指针指向的值

示例代码

```
#include <stdio.h>

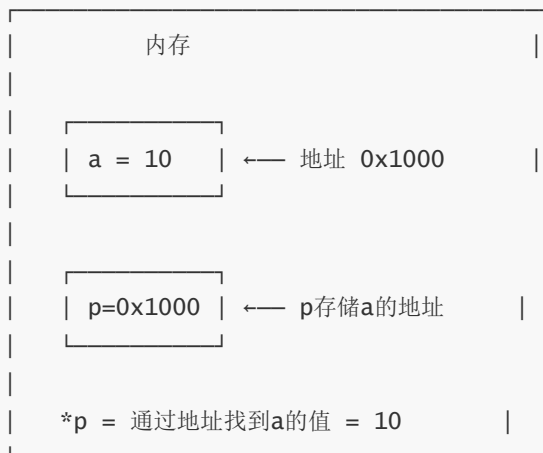
int main()
{
    int a = 10;
    int *p = &a;    // p指向a

    printf("a的值: %d\n", a);        // 10
    printf("a的地址: %p\n", &a);    // 类似0x7fff5bfff8ac
    printf("p的值: %p\n", p);        // 和&a相同
    printf("p指向的值: %d\n", *p);   // 10

    // 通过指针修改a的值
    *p = 20;
    printf("修改后a的值: %d\n", a);  // 20

    return 0;
}
```

图解



8.3 指针的本质

为什么需要指针？

场景1：函数修改外部变量

- └ 普通变量：函数内修改，外面不变
- └ 指针：传递地址，可以修改原值

场景2：动态内存分配

- └ 运行时才知道需要多少内存
- └ 必须用指针来管理

场景3：高效传递大数据

└─ 传递整个数组？太慢

└─ 传递数组首地址？高效

指针的大小

```
#include <stdio.h>

int main()
{
    int a = 10;
    char c = 'A';
    double d = 3.14;

    int *pi = &a;
    char *pc = &c;
    double *pd = &d;

    // 指针的大小和类型无关，都是地址
    printf("int指针大小: %zu\n", sizeof(pi));    // 8字节(64位系统)
    printf("char指针大小: %zu\n", sizeof(pc));    // 8字节
    printf("double指针大小: %zu\n", sizeof(pd));    // 8字节

    return 0;
}
```

8.4 指针与数组

数组名就是地址

```
#include <stdio.h>

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};

    // 数组名就是首元素地址
    printf("arr的地址: %p\n", arr);
    printf("arr[0]的地址: %p\n", &arr[0]);    // 相同!

    // 用指针访问数组
    int *p = arr;
    for (int i = 0; i < 5; i++) {
        printf("arr[%d] = %d\n", i, *(p + i));
    }

    return 0;
}
```

指针算术运算

```
#include <stdio.h>

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;

    // p+1 不是地址+1, 而是地址+sizeof(int)
    printf("p的地址: %p\n", p);        // 0x1000 (假设)
    printf("p+1的地址: %p\n", p+1);    // 0x1004 (加了4字节)

    // 等价关系
    // arr[i] 等价于 *(arr + i) 等价于 *(p + i) 等价于 p[i]

    return 0;
}
```

生活类比：街道门牌号

数组 = 一条街上的房子

arr[0] → 1号房子

arr[1] → 2号房子

arr[2] → 3号房子

p = arr → p指向1号房子

p + 1 → 指向下一栋房子（2号）

p + 2 → 指向下下栋房子（3号）

*p → 取出当前房子的内容

8.5 指针与字符串

字符串的本质

```
#include <stdio.h>

int main()
{
    // 方式1: 字符数组
    char str1[] = "Hello";

    // 方式2: 字符指针
    char *str2 = "world";

    printf("%s\n", str1); // Hello
    printf("%s\n", str2); // world
}
```

```

// 区别:
// str1可以修改内容
str1[0] = 'h'; // OK

// str2指向常量, 不建议修改
// str2[0] = 'w'; // 危险! 可能崩溃

return 0;
}

```

字符串操作

```

#include <stdio.h>

// 自定义字符串长度函数
int myStrlen(char *str) {
    int len = 0;
    while (*str != '\0') {
        len++;
        str++;
    }
    return len;
}

int main()
{
    char *str = "Hello world";
    printf("字符串长度: %d\n", myStrlen(str)); // 11

    return 0;
}

```

8.6 指针在项目中的应用

应用1: 交换两个变量

```

#include <stdio.h>

// 错误: 值传递, 不会改变原值
void swapWrong(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

// 正确: 指针传递
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
}

```

```

        *b = temp;
    }

    int main()
    {
        int x = 10, y = 20;

        printf("交换前: x=%d, y=%d\n", x, y);
        swap(&x, &y);
        printf("交换后: x=%d, y=%d\n", x, y);

        return 0;
    }

```

应用2：返回多个值

```

#include <stdio.h>

// 通过指针返回多个值
void getMinMax(int arr[], int len, int *min, int *max) {
    *min = arr[0];
    *max = arr[0];

    for (int i = 1; i < len; i++) {
        if (arr[i] < *min) *min = arr[i];
        if (arr[i] > *max) *max = arr[i];
    }
}

int main()
{
    int arr[] = {30, 10, 50, 20, 40};
    int min, max;

    getMinMax(arr, 5, &min, &max);

    printf("最小值: %d\n", min); // 10
    printf("最大值: %d\n", max); // 50

    return 0;
}

```

应用3：嵌入式开发中操作寄存器

```

// STM32中操作GPIO寄存器
#define GPIOA_BASE 0x40010800
#define GPIOA      ((GPIO_TypeDef *)GPIOA_BASE)

// 通过指针直接操作硬件
GPIOA->ODR = 0x01; // 设置PA0为高电平

```

第八章小结

本章要点

1. 指针是存储地址的变量
2. &取地址，*解引用
3. 指针可以修改函数外的变量
4. 数组名就是首元素地址
5. 指针在嵌入式、系统开发中非常重要

动手练习

练习1：用指针遍历数组。

```
#include <stdio.h>

int main()
{
    int arr[] = {1, 2, 3, 4, 5};
    int *p = arr;

    for (int i = 0; i < 5; i++) {
        printf("%d ", *(p + i));
    }

    return 0;
}
```

练习2：用指针实现字符串复制。

```
#include <stdio.h>

void myStrcpy(char *dest, char *src) {
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0';
}

int main()
{
    char src[] = "Hello";
    char dest[20];

    myStrcpy(dest, src);
    printf("复制结果: %s\n", dest);

    return 0;
}
```

```
}
```

第九章：指针与函数

9.1 函数指针

生活类比：遥控器

函数 = 电视机
函数指针 = 遥控器

遥控器可以控制不同的电视
函数指针可以指向不同的函数

按遥控器按键 → 电视执行对应操作
调用函数指针 → 执行指向的函数

什么是函数指针？

函数指针是指向函数的指针，可以通过它调用不同的函数。

返回类型 (*指针名)(参数列表);

基本用法

```
#include <stdio.h>

// 定义几个函数
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

int main()
{
    // 定义函数指针
    int (*operation)(int, int);

    // 指向add函数
    operation = add;
    printf("10 + 5 = %d\n", operation(10, 5)); // 15

    // 指向sub函数
    operation = sub;
    printf("10 - 5 = %d\n", operation(10, 5)); // 5

    // 指向mul函数
    operation = mul;
    printf("10 * 5 = %d\n", operation(10, 5)); // 50
}
```

```
    return 0;
}
```

图解

内存	
add函数代码	← 地址 0x1000
sub函数代码	← 地址 0x2000
mul函数代码	← 地址 0x3000
operation = add	→ operation = 0x1000
operation()	→ 执行0x1000处的代码

9.2 函数指针的实际应用

应用1：回调函数

```
#include <stdio.h>
#include <stdlib.h>

// 回调函数类型
typedef int (*CompareFunc)(int, int);

// 升序比较
int ascending(int a, int b) {
    return a > b;
}

// 降序比较
int descending(int a, int b) {
    return a < b;
}

// 冒泡排序，使用函数指针决定排序方式
void bubbleSort(int arr[], int len, CompareFunc compare) {
    for (int i = 0; i < len - 1; i++) {
        for (int j = 0; j < len - 1 - i; j++) {
            if (compare(arr[j], arr[j + 1])) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main()
```



```

{
    int arr[] = {5, 2, 8, 1, 9};
    int len = 5;

    // 升序排序
    bubbleSort(arr, len, ascending);
    printf("升序: ");
    for (int i = 0; i < len; i++) printf("%d ", arr[i]);
    printf("\n");

    // 降序排序
    bubbleSort(arr, len, descending);
    printf("降序: ");
    for (int i = 0; i < len; i++) printf("%d ", arr[i]);
    printf("\n");

    return 0;
}

```

应用2：菜单系统

```

#include <stdio.h>

void menuNew() { printf("新建文件\n"); }
void menuOpen() { printf("打开文件\n"); }
void menuSave() { printf("保存文件\n"); }
void menuExit() { printf("退出程序\n"); }

int main()
{
    // 函数指针数组
    void (*menu[])() = {menuNew, menuOpen, menuSave, menuExit};
    char *menuNames[] = {"新建", "打开", "保存", "退出"};

    int choice;

    while (1) {
        printf("\n===== 菜单 =====\n");
        for (int i = 0; i < 4; i++) {
            printf("%d. %s\n", i + 1, menuNames[i]);
        }
        printf("请选择: ");
        scanf("%d", &choice);

        if (choice >= 1 && choice <= 4) {
            menu[choice - 1](); // 调用对应函数
            if (choice == 4) break;
        }
    }

    return 0;
}

```

应用3：状态机（嵌入式常用）

```
#include <stdio.h>

// 状态枚举
typedef enum {
    STATE_IDLE,
    STATE_RUNNING,
    STATE_PAUSED,
    STATE_STOPPED
} State;

// 状态处理函数
void handleIdle() { printf("系统空闲，等待启动...\n"); }
void handleRunning() { printf("系统运行中...\n"); }
void handlePaused() { printf("系统已暂停\n"); }
void handleStopped() { printf("系统已停止\n"); }

int main()
{
    // 状态处理函数表
    void (*stateHandlers[])() = {
        handleIdle,
        handleRunning,
        handlePaused,
        handleStopped
    };

    State currentState = STATE_IDLE;

    // 模拟状态机运行
    stateHandlers[currentState]() ; // 输出：系统空闲

    currentState = STATE_RUNNING;
    stateHandlers[currentState]() ; // 输出：系统运行中

    return 0;
}
```

9.3 指针函数

什么是指针函数？

指针函数是返回指针的函数。

返回类型* 函数名(参数列表);

区别对比

概念	定义	本质
函数指针	<code>int (*p)(int)</code>	指向函数的指针
指针函数	<code>int* func(int)</code>	返回指针的函数

记忆技巧

看括号的位置：

`int (*p)(int);`

// (*p) 在括号里 → p是指针 → 函数指针

`int* func(int);`

// 没有括号包住* → 返回int* → 指针函数

指针函数示例

```
#include <stdio.h>
#include <stdlib.h>

// 返回动态分配的数组指针
int* createArray(int size) {
    int *arr = (int*)malloc(size * sizeof(int));
    return arr; // 返回指针
}

// 返回数组中的最大值指针
int* findMax(int arr[], int len) {
    int *max = &arr[0];
    for (int i = 1; i < len; i++) {
        if (arr[i] > *max) {
            max = &arr[i];
        }
    }
    return max; // 返回最大值的地址
}

int main()
{
    // 使用指针函数创建数组
    int *arr = createArray(5);
    for (int i = 0; i < 5; i++) {
        arr[i] = i * 10;
    }

    printf("数组: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
// 使用指针函数找最大值
int *max = findMax(arr, 5);
printf("最大值: %d\n", *max);

free(arr); // 释放内存
return 0;
}
```

9.4 项目中的实际应用

应用场景

技术	应用场景
函数指针	回调函数、事件处理、插件系统、状态机
指针函数	动态内存分配、返回数组、链表创建

嵌入式开发中的应用

```
// STM32中断向量表就是函数指针数组
typedef void (*IRQHandler)(void);

IRQHandler irqTable[] = {
    IRQ0_Handler,
    IRQ1_Handler,
    // ...
};

// 事件驱动框架
typedef void (*EventHandler)(void* data);

struct Event {
    int type;
    void* data;
    EventHandler handler;
};
```

第九章小结

本章要点

- 1. 函数指针：指向函数的指针，可以动态调用不同函数
- 2. 指针函数：返回指针的函数
- 3. 函数指针常用于回调、菜单系统、状态机
- 4. 看括号位置区分两者

对比总结

```
// 函数指针
int (*fp)(int, int);    // fp是一个指针，指向函数
fp = add;               // 指向add函数
result = fp(1, 2);      // 调用

// 指针函数
int* func(int size);    // func是一个函数，返回int*
int *p = func(10);      // 调用，得到指针
```

动手练习

练习：用函数指针实现一个简单的计算器。

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }
int div(int a, int b) { return b != 0 ? a / b : 0; }

int main()
{
    int (*calc[])(int, int) = {add, sub, mul, div};
    char *ops = "+-*/";

    int a = 10, b = 5, op;

    printf("选择操作(0:+ 1:- 2:* 3:/): ");
    scanf("%d", &op);

    if (op >= 0 && op < 4) {
        printf("%d %c %d = %d\n", a, ops[op], b, calc[op](a, b));
    }

    return 0;
}
```

第十章：链表

10.1 为什么需要链表？

数组的局限性

数组的缺点：

- 大小固定，不能动态扩展
- 插入/删除元素需要移动大量数据
- 内存必须连续

例如：

数组 [A][B][C][D][E]

要在B后面插入X：

[A][B][X][C][D][E] ← C、D、E都要往后移！

链表的优势

链表的优点：

- └— 大小动态，随时增加删除
- └— 插入/删除只需修改指针
- └— 内存不需要连续

链表：A → B → C → D → E

要在B后面插入X：

A → B → X → C → D → E

只需改两个指针！

生活类比：寻宝游戏

链表 = 寻宝游戏

每个线索指向下一个线索的位置

线索1: "去大树下"

↓

线索2: "去喷泉边"

↓

线索3: "去图书馆"

↓

宝藏！

你不需要知道所有线索的位置

只需要跟着线索一步步找到宝藏

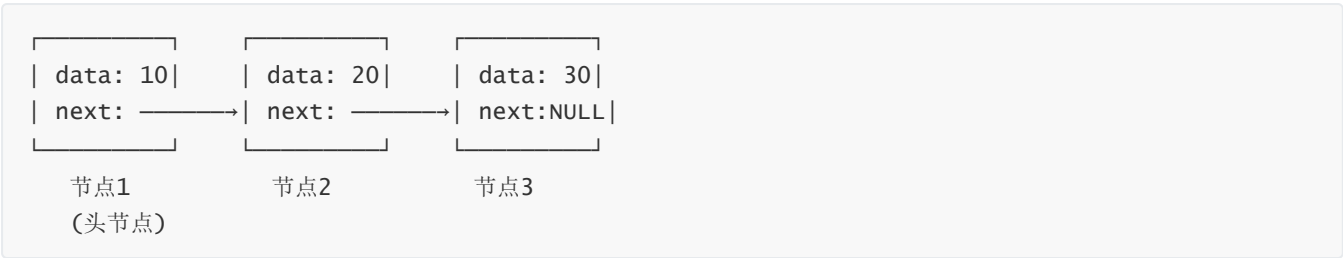
10.2 链表节点定义

节点结构

```
// 链表节点
struct Node {
    int data;           // 数据
    struct Node *next;  // 指向下一个节点的指针
};

typedef struct Node Node;
```

图解



10.3 单向链表基本操作

创建节点

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 创建新节点
Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

插入节点

```
// 在头部插入
Node* insertHead(Node *head, int data) {
    Node *newNode = createNode(data);
    newNode->next = head;
    return newNode; // 新节点成为头节点
}

// 在尾部插入
Node* insertTail(Node *head, int data) {
    Node *newNode = createNode(data);

    if (head == NULL) {
        return newNode; // 空链表，新节点成为头节点
    }

    Node *current = head;
    while (current->next != NULL) {
        current = current->next;
    }
}
```

```

    }
    current->next = newNode;
    return head;
}

```

遍历链表

```

// 打印链表
void printList(Node *head) {
    Node *current = head;
    printf("链表: ");
    while (current != NULL) {
        printf("%d → ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

// 计算链表长度
int getLength(Node *head) {
    int len = 0;
    Node *current = head;
    while (current != NULL) {
        len++;
        current = current->next;
    }
    return len;
}

```

删除节点

```

// 删除指定值的节点
Node* deleteNode(Node *head, int value) {
    if (head == NULL) return NULL;

    // 删除头节点
    if (head->data == value) {
        Node *temp = head;
        head = head->next;
        free(temp);
        return head;
    }

    // 删除中间或尾部节点
    Node *current = head;
    while (current->next != NULL) {
        if (current->next->data == value) {
            Node *temp = current->next;
            current->next = temp->next;
            free(temp);
            return head;
        }
        current = current->next;
    }
    return head;
}

```



```

    }
    current = current->next;
}

printf("未找到值为%d的节点\n", value);
return head;
}

```

完整示例

```

int main()
{
    Node *head = NULL;

    // 插入节点
    head = insertTail(head, 10);
    head = insertTail(head, 20);
    head = insertTail(head, 30);
    head = insertHead(head, 5);

    printList(head); // 5 → 10 → 20 → 30 → NULL

    printf("链表长度: %d\n", getLength(head)); // 4

    // 删除节点
    head = deleteNode(head, 20);
    printList(head); // 5 → 10 → 30 → NULL

    // 释放内存
    while (head != NULL) {
        Node *temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}

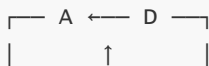
```

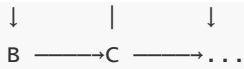
10.4 循环链表

生活类比：圆桌会议

循环链表 = 圆桌会议

每个人指向下一个人
 最后一个人指向第一个人
 形成闭环





没有起点终点，可以从任何位置开始

循环链表定义

```
// 循环链表：最后一个节点指向头节点
// 判断结束：current->next == head（而不是NULL）
```

循环链表操作

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 创建循环链表
Node* createCircularList(int n) {
    if (n <= 0) return NULL;

    Node *head = createNode(1);
    Node *current = head;

    for (int i = 2; i <= n; i++) {
        current->next = createNode(i);
        current = current->next;
    }

    current->next = head; // 尾节点指向头节点，形成循环
    return head;
}

// 遍历循环链表
void printCircularList(Node *head) {
    if (head == NULL) return;

    Node *current = head;
    printf("循环链表: ");

    do {
        printf("%d → ", current->data);
        current = current->next;
    } while (current != head); // 回到起点时停止

    printf("(回到起点)\n");
}
```

```

// 经典应用：约瑟夫问题
int josephus(int n, int k) {
    // 创建n个人的循环链表
    Node *head = createNode(1);
    Node *current = head;

    for (int i = 2; i <= n; i++) {
        current->next = createNode(i);
        current = current->next;
    }
    current->next = head; // 形成循环

    // 开始报数
    Node *prev = current; // 指向最后一个节点
    current = head;

    while (current->next != current) {
        // 数k-1个人
        for (int i = 1; i < k; i++) {
            prev = current;
            current = current->next;
        }

        // 删除第k个人
        printf("淘汰: %d\n", current->data);
        prev->next = current->next;
        Node *temp = current;
        current = current->next;
        free(temp);
    }

    int survivor = current->data;
    free(current);
    return survivor;
}

int main()
{
    // 演示循环链表
    Node *head = createCircularList(5);
    printCircularList(head);

    // 约瑟夫问题：10个人，每3个人淘汰一个
    printf("\n约瑟夫问题(10人，数到3淘汰): \n");
    int survivor = josephus(10, 3);
    printf("最后幸存者: %d\n", survivor);

    return 0;
}

```

10.5 链表在项目中的应用

应用1：实现栈

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 栈：后进先出(LIFO)
typedef struct {
    Node *top;
} Stack;

void push(Stack *s, int data) {
    Node *newNode = createNode(data);
    newNode->next = s->top;
    s->top = newNode;
}

int pop(Stack *s) {
    if (s->top == NULL) {
        printf("栈空! \n");
        return -1;
    }
    Node *temp = s->top;
    int data = temp->data;
    s->top = temp->next;
    free(temp);
    return data;
}

// 应用：撤销功能、函数调用栈、表达式求值
```

应用2：实现队列

```
// 队列：先进先出(FIFO)
typedef struct {
    Node *front; // 队头
    Node *rear;  // 队尾
} Queue;

void enqueue(Queue *q, int data) {
    Node *newNode = createNode(data);
    if (q->rear == NULL) {
        q->front = q->rear = newNode;
    } else {
        q->rear->next = newNode;
    }
}
```

```
        q->rear = newNode;
    }
}

int dequeue(Queue *q) {
    if (q->front == NULL) {
        printf("队列空! \n");
        return -1;
    }
    Node *temp = q->front;
    int data = temp->data;
    q->front = temp->next;
    if (q->front == NULL) {
        q->rear = NULL;
    }
    free(temp);
    return data;
}

// 应用：任务调度、消息队列、缓冲区
```

应用3：内存管理

操作系统中的内存管理：

空闲内存块用链表连接



申请内存时：从链表中找到合适的块
释放内存时：将块重新加入链表

实际项目中的应用

应用	链表类型	说明
操作系统进程调度	循环链表	进程轮转调度
浏览器历史记录	双向链表	前进/后退功能
音乐播放列表	循环链表	列表循环播放
内存分配器	单向链表	空闲块管理
哈希表冲突处理	单向链表	链地址法

10.6 链表 vs 数组

对比

特性	数组	链表
内存	连续	不连续
大小	固定	动态
访问	O(1)随机访问	O(n)顺序访问
插入/删除	O(n)需要移动	O(1)只需改指针
空间利用	无额外开销	每个节点需要指针

选择建议

用数组：

- 需要频繁随机访问
- 数据量固定
- 内存紧张

用链表：

- 需要频繁插入删除
- 数据量变化大
- 不需要随机访问

第十章小结

本章要点

- 链表是动态数据结构，大小可变
- 每个节点包含数据和指向下一节点的指针
- 循环链表尾节点指向头节点
- 链表适合频繁插入删除的场景
- 数组适合随机访问的场景

动手练习

练习1：实现链表的反转。

```
Node* reverseList(Node *head) {  
    Node *prev = NULL;  
    Node *current = head;  
    Node *next = NULL;  
  
    while (current != NULL) {
```

```
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }

    return prev;
}
```

练习2：实现约瑟夫问题。

附录：常见问题与练习题

常见问题

Q1: printf里的%d、%f、%c是什么？

答：这些叫格式符，告诉printf用什么格式输出变量。

- `%d`：输出整数
- `%f`：输出小数
- `%c`：输出字符

Q2：为什么整数除法5/3结果是1而不是1.67？

答：整数除以整数，结果是整数，小数部分会被舍去。想得到小数结果，至少有一个数要写成小数形式，如 `5.0/3` 或 `5/3.0`。

Q3: a++和++a有什么区别？

答：

- `a++`：先用a的值，再加1
- `++a`：先加1，再用a的值

Q4：为什么我的程序报错了？

常见错误：

1. 忘记分号 `;`
 2. 拼写错误（如 `print` 写成 `printf`）
 3. 中文标点符号（一定要用英文标点）
 4. 括号不匹配
-

练习题答案

第一章练习

```
// 练习1答案
#include <stdio.h>
int main()
{
    printf("我的名字是小明\n");
    return 0;
}
```

```
// 练习2答案
#include <stdio.h>
int main()
{
    printf("第一行\n");
    printf("第二行\n");
    printf("第三行\n");
    return 0;
}
```

第二章练习

```
// 练习1答案
#include <stdio.h>
int main()
{
    int age = 20;
    float height = 1.75;
    char grade = 'B';

    printf("年龄: %d岁\n", age);
    printf("身高: %.2f米\n", height);
    printf("成绩等级: %c\n", grade);
    return 0;
}
```

```
// 练习2答案
#include <stdio.h>
#define PI 3.14159
int main()
{
    float radius = 5.0;
    float area = PI * radius * radius;

    printf("半径 = %.2f\n", radius);
    printf("面积 = %.2f\n", area);
    return 0;
}
```


第四章练习

```
// 练习1答案
#include <stdio.h>
int main()
{
    int num = -5;
    if (num > 0)
        printf("%d 是正数\n", num);
    else if (num < 0)
        printf("%d 是负数\n", num);
    else
        printf("%d 是零\n", num);
    return 0;
}

// 练习2答案
#include <stdio.h>
int main()
{
    int a = 10, b = 3;
    char op = '+';

    switch (op) {
        case '+': printf("%d + %d = %d\n", a, b, a + b); break;
        case '-': printf("%d - %d = %d\n", a, b, a - b); break;
        case '*': printf("%d * %d = %d\n", a, b, a * b); break;
        case '/': printf("%d / %d = %d\n", a, b, a / b); break;
        default: printf("不支持的操作\n"); break;
    }
    return 0;
}
```

第五章练习

```
// 练习1答案
#include <stdio.h>
int main()
{
    int sum = 0;
    for (int i = 1; i <= 100; i++)
        sum += i;
    printf("1到100的和 = %d\n", sum);
    return 0;
}

// 练习2答案
#include <stdio.h>
int main()
{
    for (int i = 1; i <= 9; i++) {
```

```

        for (int j = 1; j <= i; j++)
            printf("%d×%d=%d\t", j, i, i*j);
        printf("\n");
    }
    return 0;
}

// 练习3答案
#include <stdio.h>
int main()
{
    for (int i = 1; i <= 100; i++)
        if (i % 2 == 0)
            printf("%d ", i);
    return 0;
}

```

第六章练习

```

// 练习1答案
#include <stdio.h>
int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int len = sizeof(arr) / sizeof(arr[0]);
    int sum = 0;
    for (int i = 0; i < len; i++)
        sum += arr[i];
    printf("和 = %d\n", sum);
    printf("平均值 = %.2f\n", (float)sum / len);
    return 0;
}

// 练习2答案
#include <stdio.h>
int main()
{
    int arr[] = {10, 50, 30, 80, 20};
    int len = sizeof(arr) / sizeof(arr[0]);
    int max = arr[0];
    for (int i = 1; i < len; i++)
        if (arr[i] > max) max = arr[i];
    printf("最大值 = %d\n", max);
    return 0;
}

```

第七章练习

```
// 练习1答案
#include <stdio.h>
float circleArea(float radius) {
    return 3.14159 * radius * radius;
}
int main()
{
    float r = 5.0;
    printf("半径 %.2f 的圆面积 = %.2f\n", r, circleArea(r));
    return 0;
}

// 练习2答案
#include <stdio.h>
int isLeapYear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}
int main()
{
    int year = 2024;
    if (isLeapYear(year))
        printf("%d 是闰年\n", year);
    else
        printf("%d 不是闰年\n", year);
    return 0;
}
```

第十章练习

```
// 练习1答案: 链表反转
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

Node* reverseList(Node *head) {
    Node *prev = NULL;
    Node *current = head;
```

```
Node *next = NULL;

while (current != NULL) {
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
}

return prev;
}

int main()
{
    Node *head = createNode(1);
    head->next = createNode(2);
    head->next->next = createNode(3);

    printf("原链表: 1 → 2 → 3 → NULL\n");
    head = reverseList(head);
    printf("反转后: 3 → 2 → 1 → NULL\n");

    return 0;
}
```

下一步学习建议

学完这十章，你已经掌握了：

【基础篇】

- ☒ C语言基本概念与环境搭建
- ☒ 变量和数据类型
- ☒ 基本运算符
- ☒ 流程控制 (if-else、switch)
- ☒ 循环 (for、while)

【进阶篇】

- ☒ 数组 (一维、二维、多维)
- ☒ 函数
- ☒ 指针 (C语言的灵魂)
- ☒ 指针与函数 (函数指针、指针函数)
- ☒ 链表 (单向链表、循环链表)

接下来可以学习

主题	应用场景
结构体与联合体	复杂数据类型
文件操作	数据持久化存储
动态内存管理	malloc、free、内存泄漏
预处理器	宏定义、条件编译
多文件编程	大型项目组织
数据结构	树、图、哈希表

推荐学习路径

当前水平 → 进阶方向

├— 嵌入式开发

└— 学习STM32、RTOS、驱动开发

├— 系统编程

└— 学习Linux编程、网络编程

├— 数据结构与算法

└— 学习树、图、排序、查找算法

└— 游戏开发

└— 学习游戏引擎、图形编程

温馨提示：学习编程最重要的是动手实践，多写代码，多犯错，多总结！

指针和链表是C语言最核心的内容，一定要多练习！

祝你学习顺利！

本笔记持续更新中...



知识点项目应用速查表

知识点	项目应用
数组	传感器数据缓存、图像处理、游戏地图
二维数组	表格数据、棋盘游戏、像素矩阵
多维数组	RGB图像、视频帧、3D地图

知识点	项目应用
指针	嵌入式寄存器操作、动态内存、函数参数传递
函数指针	回调函数、状态机、事件处理、插件系统
指针函数	动态内存分配、返回数组、链表创建
单向链表	内存管理、任务队列、哈希表
循环链表	进程调度、轮询系统、循环播放列表

第十一章：C语言常用框架

C语言本身没有像Java、Python那样丰富的框架生态，但在特定领域有一些经典的"框架"和代码组织方式

11.1 嵌入式开发框架

11.1.1 HAL硬件抽象层框架

什么是HAL？

HAL（Hardware Abstraction Layer）是嵌入式开发中最常用的框架思想，将硬件操作抽象成统一接口。

应用场景

应用位置	说明
STM32开发	ST官方HAL库
Arduino	底层硬件抽象
Linux驱动	统一的设备驱动接口
跨平台嵌入式	同一代码适配不同硬件

代码示例

```
// hal_gpio.h - 硬件抽象层头文件
#ifndef HAL_GPIO_H
#define HAL_GPIO_H

#include <stdint.h>

// GPIO配置结构体
typedef struct {
    uint8_t port;
    uint8_t pin;
    uint8_t mode;      // 输入/输出/复用
    uint8_t pull;      // 上拉/下拉/无
```

```

} GPIO_Config;

// HAL接口定义（统一接口，不同硬件有不同实现）
void HAL_GPIO_Init(GPIO_Config *config);
void HAL_GPIO_WritePin(uint8_t port, uint8_t pin, uint8_t state);
uint8_t HAL_GPIO_ReadPin(uint8_t port, uint8_t pin);
void HAL_GPIO_TogglePin(uint8_t port, uint8_t pin);

#endif

```

```

// hal_gpio_stm32.c - STM32平台实现
#include "hal_gpio.h"
#include "stm32f4xx.h"

void HAL_GPIO_Init(GPIO_Config *config) {
    // STM32特定的GPIO初始化代码
    GPIO_TypeDef *gpioPort;
    switch(config->port) {
        case 0: gpioPort = GPIOA; break;
        case 1: gpioPort = GPIOB; break;
        case 2: gpioPort = GPIOC; break;
    }
    // ... STM32寄存器配置
}

void HAL_GPIO_WritePin(uint8_t port, uint8_t pin, uint8_t state) {
    // STM32写GPIO实现
}

```

```

// hal_gpio_esp32.c - ESP32平台实现
#include "hal_gpio.h"
#include "driver/gpio.h"

void HAL_GPIO_Init(GPIO_Config *config) {
    // ESP32特定的GPIO初始化代码
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << config->pin),
        .mode = config->mode,
        .pull_up_en = config->pull,
    };
    gpio_config(&io_conf);
}

void HAL_GPIO_WritePin(uint8_t port, uint8_t pin, uint8_t state) {
    gpio_set_level(pin, state);
}

```

优劣势分析

优势	劣势
✔ 代码可移植性强	✘ 增加代码层次，性能有损耗
✔ 上层应用不关心硬件细节	✘ 需要为每个平台编写适配层
✔ 便于团队协作（硬件/软件分离）	✘ 调试时需要跨层定位问题
✔ 易于单元测试（可模拟硬件）	✘ 代码量增加

选择建议

推荐使用HAL的场景：

- └— 产品需要支持多种硬件平台
- └— 团队分工明确（驱动层/应用层）
- └— 代码需要长期维护和升级
- └— 需要进行单元测试

不推荐使用HAL的场景：

- └— 对性能要求极高
- └— 只针对单一硬件平台
- └— 项目周期短、代码量小
- └— 团队规模小，不需要分层

11.1.2 事件驱动框架

什么是事件驱动？

事件驱动框架通过事件队列和事件处理器来组织代码，适合响应用户操作、传感器数据等异步场景。

应用场景

应用位置	说明
GUI系统	按钮点击、触摸事件
游戏开发	键盘输入、碰撞检测
嵌入式系统	定时器事件、中断处理
物联网设备	传感器数据、网络消息

代码示例

```
// event_system.h - 事件驱动框架头文件
#ifndef EVENT_SYSTEM_H
#define EVENT_SYSTEM_H

#include <stdint.h>
```



```

// 事件类型定义
typedef enum {
    EVENT_NONE = 0,
    EVENT_KEY_PRESS,
    EVENT_KEY_RELEASE,
    EVENT_TIMER,
    EVENT_SENSOR_DATA,
    EVENT_NETWORK_RX,
    EVENT_CUSTOM_START = 100
} EventType;

// 事件结构体
typedef struct {
    EventType type;
    uint32_t timestamp;
    void *data;
    uint16_t dataLen;
} Event;

// 事件处理函数类型
typedef void (*EventHandler)(Event *event);

// 框架API
void EventSystem_Init(void);
void EventSystem_Post(Event *event);           // 发布事件
void EventSystem_Register(EventType type, EventHandler handler); // 注册处理器
void EventSystem_Process(void);                // 处理事件循环

#endif

```

```

// event_system.c - 事件驱动框架实现
#include "event_system.h"
#include <string.h>

#define MAX_EVENTS 32
#define MAX_HANDLERS 16

static Event eventQueue[MAX_EVENTS];
static int head = 0, tail = 0;
static EventHandler handlers[EVENT_CUSTOM_START + MAX_HANDLERS];

void EventSystem_Init(void) {
    memset(eventQueue, 0, sizeof(eventQueue));
    memset(handlers, 0, sizeof(handlers));
    head = tail = 0;
}

void EventSystem_Post(Event *event) {
    eventQueue[tail] = *event;
    tail = (tail + 1) % MAX_EVENTS;
}

```

```

void EventSystem_Register(EventType type, EventHandler handler) {
    if (type < EVENT_CUSTOM_START + MAX_HANDLERS) {
        handlers[type] = handler;
    }
}

void EventSystem_Process(void) {
    while (head != tail) {
        Event *evt = &eventQueue[head];

        if (handlers[evt->type] != NULL) {
            handlers[evt->type](evt); // 调用注册的处理器
        }

        head = (head + 1) % MAX_EVENTS;
    }
}

```

```

// main.c - 使用示例
#include "event_system.h"
#include <stdio.h>

// 键盘事件处理器
void OnKeyPress(Event *event) {
    printf("按键按下: %c\n", *(char*)event->data);
}

// 定时器事件处理器
void OnTimer(Event *event) {
    printf("定时器触发\n");
}

int main() {
    // 初始化事件系统
    EventSystem_Init();

    // 注册事件处理器
    EventSystem_Register(EVENT_KEY_PRESS, OnKeyPress);
    EventSystem_Register(EVENT_TIMER, OnTimer);

    // 模拟发送事件
    char key = 'A';
    Event keyEvent = {EVENT_KEY_PRESS, 0, &key, 1};
    EventSystem_Post(&keyEvent);

    Event timerEvent = {EVENT_TIMER, 0, NULL, 0};
    EventSystem_Post(&timerEvent);

    // 事件循环
    while (1) {
        EventSystem_Process();
    }
}

```

```
        // 其他任务...
    }

    return 0;
}
```

优劣势分析

优势	劣势
✔ 解耦：事件发送者和处理者无直接依赖	✘ 调试困难：事件流难以追踪
✔ 灵活：可动态注册/注销处理器	✘ 内存开销：需要事件队列
✔ 扩展性好：新增事件类型简单	✘ 时序问题：事件处理顺序不确定
✔ 适合异步场景	✘ 可能遗漏事件（队列满）

选择建议

推荐使用事件驱动的场景：

- └─ 系统需要响应多种异步事件
- └─ 模块间需要松耦合通信
- └─ 需要灵活扩展功能
- └─ GUI、游戏、物联网应用

不推荐使用事件驱动的场景：

- └─ 简单的顺序执行程序
- └─ 对实时性要求极高
- └─ 资源极度受限的嵌入式设备
- └─ 团队不熟悉事件驱动思想

11.2 通用代码组织框架

11.2.1 模块化框架

什么是模块化？

将代码按功能划分为独立模块，每个模块包含头文件（接口）和源文件（实现）。

应用场景

应用位置	说明
大型项目	功能模块划分
库开发	提供清晰的API
团队协作	模块负责人明确
代码复用	模块独立可用

代码示例

项目目录结构:

```
project/
├── modules/
│   ├── uart/
│   │   ├── uart.h
│   │   └── uart.c
│   ├── timer/
│   │   ├── timer.h
│   │   └── timer.c
│   └── sensor/
│       ├── sensor.h
│       └── sensor.c
├── app/
│   └── main.c
└── Makefile
```

```
// modules/uart/uart.h - UART模块头文件
#ifndef UART_H
#define UART_H

#include <stdint.h>

// 对外公开的接口（隐藏实现细节）
void UART_Init(uint32_t baudrate);
void UART_SendByte(uint8_t data);
void UART_SendString(const char *str);
uint8_t UART_ReceiveByte(void);

// 回调函数类型
typedef void (*UART_RxCallback)(uint8_t data);
void UART_SetRxCallback(UART_RxCallback callback);

#endif
```

```
// modules/uart/uart.c - UART模块实现
#include "uart.h"
#include <string.h>

// 内部变量（外部不可见）
static UART_RxCallback rxCallback = NULL;
static uint8_t rxBuffer[256];
static uint16_t rxIndex = 0;

void UART_Init(uint32_t baudrate) {
    // 硬件初始化代码
    // 配置波特率、数据位、停止位等
}

void UART_SendByte(uint8_t data) {
```

```

    // 发送单个字节
}

void UART_SendString(const char *str) {
    while (*str) {
        UART_SendByte(*str++);
    }
}

uint8_t UART_ReceiveByte(void) {
    // 接收单个字节
    return 0;
}

void UART_SetRxCallback(UART_RxCallback callback) {
    rxCallback = callback;
}

// 中断服务函数（内部函数，不暴露）
void UART_IRQHandler(void) {
    uint8_t data = UART_ReceiveByte();
    if (rxCallback != NULL) {
        rxCallback(data);
    }
}

```

```

// app/main.c - 主程序使用模块
#include "uart.h"
#include "timer.h"
#include "sensor.h"
#include <stdio.h>

// UART接收回调
void OnUartRx(uint8_t data) {
    printf("收到数据: %c\n", data);
}

int main() {
    // 初始化各模块
    UART_Init(115200);
    UART_SetRxCallback(OnUartRx);
    Timer_Init();
    Sensor_Init();

    // 主循环
    while (1) {
        // 业务逻辑
    }

    return 0;
}

```

优劣势分析

优势	劣势
✔ 职责清晰，易于维护	✘ 需要更多的文件和目录
✔ 便于团队协作	✘ 编译配置复杂
✔ 模块可独立测试	✘ 模块间依赖管理需要规划
✔ 代码复用性高	✘ 小项目可能过度设计

选择建议

推荐使用模块化的场景：

- └— 代码量超过1000行
- └— 多人协作开发
- └— 需要长期维护
- └— 功能边界清晰

不推荐使用模块化的场景：

- └— 单文件小程序（<500行）
- └— 快速原型开发
- └— 单人开发且周期短
- └— 功能高度耦合

11.2.2 配置管理框架

什么是配置管理？

通过配置文件或宏定义来控制代码的行为，实现一套代码适配多种场景。

应用场景

应用位置	说明
产品线开发	同系列不同配置
多平台适配	编译时选择平台
功能裁剪	按需启用功能
调试开关	开发/发布模式切换

代码示例

```
// config.h - 配置头文件
#ifndef CONFIG_H
#define CONFIG_H

// ===== 硬件配置 =====
```

```

#define CONFIG_BOARD_V1      1
#define CONFIG_BOARD_V2      2

// 选择当前硬件版本
#define CONFIG_BOARD_VERSION  CONFIG_BOARD_V1

// ===== 功能开关 =====
#define CONFIG_FEATURE_UART   1      // 是否启用UART
#define CONFIG_FEATURE_SPI     1      // 是否启用SPI
#define CONFIG_FEATURE_I2C     0      // 是否启用I2C
#define CONFIG_FEATURE_DEBUG   1      // 是否启用调试输出

// ===== 参数配置 =====
#define CONFIG_UART_BAUDRATE   115200
#define CONFIG_BUFFER_SIZE     256
#define CONFIG_MAX_SENSORS     8

// ===== 条件编译 =====
#if CONFIG_BOARD_VERSION == CONFIG_BOARD_V1
    #define CONFIG_LED_PIN      5
    #define CONFIG_BUTTON_PIN   6
#elif CONFIG_BOARD_VERSION == CONFIG_BOARD_V2
    #define CONFIG_LED_PIN      10
    #define CONFIG_BUTTON_PIN   11
#endif

// 调试输出宏
#if CONFIG_FEATURE_DEBUG
    #define DEBUG_PRINT(fmt, ...) printf("[DEBUG] " fmt "\n", ##__VA_ARGS__)
#else
    #define DEBUG_PRINT(fmt, ...) // 空实现
#endif

#endif

```

```

// main.c - 使用配置
#include "config.h"
#include <stdio.h>

int main() {
    DEBUG_PRINT("程序启动");

    printf("硬件版本: v%d\n", CONFIG_BOARD_VERSION);
    printf("LED引脚: %d\n", CONFIG_LED_PIN);

    #if CONFIG_FEATURE_UART
        printf("UART波特率: %d\n", CONFIG_UART_BAUDRATE);
    #endif

    #if CONFIG_FEATURE_I2C
        printf("I2C已启用\n");
    #else

```

```
printf("I2C未启用\n");
#endif

DEBUG_PRINT("初始化完成");
return 0;
}
```

优劣势分析

优势	劣势
✔ 一套代码多种配置	✘ 编译时确定，不能运行时修改
✔ 功能裁剪灵活	✘ 配置项多时难以管理
✔ 便于版本管理	✘ 配置错误难以排查
✔ 减少代码冗余	✘ 需要重新编译才能生效

选择建议

推荐使用配置管理的场景：

- └— 产品有多个版本/配置
- └— 需要功能裁剪（资源受限）
- └— 开发/发布环境切换
- └— 平台适配

不推荐使用配置管理的场景：

- └— 配置需要运行时修改
- └— 只有一个固定版本
- └— 配置项极少
- └— 需要用户自定义配置

11.3 内存管理框架

11.3.1 内存池框架

什么是内存池？

预先分配一大块内存，然后从中划分小块供程序使用，避免频繁调用malloc/free。

应用场景

应用位置	说明
嵌入式系统	避免内存碎片
实时系统	确定性的内存分配时间
游戏开发	频繁创建/销毁对象

应用位置	说明
网络服务器	固定大小的数据包缓冲

代码示例

```
// mem_pool.h - 内存池框架
#ifndef MEM_POOL_H
#define MEM_POOL_H

#include <stdint.h>
#include <stdbool.h>

// 内存池结构体
typedef struct {
    uint8_t *pool;           // 内存池基地址
    uint32_t blockSize;      // 每个块的大小
    uint32_t blockCount;     // 块的数量
    uint32_t *freeList;      // 空闲块索引
    uint32_t freeCount;      // 空闲块数量
} MemPool;

// API
bool MemPool_Init(MemPool *mp, uint32_t blockSize, uint32_t blockCount);
void* MemPool_Alloc(MemPool *mp);
void MemPool_Free(MemPool *mp, void *ptr);
void MemPool_Deinit(MemPool *mp);

#endif
```

```
// mem_pool.c - 内存池实现
#include "mem_pool.h"
#include <stdlib.h>
#include <string.h>

bool MemPool_Init(MemPool *mp, uint32_t blockSize, uint32_t blockCount) {
    // 分配内存池
    mp->pool = (uint8_t*)malloc(blockSize * blockCount);
    if (mp->pool == NULL) return false;

    // 分配空闲列表
    mp->freeList = (uint32_t*)malloc(blockCount * sizeof(uint32_t));
    if (mp->freeList == NULL) {
        free(mp->pool);
        return false;
    }

    mp->blockSize = blockSize;
    mp->blockCount = blockCount;
    mp->freeCount = blockCount;
```

```

    // 初始化空闲列表
    for (uint32_t i = 0; i < blockCount; i++) {
        mp->freeList[i] = i;
    }

    return true;
}

void* MemPool_Alloc(MemPool *mp) {
    if (mp->freeCount == 0) {
        return NULL; // 内存池已满
    }

    mp->freeCount--;
    uint32_t index = mp->freeList[mp->freeCount];
    return mp->pool + (index * mp->blockSize);
}

void MemPool_Free(MemPool *mp, void *ptr) {
    if (ptr == NULL) return;

    // 计算块索引
    uint32_t offset = (uint8_t*)ptr - mp->pool;
    uint32_t index = offset / mp->blockSize;

    // 添加回空闲列表
    mp->freeList[mp->freeCount] = index;
    mp->freeCount++;
}

void MemPool_Deinit(MemPool *mp) {
    free(mp->pool);
    free(mp->freeList);
    mp->pool = NULL;
    mp->freeList = NULL;
}

```

```

// main.c - 使用示例
#include "mem_pool.h"
#include <stdio.h>

typedef struct {
    int id;
    char name[20];
    float score;
} Student;

int main() {
    MemPool pool;

    // 创建内存池：每块sizeof(Student)大小，共10块
    if (!MemPool_Init(&pool, sizeof(Student), 10)) {

```

```
        printf("内存池初始化失败\n");
        return -1;
    }

    // 从内存池分配
    Student *s1 = (Student*)MemPool_Alloc(&pool);
    Student *s2 = (Student*)MemPool_Alloc(&pool);

    s1->id = 1;
    strcpy(s1->name, "张三");
    s1->score = 95.5;

    printf("学生: %s, 成绩: %.1f\n", s1->name, s1->score);

    // 释放回内存池
    MemPool_Free(&pool, s1);
    MemPool_Free(&pool, s2);

    // 销毁内存池
    MemPool_Deinit(&pool);

    return 0;
}
```

优劣势分析

优势	劣势
✔ 分配速度快 (O(1))	✘ 块大小固定, 不够灵活
✔ 无内存碎片	✘ 可能浪费空间 (块太大)
✔ 时间确定性, 适合实时系统	✘ 需要预估最大使用量
✔ 避免内存泄漏	✘ 初始化时占用大块内存

选择建议

推荐使用内存池的场景:

└─ 实时系统 (需要确定性分配时间)

└─ 频繁分配/释放相同大小的对象

└─ 长期运行的嵌入式系统

└─ 需要避免内存碎片

不推荐使用内存池的场景:

└─ 对象大小变化大

└─ 内存资源极度紧张

└─ 分配频率低

└─ 不需要确定性的分配时间

第十二章：C语言常用设计模式

设计模式是解决特定问题的成熟方案，虽然C语言是面向过程的，但同样可以应用许多设计模式

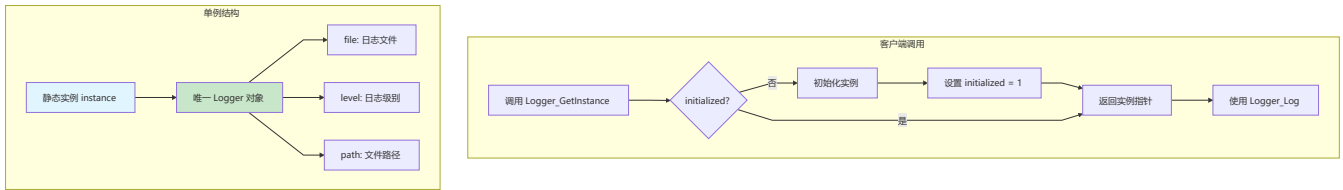
12.1 创建型模式

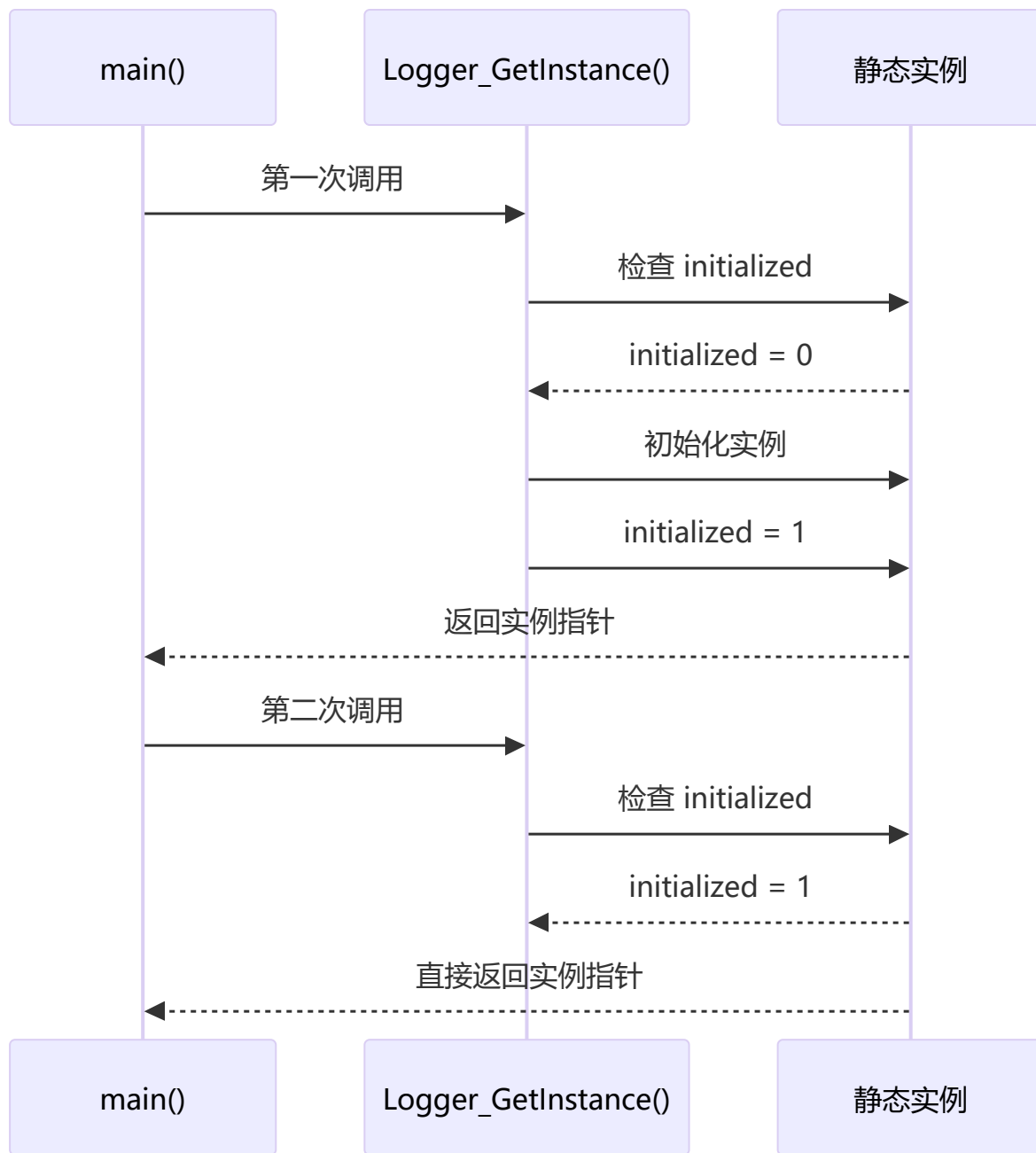
12.1.1 单例模式 (Singleton)

什么是单例？

确保一个类只有一个实例，提供全局访问点。在C语言中通过静态变量实现。

流程图





应用场景

应用位置	说明
配置管理	全局配置对象
日志系统	统一的日志记录器
数据库连接	共享连接池
硬件驱动	单一硬件实例

代码示例

```
// singleton_logger.h - 单例日志系统
#ifndef SINGLETON_LOGGER_H
#define SINGLETON_LOGGER_H

#include <stdio.h>

typedef struct {
    FILE *file;
    int level;
    char path[256];
} Logger;

// 获取单例实例
Logger* Logger_GetInstance(void);

// 日志接口
void Logger_SetFile(const char *path);
void Logger_Log(int level, const char *message);
void Logger_Close(void);

// 日志级别
#define LOG_DEBUG    0
#define LOG_INFO     1
#define LOG_WARN     2
#define LOG_ERROR    3

#endif
```

```
// singleton_logger.c - 单例实现
#include "singleton_logger.h"
#include <string.h>
#include <stdarg.h>

// 静态实例（单例核心）
static Logger instance = {NULL, LOG_INFO, ""};
static int initialized = 0;

Logger* Logger_GetInstance(void) {
    if (!initialized) {
        // 初始化默认配置
        instance.file = stdout;
        instance.level = LOG_INFO;
        strcpy(instance.path, "stdout");
        initialized = 1;
    }
    return &instance;
}

void Logger_SetFile(const char *path) {
    Logger *logger = Logger_GetInstance();
```

```

    if (logger->file != stdout && logger->file != NULL) {
        fclose(logger->file);
    }
    logger->file = fopen(path, "a");
    strcpy(logger->path, path);
}

void Logger_Log(int level, const char *message) {
    Logger *logger = Logger_GetInstance();

    if (level < logger->level) return;

    const char *levelStr[] = {"DEBUG", "INFO", "WARN", "ERROR"};
    fprintf(logger->file, "[%s] %s\n", levelStr[level], message);
    fflush(logger->file);
}

void Logger_Close(void) {
    Logger *logger = Logger_GetInstance();
    if (logger->file != stdout && logger->file != NULL) {
        fclose(logger->file);
        logger->file = stdout;
    }
}

```

```

// main.c - 使用示例
#include "singleton_logger.h"

void FunctionA() {
    Logger_Log(LOG_INFO, "FunctionA 执行");
}

void FunctionB() {
    Logger_Log(LOG_WARN, "FunctionB 发出警告");
}

int main() {
    // 获取日志器（全局唯一）
    Logger *logger = Logger_GetInstance();

    // 设置日志文件
    Logger_SetFile("app.log");

    // 全局使用
    Logger_Log(LOG_INFO, "程序启动");
    FunctionA();
    FunctionB();
    Logger_Log(LOG_ERROR, "发生错误");

    Logger_Close();
    return 0;
}

```

优劣势分析

优势	劣势
✔ 全局唯一访问点	✘ 全局状态，难以追踪
✔ 延迟初始化	✘ 多线程需要加锁
✔ 节省内存	✘ 单元测试困难
✔ 统一管理资源	✘ 违反单一职责原则

选择建议

推荐使用单例的场景：

- └— 确实只需要一个实例（硬件、配置）
- └— 需要全局访问点
- └— 资源昂贵，需要延迟加载
- └— 需要统一管理

不推荐使用单例的场景：

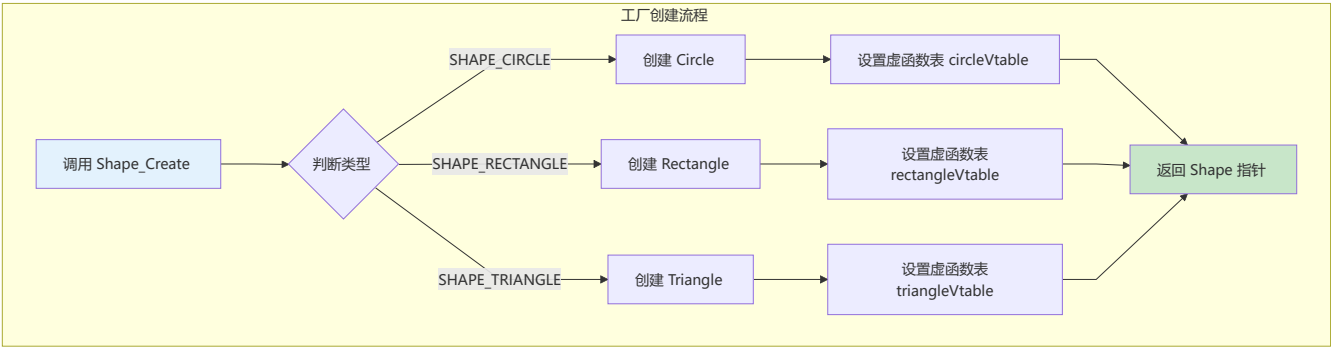
- └— 需要多个实例
- └— 多线程环境且无保护
- └— 需要频繁进行单元测试
- └— 实例状态需要独立

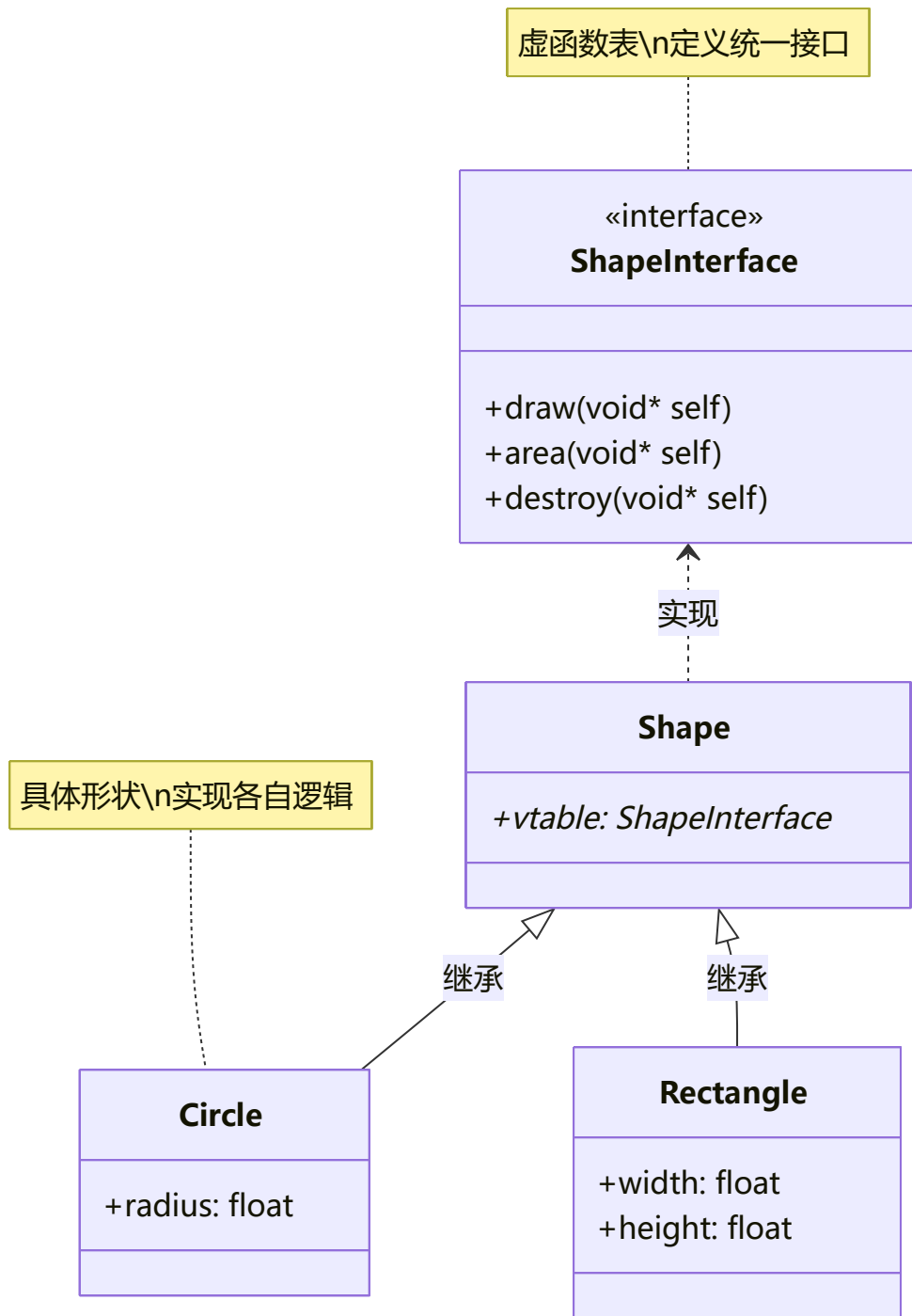
12.1.2 工厂模式（Factory）

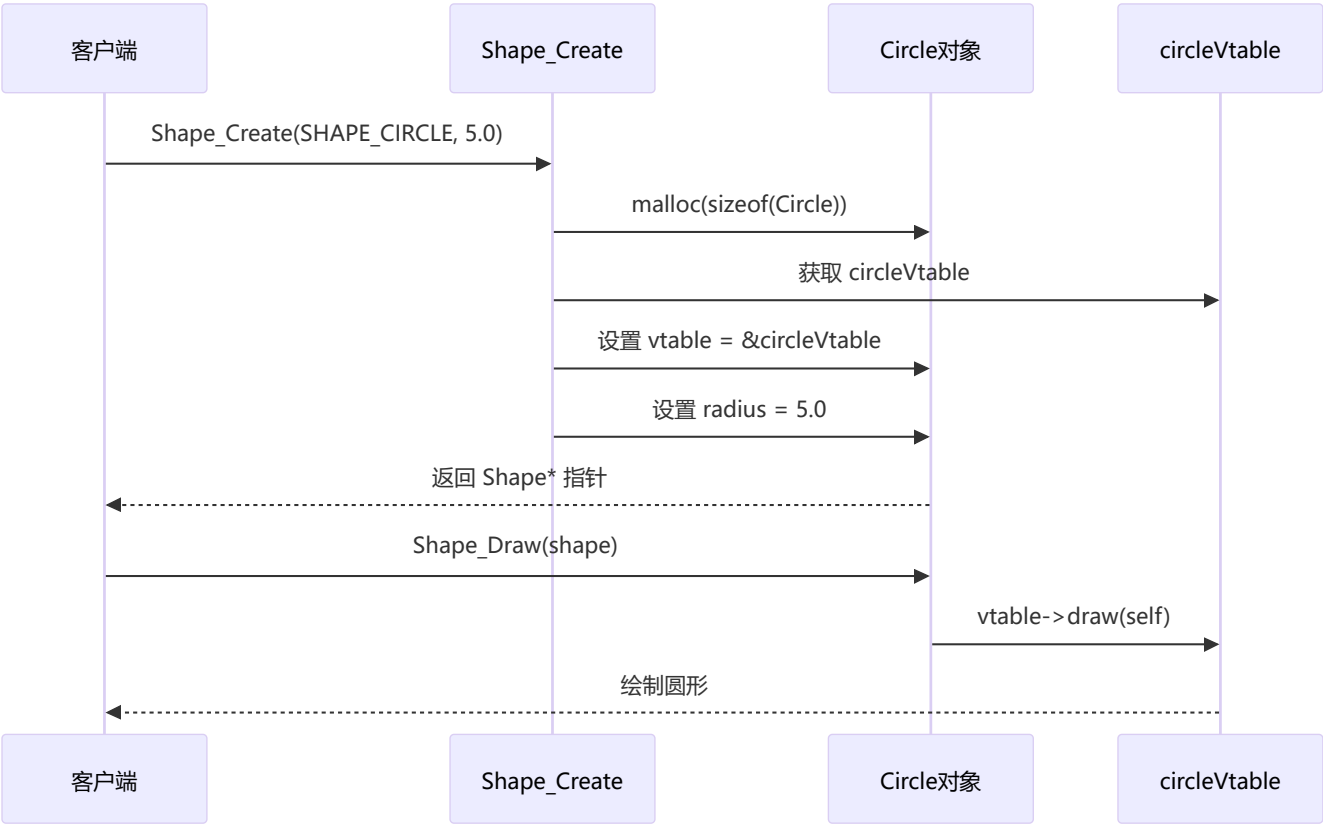
什么是工厂？

将对象的创建过程封装起来，调用者不需要知道具体创建细节。

流程图







应用场景

应用位置	说明
跨平台开发	根据平台创建不同实现
驱动开发	创建不同设备驱动
协议解析	创建不同协议处理器
UI框架	创建不同控件

代码示例

```
// factory_shape.h - 形状工厂
#ifndef FACTORY_SHAPE_H
#define FACTORY_SHAPE_H

// 形状类型枚举
typedef enum {
    SHAPE_CIRCLE,
    SHAPE_RECTANGLE,
    SHAPE_TRIANGLE
} ShapeType;

// 形状接口（虚函数表）
typedef struct ShapeInterface {
    void (*draw)(void *self);
    float (*area)(void *self);
};
```

```

    void (*destroy)(void *self);
} ShapeInterface;

// 形状基类
typedef struct {
    const ShapeInterface *vtable;
} Shape;

// 具体形状：圆形
typedef struct {
    Shape base;
    float radius;
} Circle;

// 具体形状：矩形
typedef struct {
    Shape base;
    float width;
    float height;
} Rectangle;

// 工厂函数
Shape* Shape_Create(ShapeType type, ...);
void Shape_Destroy(Shape *shape);

// 公共接口（通过虚函数表调用）
void Shape_Draw(Shape *shape);
float Shape_Area(Shape *shape);

#endif

```

```

// factory_shape.c - 工厂实现
#include "factory_shape.h"
#include <stdlib.h>
#include <stdio.h>
#include <stdarg.h>
#include <math.h>

// ===== 圆形实现 =====
static void Circle_Draw(void *self) {
    Circle *circle = (Circle*)self;
    printf("绘制圆形, 半径: %.2f\n", circle->radius);
}

static float Circle_Area(void *self) {
    Circle *circle = (Circle*)self;
    return 3.14159f * circle->radius * circle->radius;
}

static void Circle_Destroy(void *self) {
    free(self);
}

```

```

static const ShapeInterface circlevtable = {
    Circle_Draw,
    Circle_Area,
    Circle_Destroy
};

// ===== 矩形实现 =====
static void Rectangle_Draw(void *self) {
    Rectangle *rect = (Rectangle*)self;
    printf("绘制矩形, 宽: %.2f, 高: %.2f\n", rect->width, rect->height);
}

static float Rectangle_Area(void *self) {
    Rectangle *rect = (Rectangle*)self;
    return rect->width * rect->height;
}

static void Rectangle_Destroy(void *self) {
    free(self);
}

static const ShapeInterface rectanglevtable = {
    Rectangle_Draw,
    Rectangle_Area,
    Rectangle_Destroy
};

// ===== 工厂函数 =====
Shape* Shape_Create(ShapeType type, ...) {
    va_list args;
    va_start(args, type);

    Shape *shape = NULL;

    switch (type) {
        case SHAPE_CIRCLE: {
            Circle *circle = malloc(sizeof(Circle));
            circle->base.vtable = &circlevtable;
            circle->radius = (float)va_arg(args, double);
            shape = (Shape*)circle;
            break;
        }
        case SHAPE_RECTANGLE: {
            Rectangle *rect = malloc(sizeof(Rectangle));
            rect->base.vtable = &rectanglevtable;
            rect->width = (float)va_arg(args, double);
            rect->height = (float)va_arg(args, double);
            shape = (Shape*)rect;
            break;
        }
        default:
            break;
    }
}

```

```

    }

    va_end(args);
    return shape;
}

void Shape_Destroy(Shape *shape) {
    if (shape && shape->vtable && shape->vtable->destroy) {
        shape->vtable->destroy(shape);
    }
}

// ===== 公共接口 =====
void Shape_Draw(Shape *shape) {
    if (shape && shape->vtable && shape->vtable->draw) {
        shape->vtable->draw(shape);
    }
}

float Shape_Area(Shape *shape) {
    if (shape && shape->vtable && shape->vtable->area) {
        return shape->vtable->area(shape);
    }
    return 0;
}

```

```

// main.c - 使用示例
#include "factory_shape.h"
#include <stdio.h>

int main() {
    // 使用工厂创建形状
    Shape *circle = Shape_Create(SHAPE_CIRCLE, 5.0);
    Shape *rect = Shape_Create(SHAPE_RECTANGLE, 4.0, 6.0);

    // 统一接口操作
    Shape_Draw(circle);
    printf("圆形面积: %.2f\n", Shape_Area(circle));

    Shape_Draw(rect);
    printf("矩形面积: %.2f\n", Shape_Area(rect));

    // 销毁
    Shape_Destroy(circle);
    Shape_Destroy(rect);

    return 0;
}

```

优劣势分析

优势	劣势
✔ 创建逻辑集中管理	✘ 增加代码复杂度
✔ 调用者无需知道具体类型	✘ 每新增类型需修改工厂
✔ 便于扩展新类型	✘ C语言实现较为繁琐
✔ 解耦创建和使用	✘ 需要手动管理内存

选择建议

推荐使用工厂模式的场景：

- └─ 需要根据条件创建不同对象
- └─ 创建过程复杂
- └─ 需要统一管理对象创建
- └─ 对象类型可能扩展

不推荐使用工厂模式的场景：

- └─ 对象类型固定且简单
- └─ 直接创建足够清晰
- └─ 不需要扩展
- └─ 项目代码量小

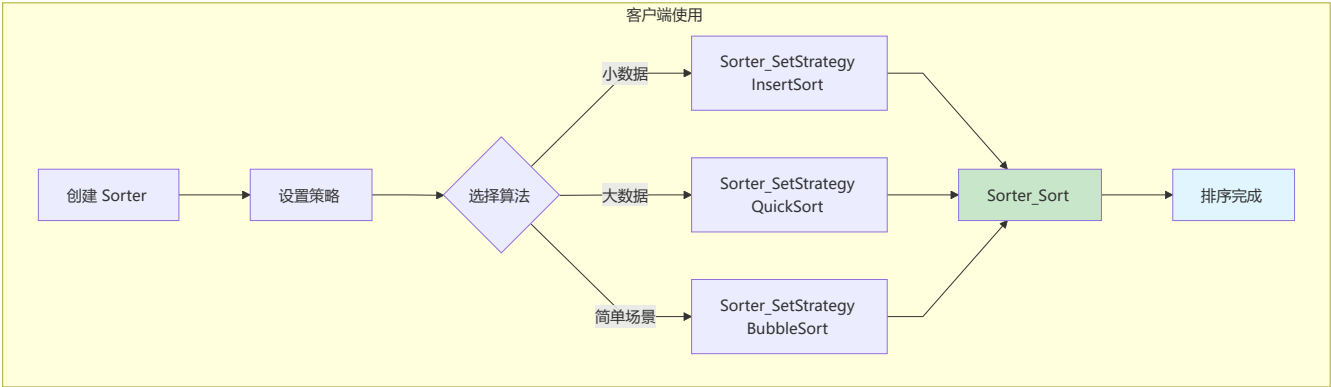
12.2 行为型模式

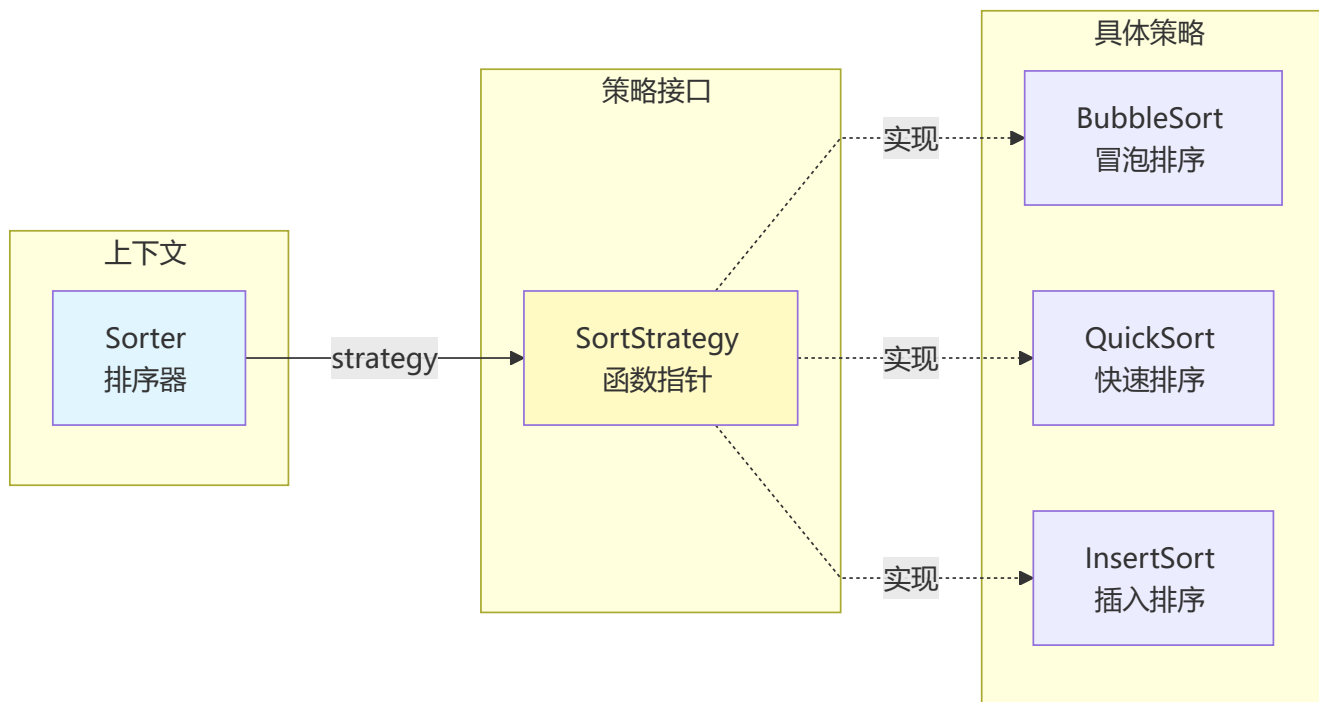
12.2.1 策略模式（Strategy）

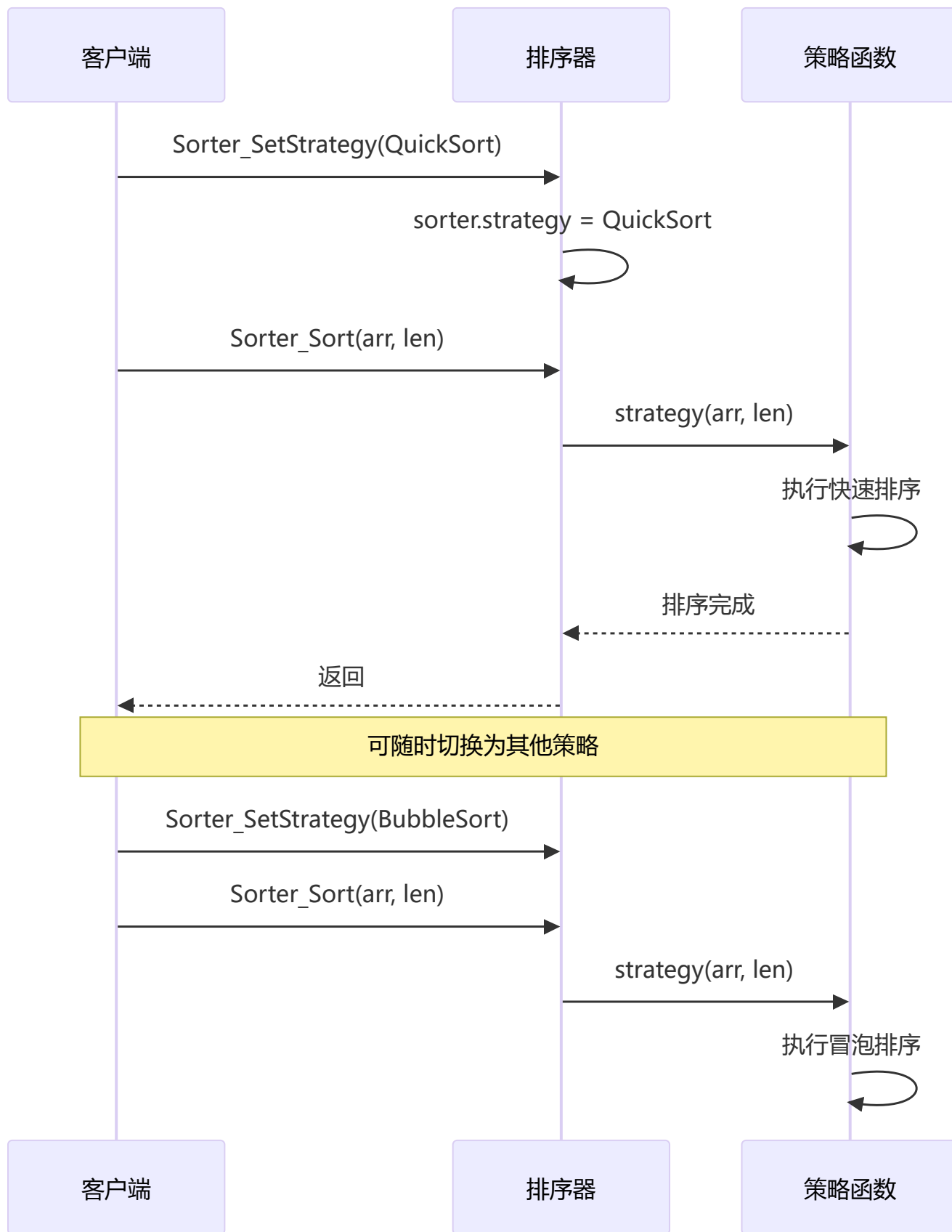
什么是策略模式？

定义一系列算法，把它们封装起来，并使它们可以互相替换。

流程图







应用场景

应用位置	说明
排序算法	选择不同排序方法
支付系统	不同支付方式
压缩算法	选择不同压缩方式
路径规划	不同寻路算法

代码示例

```
// strategy_sort.h - 排序策略
#ifndef STRATEGY_SORT_H
#define STRATEGY_SORT_H

#include <stddef.h>

// 排序策略接口
typedef void (*SortStrategy)(int *arr, size_t len);

// 具体策略
void BubbleSort(int *arr, size_t len);
void QuickSort(int *arr, size_t len);
void InsertSort(int *arr, size_t len);

// 排序器（上下文）
typedef struct {
    SortStrategy strategy;
} Sorter;

void Sorter_SetStrategy(Sorter *sorter, SortStrategy strategy);
void Sorter_Sort(Sorter *sorter, int *arr, size_t len);

#endif

// strategy_sort.c - 策略实现
#include "strategy_sort.h"

// ===== 冒泡排序策略 =====
void BubbleSort(int *arr, size_t len) {
    for (size_t i = 0; i < len - 1; i++) {
        for (size_t j = 0; j < len - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```

}

// ===== 快速排序策略 =====
static void quickSortHelper(int *arr, int low, int high) {
    if (low < high) {
        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        int pi = i + 1;
        quickSortHelper(arr, low, pi - 1);
        quickSortHelper(arr, pi + 1, high);
    }
}

void QuickSort(int *arr, size_t len) {
    quickSortHelper(arr, 0, len - 1);
}

// ===== 插入排序策略 =====
void InsertSort(int *arr, size_t len) {
    for (size_t i = 1; i < len; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// ===== 排序器实现 =====
void Sorter_SetStrategy(Sorter *sorter, SortStrategy strategy) {
    sorter->strategy = strategy;
}

void Sorter_Sort(Sorter *sorter, int *arr, size_t len) {
    if (sorter->strategy != NULL) {
        sorter->strategy(arr, len);
    }
}

```

```
}  
}
```

```
// main.c - 使用示例  
#include "strategy_sort.h"  
#include <stdio.h>  
#include <stdlib.h>  
#include <time.h>  
  
void PrintArray(int *arr, size_t len) {  
    for (size_t i = 0; i < len; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}  
  
int main() {  
    int arr1[] = {64, 34, 25, 12, 22, 11, 90};  
    int arr2[] = {64, 34, 25, 12, 22, 11, 90};  
    int arr3[] = {64, 34, 25, 12, 22, 11, 90};  
    size_t len = sizeof(arr1) / sizeof(arr1[0]);  
  
    Sorter sorter;  
  
    // 使用冒泡排序  
    Sorter_SetStrategy(&sorter, BubbleSort);  
    Sorter_Sort(&sorter, arr1, len);  
    printf("冒泡排序: ");  
    PrintArray(arr1, len);  
  
    // 使用快速排序  
    Sorter_SetStrategy(&sorter, QuickSort);  
    Sorter_Sort(&sorter, arr2, len);  
    printf("快速排序: ");  
    PrintArray(arr2, len);  
  
    // 使用插入排序  
    Sorter_SetStrategy(&sorter, InsertSort);  
    Sorter_Sort(&sorter, arr3, len);  
    printf("插入排序: ");  
    PrintArray(arr3, len);  
  
    // 根据数据规模选择策略  
    size_t largeSize = 10000;  
    int *largeArr = malloc(largeSize * sizeof(int));  
  
    if (largeSize < 100) {  
        Sorter_SetStrategy(&sorter, InsertSort); // 小数据用插入  
    } else {  
        Sorter_SetStrategy(&sorter, QuickSort); // 大数据用快排  
    }  
}
```

```
    free(largeArr);  
    return 0;  
}
```

优劣势分析

优势	劣势
✔ 算法可以自由切换	✘ 客户端需要知道所有策略
✔ 避免使用多重条件判断	✘ 策略过多会增加类数量
✔ 扩展性好，新增策略简单	✘ 策略需要对外暴露
✔ 易于单元测试	✘ C语言实现需要函数指针

选择建议

推荐使用策略模式的场景：

- └— 有多种方式完成同一任务
- └— 需要在运行时切换算法
- └— 算法可能经常变化
- └— 需要隔离算法实现

不推荐使用策略模式的场景：

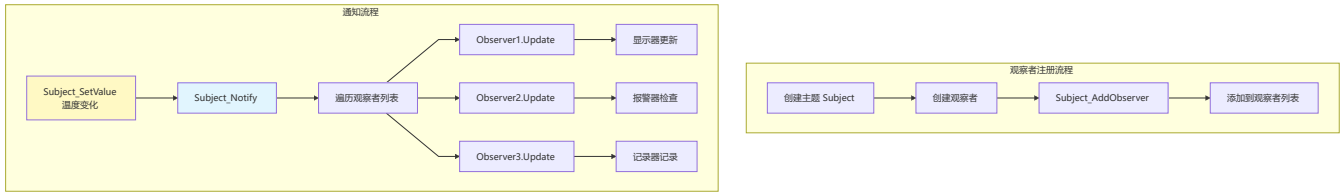
- └— 算法固定且唯一
- └— 算法简单，不值得封装
- └— 客户端不需要选择算法
- └— 策略之间差异小

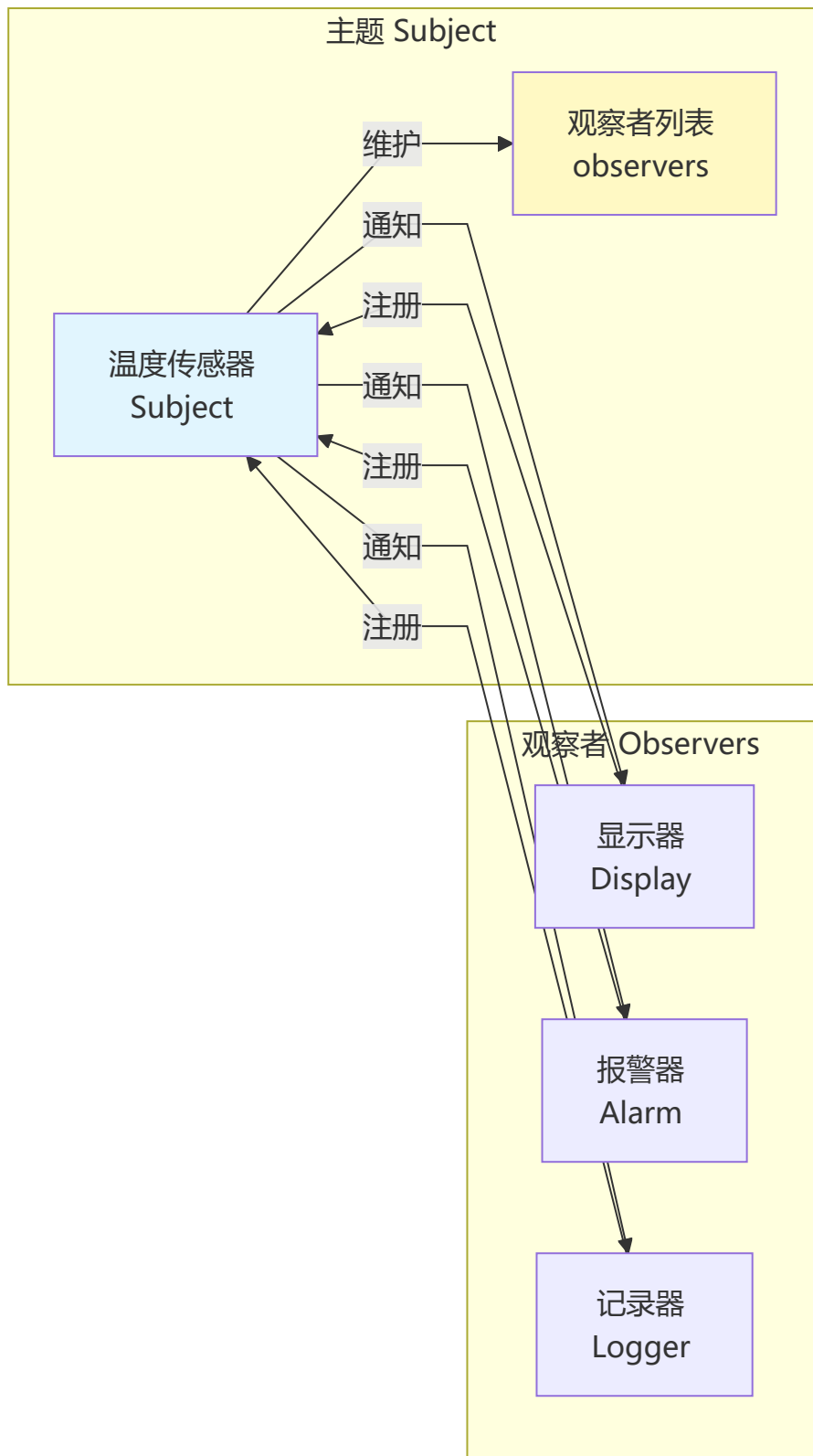
12.2.2 观察者模式（Observer）

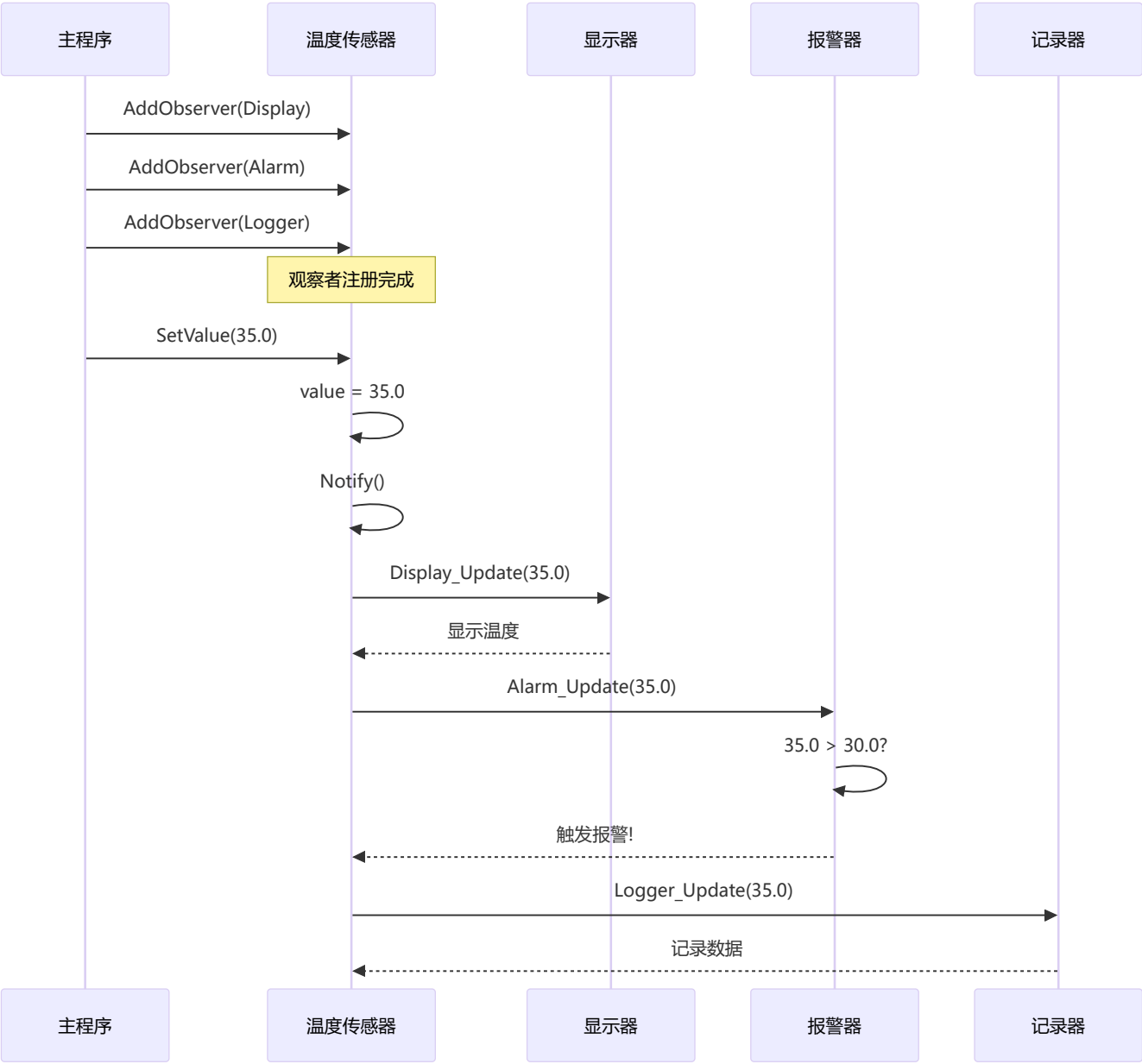
什么是观察者模式？

定义对象间的一对多依赖关系，当一个对象状态改变时，所有依赖它的对象都会收到通知。

流程图







应用场景

应用位置	说明
GUI事件	按钮点击通知多个监听器
股票行情	价格变动通知投资者
传感器监控	数据变化通知处理模块
消息推送	订阅/发布系统

代码示例

```
// observer_sensor.h - 传感器观察者模式
#ifndef OBSERVER_SENSOR_H
#define OBSERVER_SENSOR_H
```

```

#include <stdint.h>

#define MAX_OBSERVERS 10

// 观察者接口
typedef void (*observerCallback)(void *observer, float value);

// 观察者结构
typedef struct {
    void *instance;           // 观察者实例
    observerCallback update; // 更新回调
} observer;

// 被观察者（主题）
typedef struct {
    float value;              // 当前值
    observer observers[MAX_OBSERVERS]; // 观察者列表
    int observerCount;        // 观察者数量
} subject;

// 主题操作
void Subject_Init(subject *subject);
void Subject_AddObserver(subject *subject, void *instance, observerCallback callback);
void Subject_RemoveObserver(subject *subject, void *instance);
void Subject_Notify(subject *subject);
void Subject_SetValue(subject *subject, float value);

#endif

```

```

// observer_sensor.c - 观察者模式实现
#include "observer_sensor.h"
#include <string.h>

void Subject_Init(subject *subject) {
    subject->value = 0;
    subject->observerCount = 0;
    memset(subject->observers, 0, sizeof(subject->observers));
}

void Subject_AddObserver(subject *subject, void *instance, observerCallback callback) {
    if (subject->observerCount < MAX_OBSERVERS) {
        subject->observers[subject->observerCount].instance = instance;
        subject->observers[subject->observerCount].update = callback;
        subject->observerCount++;
    }
}

void Subject_RemoveObserver(subject *subject, void *instance) {
    for (int i = 0; i < subject->observerCount; i++) {
        if (subject->observers[i].instance == instance) {
            // 移动后面的观察者
            for (int j = i; j < subject->observerCount - 1; j++) {

```

```

        subject->observers[j] = subject->observers[j + 1];
    }
    subject->observerCount--;
    break;
}
}
}

void Subject_Notify(Subject *subject) {
    for (int i = 0; i < subject->observerCount; i++) {
        if (subject->observers[i].update != NULL) {
            subject->observers[i].update(subject->observers[i].instance, subject->value);
        }
    }
}

void Subject_SetValue(Subject *subject, float value) {
    subject->value = value;
    Subject_Notify(subject); // 值改变时通知所有观察者
}

```

```

// main.c - 使用示例
#include "observer_sensor.h"
#include <stdio.h>

// 具体观察者: 显示器
typedef struct {
    char name[20];
} Display;

void Display_Update(void *instance, float value) {
    Display *display = (Display*)instance;
    printf("[%s] 显示温度: %.2f°C\n", display->name, value);
}

// 具体观察者: 报警器
typedef struct {
    float threshold;
} Alarm;

void Alarm_Update(void *instance, float value) {
    Alarm *alarm = (Alarm*)instance;
    if (value > alarm->threshold) {
        printf("[报警器] 警告! 温度 %.2f°C 超过阈值 %.2f°C\n", value, alarm->threshold);
    }
}

// 具体观察者: 记录器
typedef struct {
    int recordCount;
} Logger;

```



```

void Logger_Update(void *instance, float value) {
    Logger *logger = (Logger*)instance;
    logger->recordCount++;
    printf("[记录器] 记录 #d: %.2f°C\n", logger->recordCount, value);
}

int main() {
    // 创建主题
    Subject temperatureSensor;
    Subject_Init(&temperatureSensor);

    // 创建观察者
    Display display1 = {"客厅显示器"};
    Display display2 = {"卧室显示器"};
    Alarm alarm = {30.0f};
    Logger logger = {0};

    // 注册观察者
    Subject_AddObserver(&temperatureSensor, &display1, Display_Update);
    Subject_AddObserver(&temperatureSensor, &display2, Display_Update);
    Subject_AddObserver(&temperatureSensor, &alarm, Alarm_Update);
    Subject_AddObserver(&temperatureSensor, &logger, Logger_Update);

    // 模拟温度变化
    printf("=== 温度变化 25°C ===\n");
    Subject_SetValue(&temperatureSensor, 25.0f);

    printf("\n=== 温度变化 28°C ===\n");
    Subject_SetValue(&temperatureSensor, 28.0f);

    printf("\n=== 温度变化 35°C（触发报警）===\n");
    Subject_SetValue(&temperatureSensor, 35.0f);

    return 0;
}

```

优劣势分析

优势	劣势
✅ 主题和观察者松耦合	❌ 通知顺序不确定
✅ 动态添加/删除观察者	❌ 如果观察者很多，通知耗时
✅ 符合开闭原则	❌ 观察者可能收到不关心的通知
✅ 广播通信机制	❌ 调试困难，流程不直观

选择建议

推荐使用观察者模式的场景：

- └— 一个对象变化需要通知多个对象
- └— 对象间需要松耦合
- └— 需要动态管理订阅关系
- └— 事件驱动系统

不推荐使用观察者模式的场景：

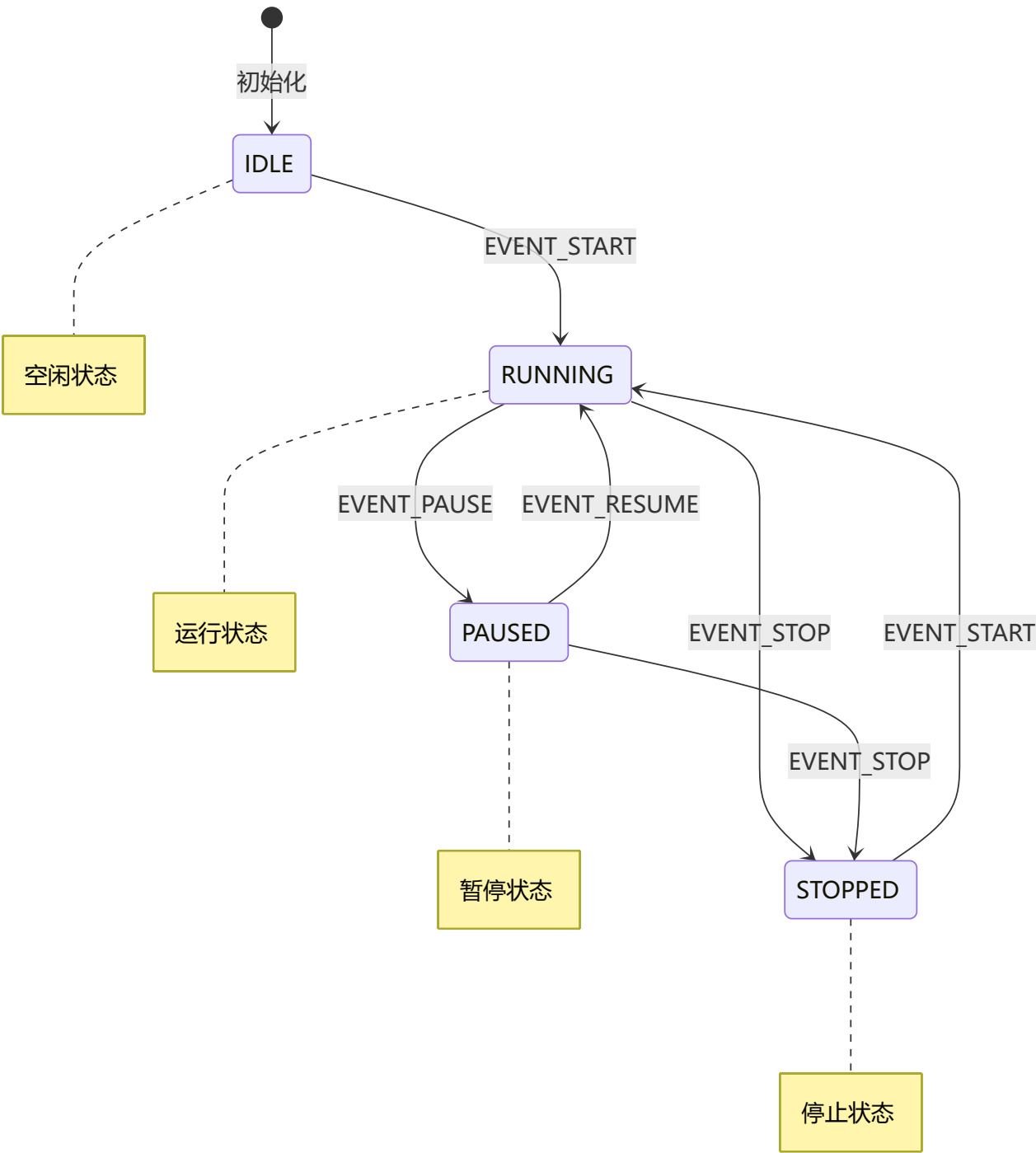
- └— 观察者数量固定且少
- └— 对实时性要求极高
- └— 不需要动态订阅
- └— 简单的通知场景

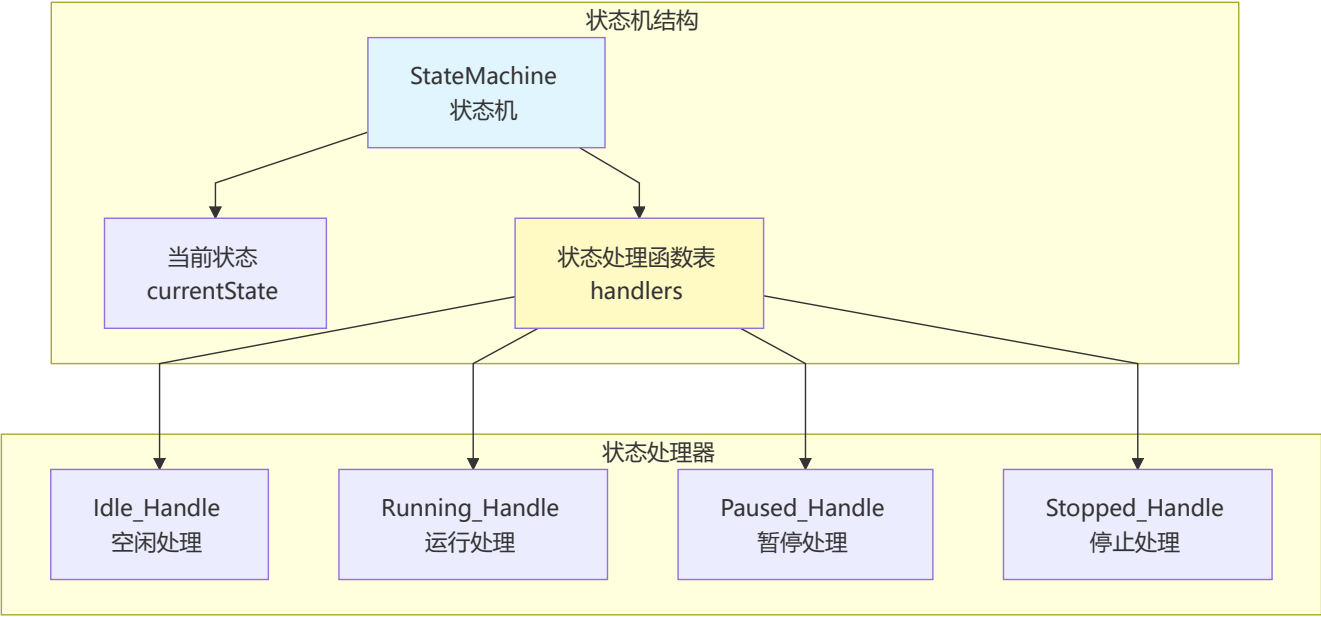
12.2.3 状态模式 (State)

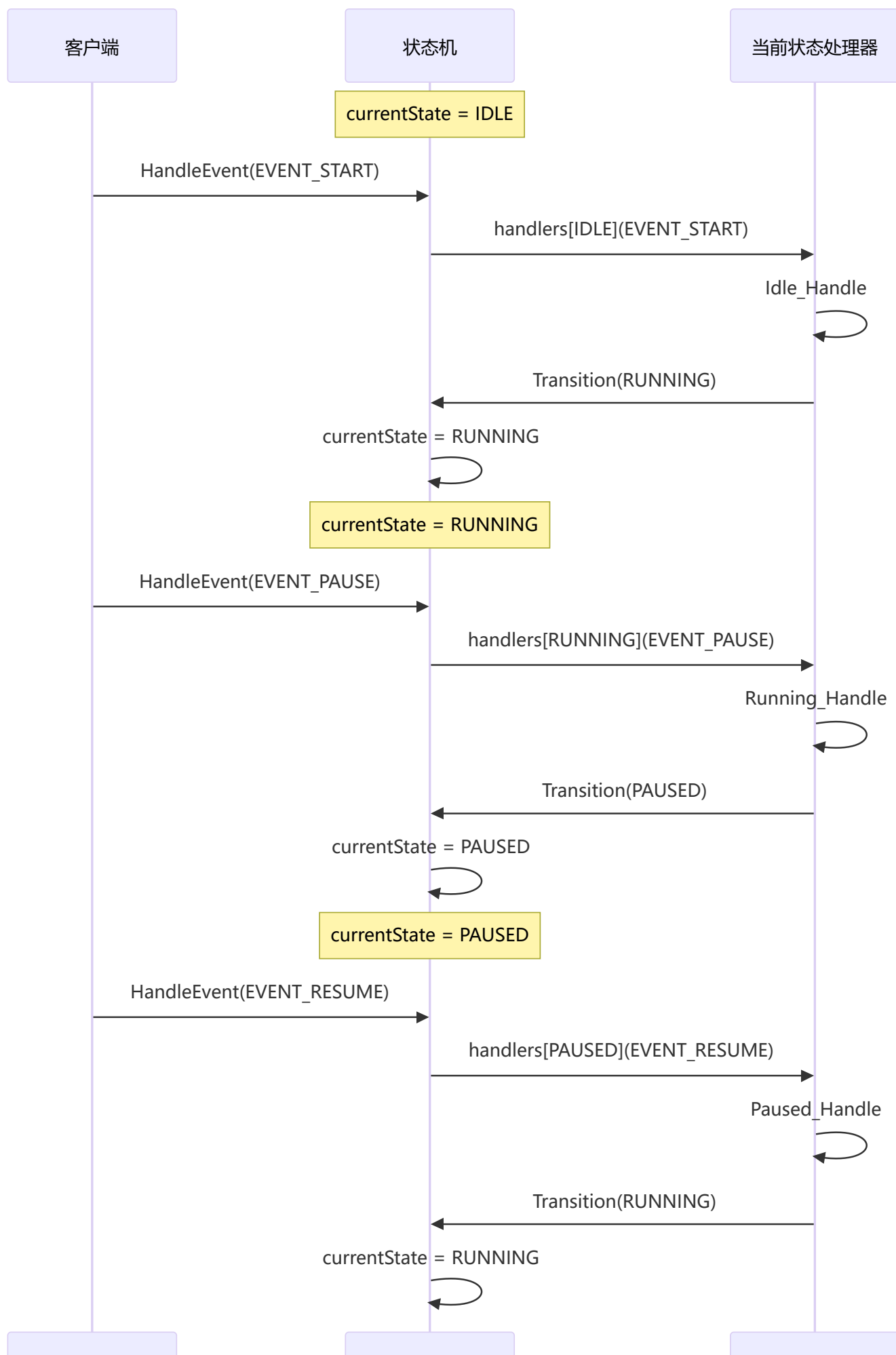
什么是状态模式？

允许对象在内部状态改变时改变其行为，对象看起来好像修改了它的类。

流程图







应用场景

应用位置	说明
状态机	协议状态、设备状态
游戏AI	角色行为状态
订单系统	订单状态流转
TCP连接	连接状态管理

代码示例

```
// state_machine.h - 状态机模式
#ifndef STATE_MACHINE_H
#define STATE_MACHINE_H

#include <stdio.h>

// 状态枚举
typedef enum {
    STATE_IDLE,
    STATE_RUNNING,
    STATE_PAUSED,
    STATE_STOPPED,
    STATE_COUNT
} StateType;

// 事件枚举
typedef enum {
    EVENT_START,
    EVENT_PAUSE,
    EVENT_RESUME,
    EVENT_STOP,
    EVENT_COUNT
} EventType;

// 状态机结构
typedef struct StateMachine StateMachine;

// 状态处理函数类型
typedef void (*StateHandler)(StateMachine *sm, EventType event);

// 状态机结构
struct StateMachine {
    StateType currentState;
    StateHandler handlers[STATE_COUNT];
    void *context; // 上下文数据
};
```

```
// 状态机操作
void StateMachine_Init(StateMachine *sm);
void StateMachine_HandleEvent(StateMachine *sm, EventType event);
void StateMachine_Transition(StateMachine *sm, StateType newState);
const char* StateMachine_GetStateName(StateType state);

#endif
```

```
// state_machine.c - 状态机实现
#include "state_machine.h"

// 状态名称
static const char *stateNames[] = {
    "空闲",
    "运行中",
    "暂停",
    "已停止"
};

const char* StateMachine_GetStateName(StateType state) {
    if (state >= 0 && state < STATE_COUNT) {
        return stateNames[state];
    }
    return "未知";
}

// ===== 各状态的处理函数 =====

void Idle_Handle(StateMachine *sm, EventType event) {
    switch (event) {
        case EVENT_START:
            printf("启动系统\n");
            StateMachine_Transition(sm, STATE_RUNNING);
            break;
        default:
            printf("空闲状态下忽略事件: %d\n", event);
            break;
    }
}

void Running_Handle(StateMachine *sm, EventType event) {
    switch (event) {
        case EVENT_PAUSE:
            printf("暂停系统\n");
            StateMachine_Transition(sm, STATE_PAUSED);
            break;
        case EVENT_STOP:
            printf("停止系统\n");
            StateMachine_Transition(sm, STATE_STOPPED);
            break;
        default:
```

```

        printf("运行状态下忽略事件: %d\n", event);
        break;
    }
}

void Paused_Handle(StateMachine *sm, EventType event) {
    switch (event) {
        case EVENT_RESUME:
            printf("恢复系统\n");
            StateMachine_Transition(sm, STATE_RUNNING);
            break;
        case EVENT_STOP:
            printf("停止系统\n");
            StateMachine_Transition(sm, STATE_STOPPED);
            break;
        default:
            printf("暂停状态下忽略事件: %d\n", event);
            break;
    }
}

void Stopped_Handle(StateMachine *sm, EventType event) {
    switch (event) {
        case EVENT_START:
            printf("重新启动系统\n");
            StateMachine_Transition(sm, STATE_RUNNING);
            break;
        default:
            printf("已停止状态下忽略事件: %d\n", event);
            break;
    }
}

// ===== 状态机操作 =====

void StateMachine_Init(StateMachine *sm) {
    sm->currentState = STATE_IDLE;
    sm->handlers[STATE_IDLE] = Idle_Handle;
    sm->handlers[STATE_RUNNING] = Running_Handle;
    sm->handlers[STATE_PAUSED] = Paused_Handle;
    sm->handlers[STATE_STOPPED] = Stopped_Handle;
    sm->context = NULL;

    printf("状态机初始化, 当前状态: %s\n", StateMachine_GetStateName(sm->currentState));
}

void StateMachine_HandleEvent(StateMachine *sm, EventType event) {
    printf("\n收到事件: ");
    switch (event) {
        case EVENT_START: printf("START"); break;
        case EVENT_PAUSE: printf("PAUSE"); break;
        case EVENT_RESUME: printf("RESUME"); break;
        case EVENT_STOP: printf("STOP"); break;
    }
}

```



```

    }
    printf("\n");

    if (sm->handlers[sm->currentState] != NULL) {
        sm->handlers[sm->currentState](sm, event);
    }
}

void StateMachine_Transition(StateMachine *sm, StateType newState) {
    printf("状态转换: %s -> %s\n",
        StateMachine_GetStateName(sm->currentState),
        StateMachine_GetStateName(newState));
    sm->currentState = newState;
}

```

```

// main.c - 使用示例
#include "state_machine.h"

int main() {
    StateMachine sm;
    StateMachine_Init(&sm);

    // 模拟事件序列
    printf("\n=== 测试正常流程 ===\n");
    StateMachine_HandleEvent(&sm, EVENT_START);    // 空闲 -> 运行
    StateMachine_HandleEvent(&sm, EVENT_PAUSE);    // 运行 -> 暂停
    StateMachine_HandleEvent(&sm, EVENT_RESUME);    // 暂停 -> 运行
    StateMachine_HandleEvent(&sm, EVENT_STOP);      // 运行 -> 停止

    printf("\n=== 测试重新启动 ===\n");
    StateMachine_HandleEvent(&sm, EVENT_START);    // 停止 -> 运行

    printf("\n=== 测试无效事件 ===\n");
    StateMachine_HandleEvent(&sm, EVENT_RESUME);    // 运行状态下RESUME无效

    return 0;
}

```

优劣势分析

优势	劣势
✓ 消除大量条件判断	✗ 状态多时代码量大
✓ 状态转换逻辑清晰	✗ 每个状态需要单独处理
✓ 易于添加新状态	✗ 状态间的数据共享不便
✓ 封装状态特定行为	✗ 可能增加类的数量

选择建议

推荐使用状态模式的场景：

- └— 对象行为取决于状态
- └— 有大量状态相关的条件判断
- └— 状态转换规则复杂
- └— 需要清晰地管理状态

不推荐使用状态模式的场景：

- └— 状态数量很少（<3个）
- └— 状态转换简单
- └— 条件判断足够清晰
- └— 不需要扩展状态

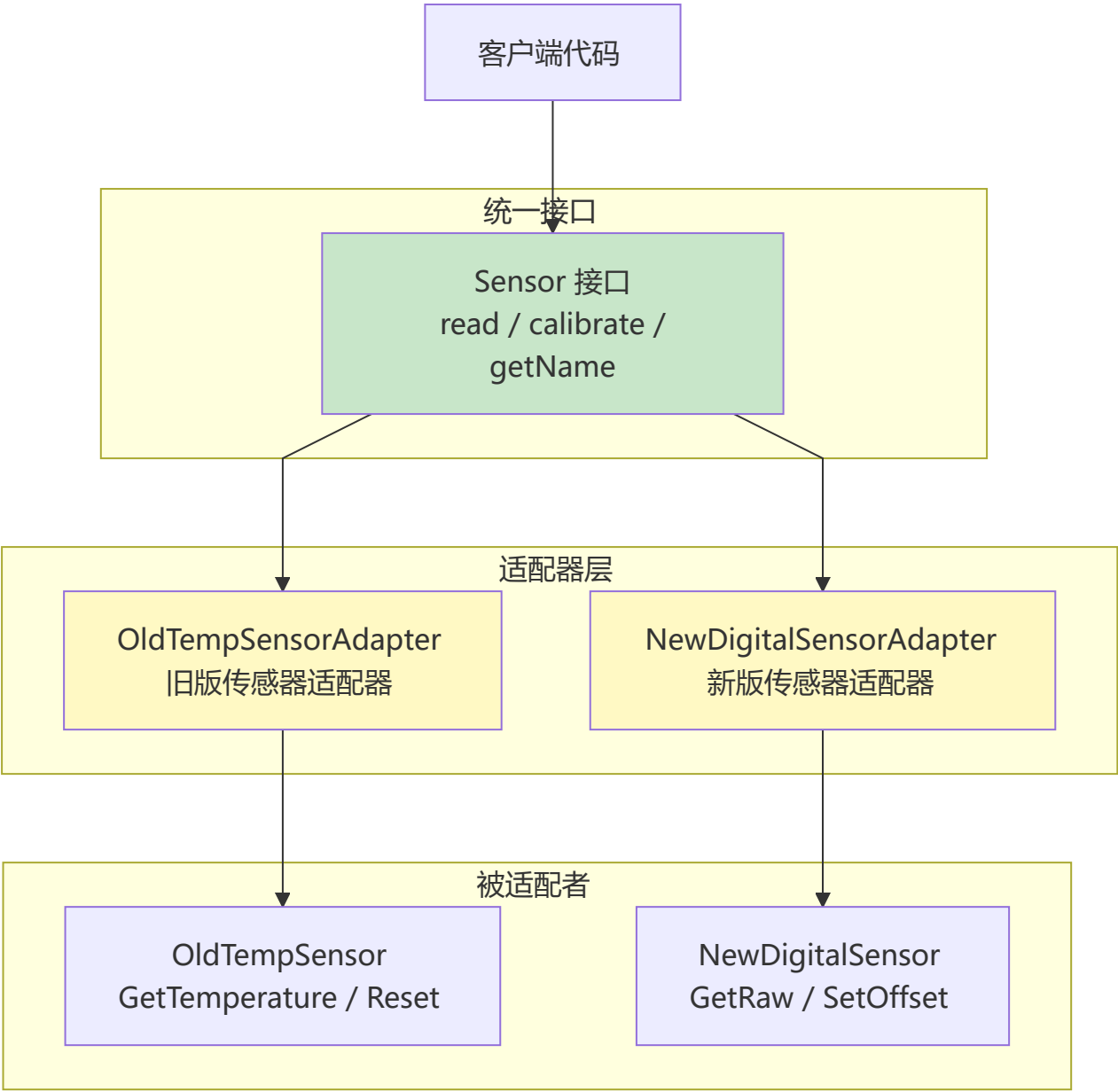
12.3 结构型模式

12.3.1 适配器模式（Adapter）

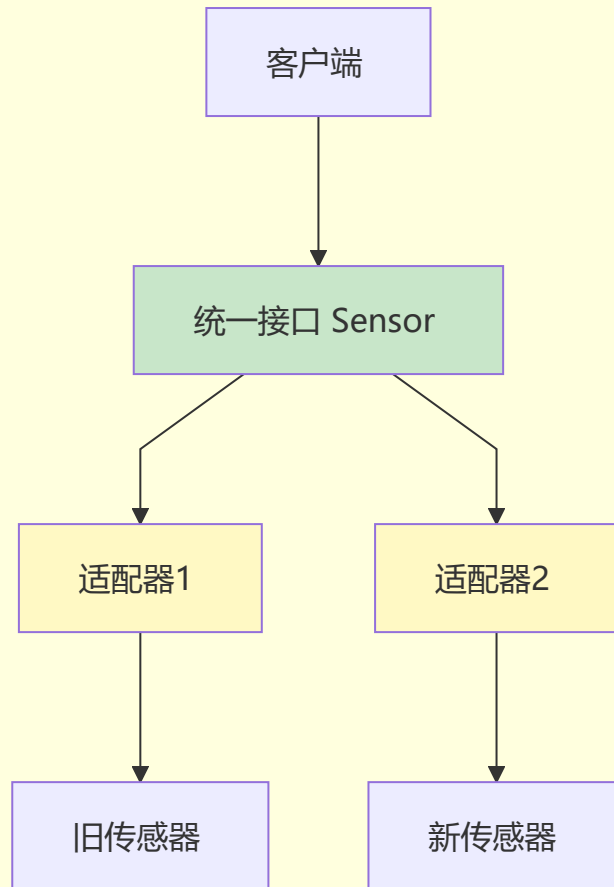
什么是适配器模式？

将一个类的接口转换成客户希望的另一个接口，使原本不兼容的类可以一起工作。

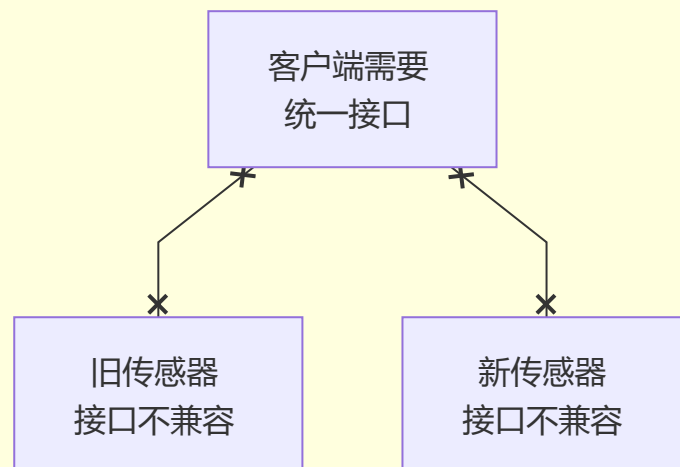
流程图

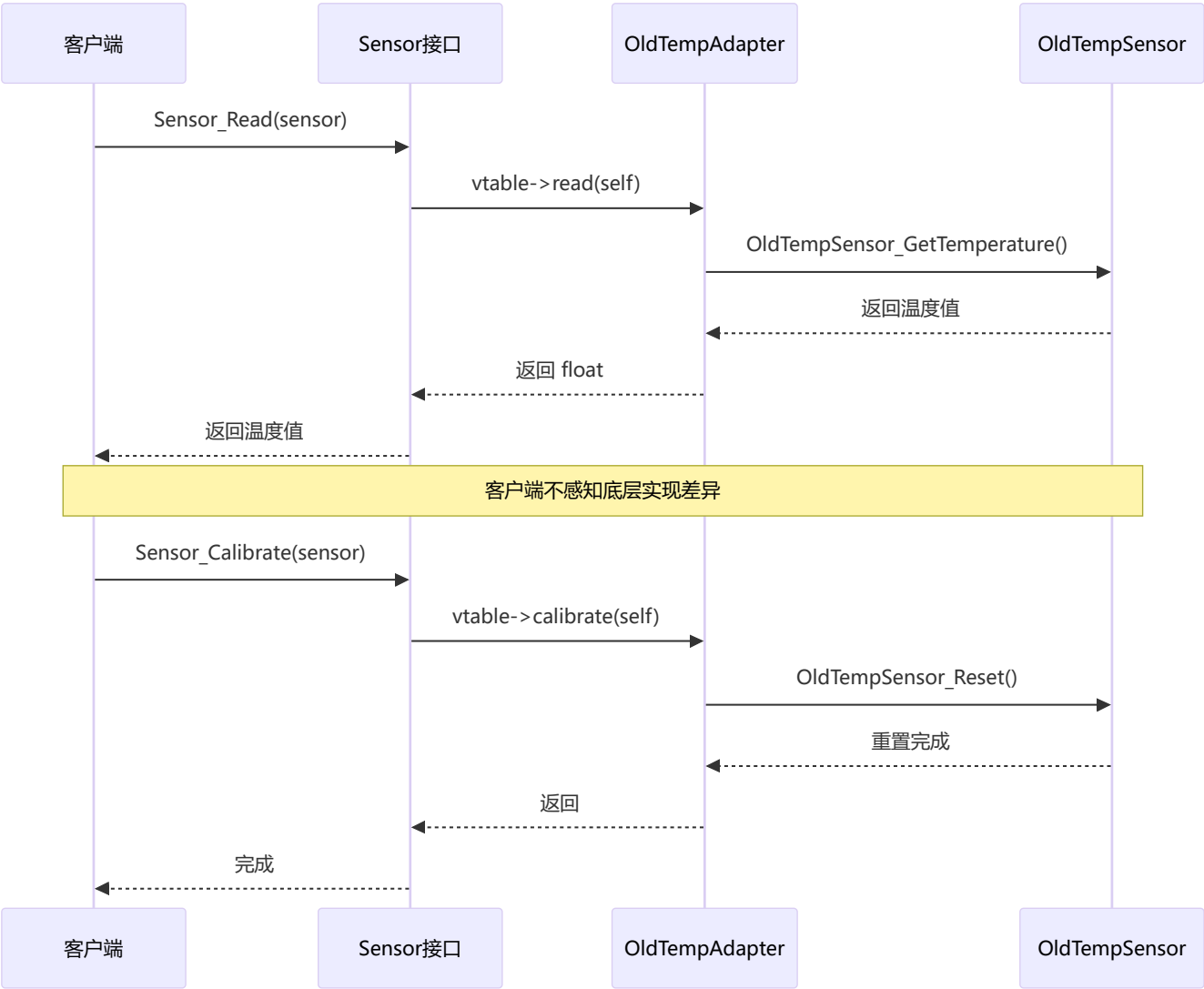


解决方案



问题





应用场景

应用位置	说明
遗留代码集成	新旧接口对接
第三方库封装	统一不同库的接口
跨平台开发	统一不同平台的API
协议转换	不同协议间的适配

代码示例

```
// adapter_sensor.h - 传感器适配器
#ifndef ADAPTER_SENSOR_H
#define ADAPTER_SENSOR_H

// ===== 目标接口（统一接口） =====
typedef struct {
    float (*read)(void *instance);
    void (*calibrate)(void *instance);
};
```

```

    const char* (*getName)(void *instance);
} SensorInterface;

typedef struct {
    const SensorInterface *vtable;
} Sensor;

float Sensor_Read(Sensor *sensor);
void Sensor_Calibrate(Sensor *sensor);
const char* Sensor_GetName(Sensor *sensor);

// ===== 被适配器A: 旧版温度传感器 =====
typedef struct {
    float lastValue;
} OldTempSensor;

void OldTempSensor_Init(OldTempSensor *sensor);
float OldTempSensor_GetTemperature(OldTempSensor *sensor);
void OldTempSensor_Reset(OldTempSensor *sensor);

// ===== 被适配器B: 新版数字传感器 =====
typedef struct {
    int rawValue;
    float offset;
} NewDigitalSensor;

void NewDigitalSensor_Init(NewDigitalSensor *sensor);
int NewDigitalSensor_GetRaw(NewDigitalSensor *sensor);
void NewDigitalSensor_SetOffset(NewDigitalSensor *sensor, float offset);

// ===== 适配器 =====
Sensor* Adapter_OldTempSensor_Create(OldTempSensor *oldSensor);
Sensor* Adapter_NewDigitalSensor_Create(NewDigitalSensor *newSensor);
void Adapter_Destroy(Sensor *sensor);

#endif

```

```

// adapter_sensor.c - 适配器实现
#include "adapter_sensor.h"
#include <stdlib.h>
#include <stdio.h>

// ===== 目标接口实现 =====
float Sensor_Read(Sensor *sensor) {
    if (sensor && sensor->vtable && sensor->vtable->read) {
        return sensor->vtable->read(sensor);
    }
    return 0;
}

void Sensor_Calibrate(Sensor *sensor) {
    if (sensor && sensor->vtable && sensor->vtable->calibrate) {

```

```

        sensor->vtable->calibrate(sensor);
    }
}

const char* Sensor_GetName(Sensor *sensor) {
    if (sensor && sensor->vtable && sensor->vtable->getName) {
        return sensor->vtable->getName(sensor);
    }
    return "Unknown";
}

// ===== 被适配器A实现 =====
void OldTempSensor_Init(OldTempSensor *sensor) {
    sensor->lastValue = 0;
}

float OldTempSensor_GetTemperature(OldTempSensor *sensor) {
    // 模拟读取
    sensor->lastValue = 25.0f + (rand() % 100) / 100.0f;
    return sensor->lastValue;
}

void OldTempSensor_Reset(OldTempSensor *sensor) {
    sensor->lastValue = 0;
    printf("旧版传感器重置\n");
}

// ===== 被适配器B实现 =====
void NewDigitalSensor_Init(NewDigitalSensor *sensor) {
    sensor->rawValue = 0;
    sensor->offset = 0;
}

int NewDigitalSensor_GetRaw(NewDigitalSensor *sensor) {
    sensor->rawValue = 2500 + (rand() % 100);
    return sensor->rawValue;
}

void NewDigitalSensor_SetOffset(NewDigitalSensor *sensor, float offset) {
    sensor->offset = offset;
    printf("新版传感器设置偏移: %.2f\n", offset);
}

// ===== 适配器A: 适配旧版传感器 =====
typedef struct {
    Sensor base;
    OldTempSensor *oldSensor;
} OldTempSensorAdapter;

static float OldTempAdapter_Read(void *instance) {
    OldTempSensorAdapter *adapter = (OldTempSensorAdapter*)instance;
    return OldTempSensor_GetTemperature(adapter->oldSensor);
}

```

```

static void OldTempAdapter_Calibrate(void *instance) {
    OldTempSensorAdapter *adapter = (OldTempSensorAdapter*)instance;
    OldTempSensor_Reset(adapter->oldSensor);
}

static const char* OldTempAdapter_GetName(void *instance) {
    return "旧版温度传感器";
}

static const SensorInterface oldTempVtable = {
    OldTempAdapter_Read,
    OldTempAdapter_Calibrate,
    OldTempAdapter_GetName
};

// ===== 适配器B: 适配新版传感器 =====
typedef struct {
    Sensor base;
    NewDigitalSensor *newSensor;
} NewDigitalSensorAdapter;

static float NewDigitalAdapter_Read(void *instance) {
    NewDigitalSensorAdapter *adapter = (NewDigitalSensorAdapter*)instance;
    int raw = NewDigitalSensor_GetRaw(adapter->newSensor);
    // 将原始值转换为温度（假设比例是0.01）
    return raw * 0.01f + adapter->newSensor->offset;
}

static void NewDigitalAdapter_Calibrate(void *instance) {
    NewDigitalSensorAdapter *adapter = (NewDigitalSensorAdapter*)instance;
    NewDigitalSensor_SetOffset(adapter->newSensor, 0.5f);
}

static const char* NewDigitalAdapter_GetName(void *instance) {
    return "新版数字传感器";
}

static const SensorInterface newDigitalVtable = {
    NewDigitalAdapter_Read,
    NewDigitalAdapter_Calibrate,
    NewDigitalAdapter_GetName
};

// ===== 适配器工厂函数 =====
Sensor* Adapter_OldTempSensor_Create(OldTempSensor *oldSensor) {
    OldTempSensorAdapter *adapter = malloc(sizeof(OldTempSensorAdapter));
    adapter->base.vtable = &oldTempVtable;
    adapter->oldSensor = oldSensor;
    return (Sensor*)adapter;
}

Sensor* Adapter_NewDigitalSensor_Create(NewDigitalSensor *newSensor) {

```



```

    NewDigitalSensorAdapter *adapter = malloc(sizeof(NewDigitalSensorAdapter));
    adapter->base.vtable = &newDigitalVtable;
    adapter->newSensor = newSensor;
    return (Sensor*)adapter;
}

void Adapter_Destroy(Sensor *sensor) {
    free(sensor);
}

```

```

// main.c - 使用示例
#include "adapter_sensor.h"
#include <stdio.h>
#include <stdlib.h>

// 统一处理函数（不关心具体传感器类型）
void ProcessSensor(Sensor *sensor) {
    printf("\n传感器: %s\n", Sensor_GetName(sensor));
    printf("读数: %.2f\n", Sensor_Read(sensor));

    printf("校准中...\n");
    Sensor_Calibrate(sensor);

    printf("校准后读数: %.2f\n", Sensor_Read(sensor));
}

int main() {
    // 创建旧版传感器
    OldTempSensor oldSensor;
    OldTempSensor_Init(&oldSensor);

    // 创建新版传感器
    NewDigitalSensor newSensor;
    NewDigitalSensor_Init(&newSensor);

    // 通过适配器统一接口
    Sensor *sensor1 = Adapter_OldTempSensor_Create(&oldSensor);
    Sensor *sensor2 = Adapter_NewDigitalSensor_Create(&newSensor);

    // 统一方式处理不同传感器
    printf("===== 处理旧版传感器 =====");
    ProcessSensor(sensor1);

    printf("\n===== 处理新版传感器 =====");
    ProcessSensor(sensor2);

    // 清理
    Adapter_Destroy(sensor1);
    Adapter_Destroy(sensor2);

    return 0;
}

```

优劣势分析

优势	劣势
✔ 复用现有代码	✘ 增加代码复杂度
✔ 解耦客户端和被适配者	✘ 过多适配器使系统凌乱
✔ 符合开闭原则	✘ 可能引入性能开销
✔ 统一不一致的接口	✘ 调试时需跟踪适配层

选择建议

推荐使用适配器模式的场景：

- └─ 需要使用现有类，但接口不匹配
- └─ 想创建可复用类，与不相关API协作
- └─ 需要统一多个类的接口
- └─ 集成遗留代码或第三方库

不推荐使用适配器模式的场景：

- └─ 接口已经统一
- └─ 可以直接修改被适配者
- └─ 不需要解耦
- └─ 系统简单，无需额外层

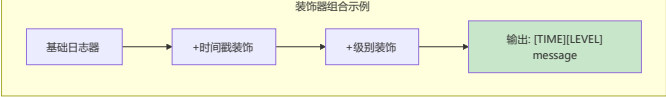
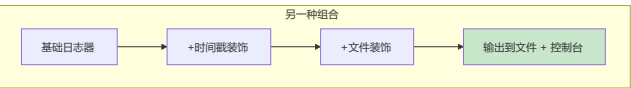
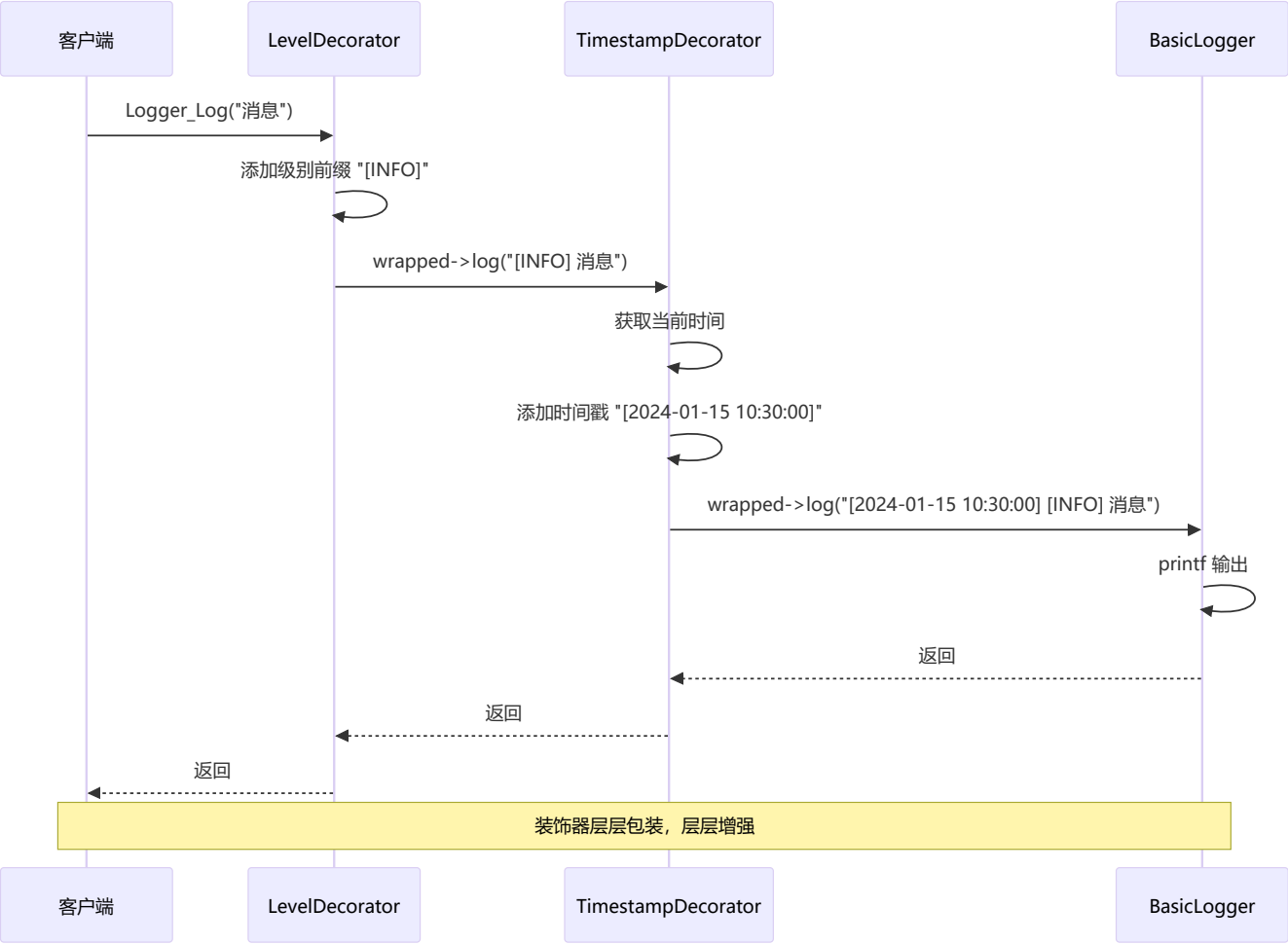
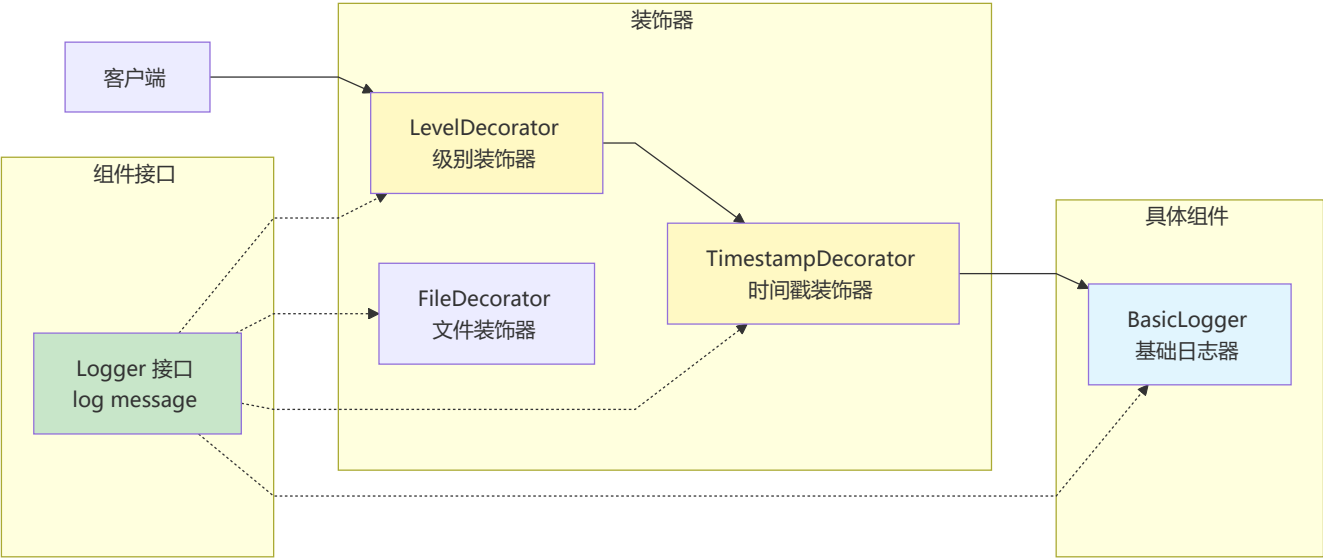
12.3.2 装饰器模式（Decorator）

什么是装饰器模式？

动态地给对象添加一些额外的职责，比生成子类更为灵活。

流程图





应用场景

应用位置	说明
I/O流	缓冲、加密、压缩
日志系统	添加时间戳、格式化
数据处理	添加验证、转换
UI组件	添加边框、滚动条

代码示例

```
// decorator_logger.h - 日志装饰器
#ifndef DECORATOR_LOGGER_H
#define DECORATOR_LOGGER_H

#include <stdio.h>
#include <time.h>

// 日志接口
typedef struct Logger Logger;

typedef void (*Logger_LogFunc)(Logger *logger, const char *message);

struct Logger {
    Logger_LogFunc log;
    Logger *wrapped; // 被装饰的日志器
};

// 基础日志器
Logger* BasicLogger_Create(void);

// 装饰器：添加时间戳
Logger* TimestampDecorator_Create(Logger *wrapped);

// 装饰器：添加日志级别
Logger* LevelDecorator_Create(Logger *wrapped, const char *level);

// 装饰器：写入文件
Logger* FileDecorator_Create(Logger *wrapped, const char *filename);

// 通用日志函数
void Logger_Log(Logger *logger, const char *message);

// 销毁
void Logger_Destroy(Logger *logger);

#endif

// decorator_logger.c - 装饰器实现
```

```

#include "decorator_logger.h"
#include <stdlib.h>
#include <string.h>

// ===== 基础日志器 =====
typedef struct {
    Logger base;
} BasicLogger;

static void BasicLogger_Log(Logger *logger, const char *message) {
    printf("%s\n", message);
}

Logger* BasicLogger_Create(void) {
    BasicLogger *logger = malloc(sizeof(BasicLogger));
    logger->base.log = BasicLogger_Log;
    logger->base.wrapped = NULL;
    return (Logger*)logger;
}

// ===== 时间戳装饰器 =====
typedef struct {
    Logger base;
} TimestampDecorator;

static void TimestampDecorator_Log(Logger *logger, const char *message) {
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char timestamp[32];
    strftime(timestamp, sizeof(timestamp), "[%Y-%m-%d %H:%M:%S]", t);

    // 先打印时间戳
    printf("%s ", timestamp);

    // 调用被装饰的日志器
    if (logger->wrapped) {
        logger->wrapped->log(logger->wrapped, message);
    }
}

Logger* TimestampDecorator_Create(Logger *wrapped) {
    TimestampDecorator *decorator = malloc(sizeof(TimestampDecorator));
    decorator->base.log = TimestampDecorator_Log;
    decorator->base.wrapped = wrapped;
    return (Logger*)decorator;
}

// ===== 日志级别装饰器 =====
typedef struct {
    Logger base;
    char level[16];
} LevelDecorator;

```

```

static void LevelDecorator_Log(Logger *logger, const char *message) {
    LevelDecorator *self = (LevelDecorator*)logger;
    printf("[%s] ", self->level);

    if (logger->wrapped) {
        logger->wrapped->log(logger->wrapped, message);
    }
}

Logger* LevelDecorator_Create(Logger *wrapped, const char *level) {
    LevelDecorator *decorator = malloc(sizeof(LevelDecorator));
    decorator->base.log = LevelDecorator_Log;
    decorator->base.wrapped = wrapped;
    strcpy(decorator->level, level);
    return (Logger*)decorator;
}

// ===== 文件装饰器 =====
typedef struct {
    Logger base;
    FILE *file;
} FileDecorator;

static void FileDecorator_Log(Logger *logger, const char *message) {
    FileDecorator *self = (FileDecorator*)logger;

    // 写入文件
    fprintf(self->file, "%s\n", message);
    fflush(self->file);

    // 同时输出到控制台
    if (logger->wrapped) {
        logger->wrapped->log(logger->wrapped, message);
    }
}

Logger* FileDecorator_Create(Logger *wrapped, const char *filename) {
    FileDecorator *decorator = malloc(sizeof(FileDecorator));
    decorator->base.log = FileDecorator_Log;
    decorator->base.wrapped = wrapped;
    decorator->file = fopen(filename, "a");
    return (Logger*)decorator;
}

// ===== 通用函数 =====
void Logger_Log(Logger *logger, const char *message) {
    if (logger && logger->log) {
        logger->log(logger, message);
    }
}

void Logger_Destroy(Logger *logger) {
    // 先销毁被装饰的对象

```

```

if (logger->wrapped) {
    Logger_Destroy(logger->wrapped);
}

// 如果是文件装饰器，关闭文件
FileDecorator *fileDec = (FileDecorator*)logger;
if (fileDec->file != stdout && fileDec->file != NULL) {
    // 检查是否是FileDecorator
    // 这里简化处理
}

free(logger);
}

```

```

// main.c - 使用示例
#include "decorator_logger.h"

int main() {
    printf("===== 基础日志器 =====\n");
    Logger *basic = BasicLogger_Create();
    Logger_Log(basic, "这是一条基础日志");
    Logger_Destroy(basic);

    printf("\n===== 带时间戳的日志器 =====\n");
    Logger *withTimestamp = TimestampDecorator_Create(BasicLogger_Create());
    Logger_Log(withTimestamp, "这是一条带时间戳的日志");
    Logger_Destroy(withTimestamp);

    printf("\n===== 带级别和时间戳的日志器 =====\n");
    Logger *logger = BasicLogger_Create();
    logger = TimestampDecorator_Create(logger);
    logger = LevelDecorator_Create(logger, "INFO");
    Logger_Log(logger, "这是一条完整的日志");
    Logger_Destroy(logger);

    printf("\n===== 不同级别的日志器 =====\n");
    Logger *debugLogger = LevelDecorator_Create(
        TimestampDecorator_Create(BasicLogger_Create()), "DEBUG");
    Logger *errorLogger = LevelDecorator_Create(
        TimestampDecorator_Create(BasicLogger_Create()), "ERROR");

    Logger_Log(debugLogger, "调试信息");
    Logger_Log(errorLogger, "错误信息");

    Logger_Destroy(debugLogger);
    Logger_Destroy(errorLogger);

    return 0;
}

```

优劣势分析

优势	劣势
✔ 动态添加功能	✘ 增加系统复杂度
✔ 比继承更灵活	✘ 装饰器嵌套过多难以理解
✔ 可以组合多个装饰器	✘ 调试困难
✔ 符合单一职责原则	✘ C语言实现较为繁琐

选择建议

推荐使用装饰器模式的场景： └─ 需要动态添加/撤销功能 └─ 需要组合多种功能 └─ 不能使用继承扩展 └─ 功能可以任意组合 不推荐使用装饰器模式的场景： └─ 功能固定，不需要组合 └─ 可以使用简单的继承 └─ 装饰器数量过多 └─ 团队不熟悉这种模式
--

12.4 设计模式选择速查表

场景	推荐模式	说明
全局唯一实例	单例模式	配置、日志、硬件驱动
根据条件创建对象	工厂模式	跨平台、多类型对象
算法需要切换	策略模式	排序、压缩、加密算法
一对多通知	观察者模式	事件系统、消息推送
状态决定行为	状态模式	状态机、协议处理
接口不兼容	适配器模式	遗留代码、第三方库集成
动态添加功能	装饰器模式	日志格式化、数据处理

第十二章小结

本章要点

- 1. C语言虽然是面向过程的，但同样可以实现面向对象的设计模式
- 2. 函数指针是C语言实现多态和策略的核心
- 3. 结构体和指针组合可以实现封装和继承
- 4. 选择设计模式要根据实际需求，不要过度设计

设计模式应用建议

选择设计模式的步骤：

- 1. 分析问题
 - ├─ 确定变化点（什么会变？）
 - └─ 确定稳定点（什么不变？）
- 2. 匹配模式
 - ├─ 变化在创建 → 创建型模式
 - ├─ 变化在行为 → 行为型模式
 - └─ 变化在结构 → 结构型模式
- 3. 评估代价
 - ├─ 复杂度增加是否值得？
 - ├─ 团队是否理解？
 - └─ 维护成本如何？
- 4. 简化实现
 - ├─ 不要生搬硬套
 - ├─ 结合C语言特点
 - └─ 适当简化

动手练习

- 练习1：**为单例日志器添加多线程保护。
- 练习2：**实现一个简单的插件框架，使用工厂模式加载不同的插件。
- 练习3：**用状态模式实现一个简单的TCP连接状态机。



框架与设计模式速查表

类型	名称	应用场景	优势	劣势
框架	HAL硬件抽象层	嵌入式跨平台	可移植性强	性能有损耗
框架	事件驱动	GUI、物联网	解耦、灵活	调试困难
框架	模块化	大型项目	职责清晰	配置复杂

类型	名称	应用场景	优势	劣势
框架	配置管理	多版本产品	灵活裁剪	需重新编译
框架	内存池	实时系统	无碎片、确定时间	块大小固定
模式	单例	日志、配置	全局访问	难以测试
模式	工厂	跨平台开发	解耦创建	增加复杂度
模式	策略	算法切换	易于扩展	客户端需知策略
模式	观察者	事件系统	松耦合	通知顺序不确定
模式	状态	状态机	消除条件判断	状态多代码量大
模式	适配器	遗留代码集成	复用现有代码	系统凌乱
模式	装饰器	功能增强	动态组合	调试困难

第十三章：GPIO（通用输入输出）

GPIO是单片机最基础的外设，是与外部世界交互的第一步

13.1 什么是GPIO？

一句话理解

GPIO（General Purpose Input/Output）是**通用输入输出引脚**，可以配置为输入或输出，用于控制外部设备或读取外部信号。

生活类比

GPIO = 开关 + 传感器

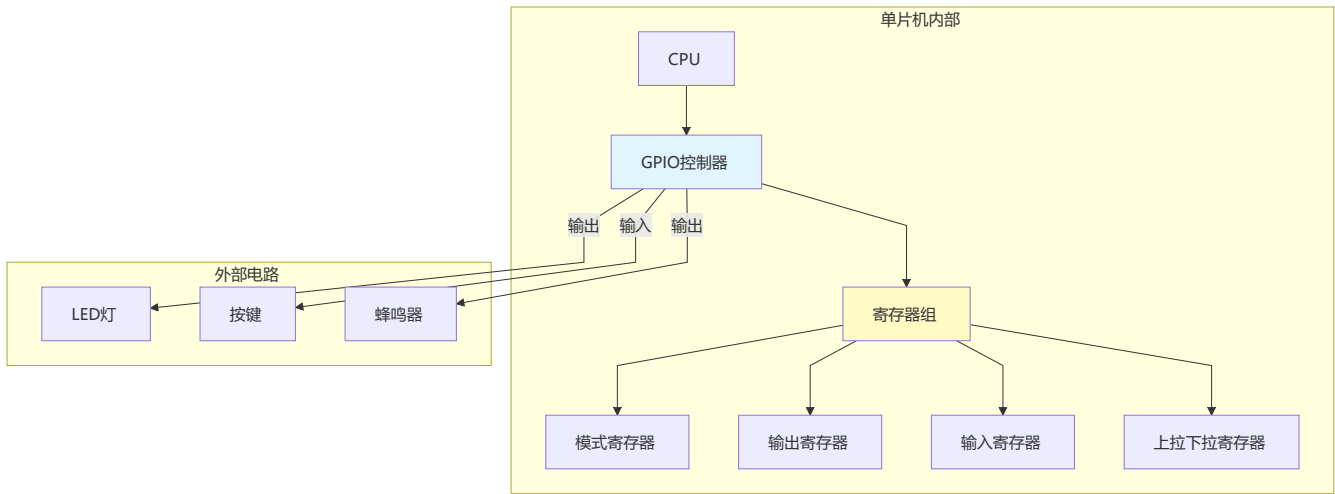
输出模式：

- GPIO = 开关 → 控制LED亮灭
- GPIO = 开关 → 控制继电器
- GPIO = 开关 → 驱动蜂鸣器

输入模式：

- GPIO = 传感器 → 读取按键状态
- GPIO = 传感器 → 读取开关状态
- GPIO = 传感器 → 接收外部信号

GPIO结构图



13.2 GPIO工作模式

输入模式

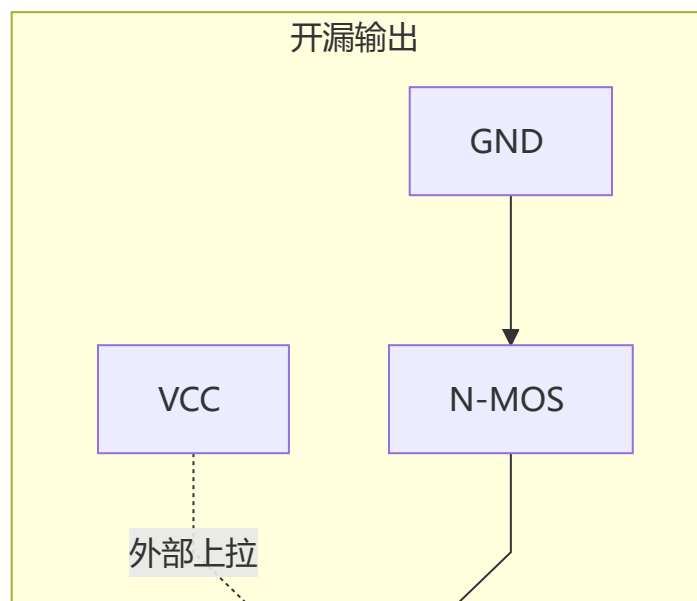
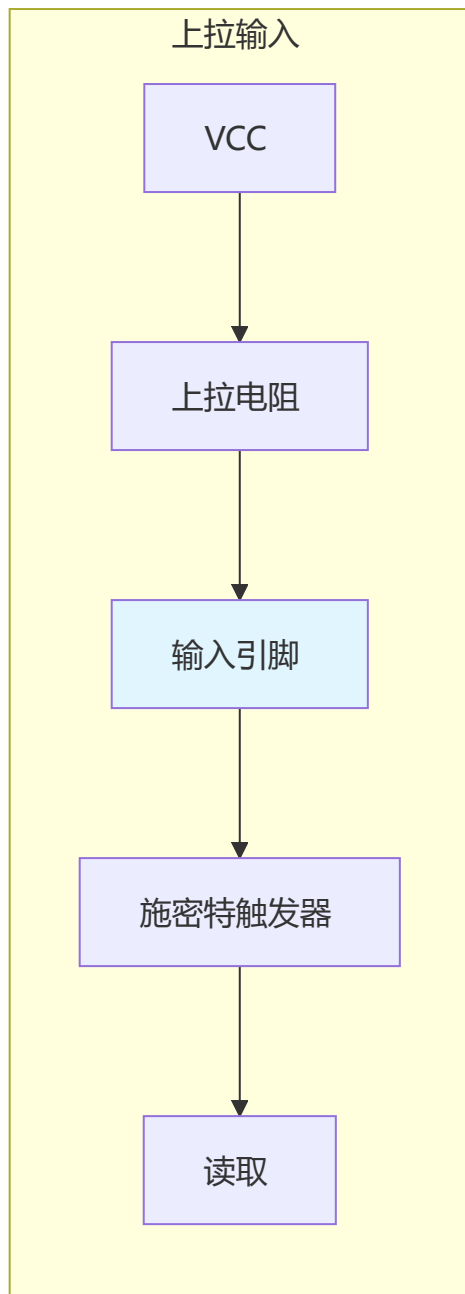
模式	说明	应用场景
浮空输入	引脚悬空，电平不确定	外部有上拉/下拉电路时
上拉输入	内部上拉电阻，默认高电平	按键检测（按键接地）
下拉输入	内部下拉电阻，默认低电平	按键检测（按键接VCC）
模拟输入	用于ADC采集	读取模拟信号

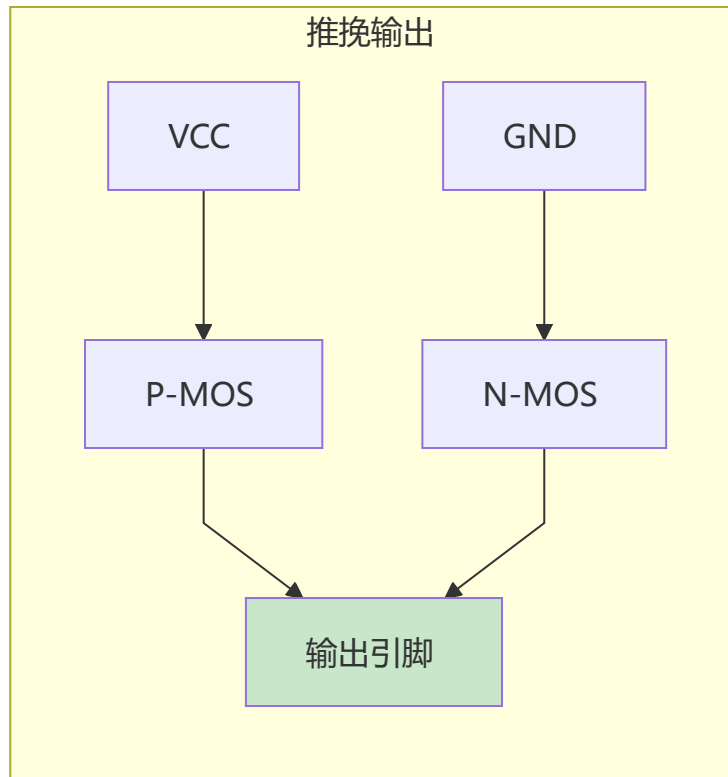
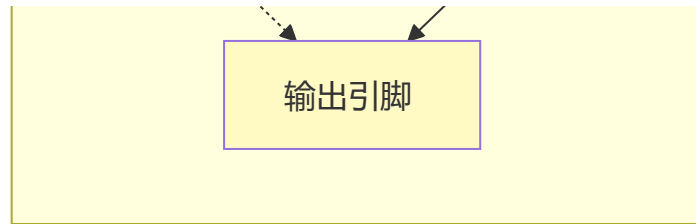
输出模式

模式	说明	应用场景
推挽输出	可输出高/低电平，驱动能力强	LED、蜂鸣器、继电器
开漏输出	只能拉低，需要外部上拉	I2C通信、电平转换
复用推挽	外设功能输出	UART TX、SPI MOSI
复用开漏	外设功能开漏输出	I2C SDA/SCL

模式对比图

模式对比图





13.3 GPIO基本操作

寄存器操作（以STM32为例）

```
// STM32 GPIO寄存器结构
typedef struct {
    volatile uint32_t CRL;    // 配置寄存器低（PIN0-PIN7）
    volatile uint32_t CRH;    // 配置寄存器高（PIN8-PIN15）
    volatile uint32_t IDR;    // 输入数据寄存器
    volatile uint32_t ODR;    // 输出数据寄存器
    volatile uint32_t BSRR;   // 置位/复位寄存器
    volatile uint32_t BRR;    // 复位寄存器
    volatile uint32_t LCKR;   // 锁定寄存器
} GPIO_TypeDef;

// GPIO模式定义（CRL/CRH寄存器）
#define GPIO_MODE_INPUT        0x00    // 输入模式
#define GPIO_MODE_OUTPUT_10MHz 0x01    // 输出模式，最大10MHz
#define GPIO_MODE_OUTPUT_2MHz  0x02    // 输出模式，最大2MHz
#define GPIO_MODE_OUTPUT_50MHz 0x03    // 输出模式，最大50MHz
```

```
// 推挽/开漏配置
#define GPIO_CNF_OUT_PP      0x00    // 推挽输出
#define GPIO_CNF_OUT_OD      0x01    // 开漏输出
#define GPIO_CNF_OUT_AF_PP    0x02    // 复用推挽
#define GPIO_CNF_OUT_AF_OD    0x03    // 复用开漏

// 输入配置
#define GPIO_CNF_IN_ANALOG    0x00    // 模拟输入
#define GPIO_CNF_IN_FLOAT     0x01    // 浮空输入
#define GPIO_CNF_IN_PU_PD     0x02    // 上拉/下拉输入
```

点亮LED

```
#include "stm32f10x.h"

// 方法1: 直接操作寄存器
void LED_Init_Register(void) {
    // 开启GPIOA时钟
    RCC->APB2ENR |= (1 << 2);

    // 配置PA5为推挽输出, 50MHz
    GPIOA->CRL &= ~(0x0F << (5 * 4));    // 清除原配置
    GPIOA->CRL |= (0x03 << (5 * 4));    // 输出模式50MHz
    GPIOA->CRL |= (0x00 << (5 * 4 + 2));    // 推挽输出

    // 设置PA5输出高电平 (LED亮)
    GPIOA->BSRR = (1 << 5);    // 置位

    // 设置PA5输出低电平 (LED灭)
    GPIOA->BRR = (1 << 5);    // 复位
}

// 方法2: 使用HAL库
void LED_Init_HAL(void) {
    GPIO_InitTypeDef GPIO_InitStructure = {0};

    // 开启GPIOA时钟
    __HAL_RCC_GPIOA_CLK_ENABLE();

    // 配置PA5
    GPIO_InitStructure.Pin = GPIO_PIN_5;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;    // 推挽输出
    GPIO_InitStructure.Pull = GPIO_NOPULL;    // 无上下拉
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;    // 高速
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 点亮LED
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);    // 高电平
    // 熄灭LED
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);    // 低电平
}
```

```
// 方法3: LED闪烁
void LED_Blink(void) {
    while (1) {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
        HAL_Delay(500); // 延时500ms
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
        HAL_Delay(500);
    }
}

// 方法4: LED翻转
void LED_Toggle(void) {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
}
```

读取按键

```
#include "stm32f10x.h"

// 按键初始化 (PA0, 上拉输入)
void Key_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure = {0};

    __HAL_RCC_GPIOA_CLK_ENABLE();

    GPIO_InitStructure.Pin = GPIO_PIN_0;
    GPIO_InitStructure.Mode = GPIO_MODE_INPUT; // 输入模式
    GPIO_InitStructure.Pull = GPIO_PULLUP; // 上拉
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
}

// 读取按键状态
uint8_t Key_Read(void) {
    // 按键按下时为低电平 (按键接地)
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET) {
        // 消抖处理
        HAL_Delay(20);
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET) {
            // 等待按键释放
            while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET);
            return 1; // 按键按下
        }
    }
    return 0; // 按键未按下
}

// 按键控制LED
void Key_Control_LED(void) {
    Key_Init();
    LED_Init_HAL();

    while (1) {
```



```
        if (key_Read()) {
            LED_Toggle(); // 按键按下，翻转LED
        }
    }
}
```

驱动蜂鸣器

```
#include "stm32f10x.h"

// 蜂鸣器初始化（PA6，推挽输出）
void Buzzer_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure = {0};

    __HAL_RCC_GPIOA_CLK_ENABLE();

    GPIO_InitStructure.Pin = GPIO_PIN_6;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

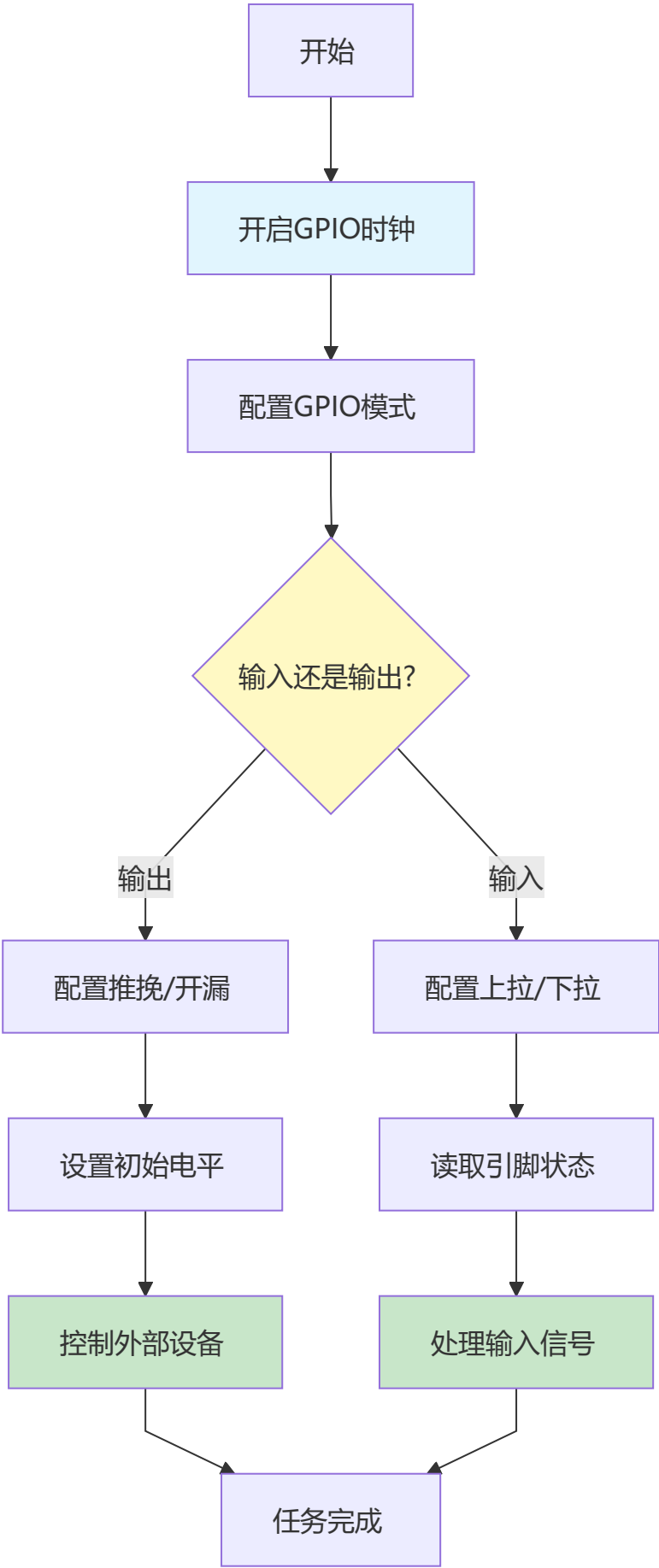
    // 初始状态：关闭
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
}

// 蜂鸣器控制
void Buzzer_On(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
}

void Buzzer_Off(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
}

// 蜂鸣器鸣叫N次
void Buzzer_Beep(uint8_t times) {
    for (uint8_t i = 0; i < times; i++) {
        Buzzer_On();
        HAL_Delay(100);
        Buzzer_Off();
        HAL_Delay(100);
    }
}
```

13.4 GPIO应用流程图



13.5 GPIO配置速查表

需求	模式	上下拉	说明
点亮LED	推挽输出	无	高电平亮或低电平亮取决于电路
读取按键	输入	上拉	按键接地，按下为低电平
I2C通信	开漏输出	外部上拉	需要外部上拉电阻
SPI MOSI	复用推挽	无	高速通信
UART TX	复用推挽	无	异步串口发送
模拟信号	模拟输入	无	连接到ADC

第十三章小结

本章要点

- 1. GPIO是单片机与外部世界交互的基础
- 2. 输入模式：浮空、上拉、下拉、模拟输入
- 3. 输出模式：推挽、开漏、复用功能
- 4. 推挽输出驱动能力强，开漏输出适合总线通信
- 5. 输入时注意消抖处理

动手练习

- 练习1：**实现流水灯（8个LED依次点亮）。
- 练习2：**实现长按按键功能（短按闪烁，长按常亮）。
- 练习3：**实现按键控制蜂鸣器鸣叫次数。

第十四章：中断系统

中断是单片机实时响应外部事件的核心机制

14.1 什么是中断？

一句话理解

中断是CPU暂停当前工作，去处理紧急事件，处理完后继续原工作的机制。

生活类比

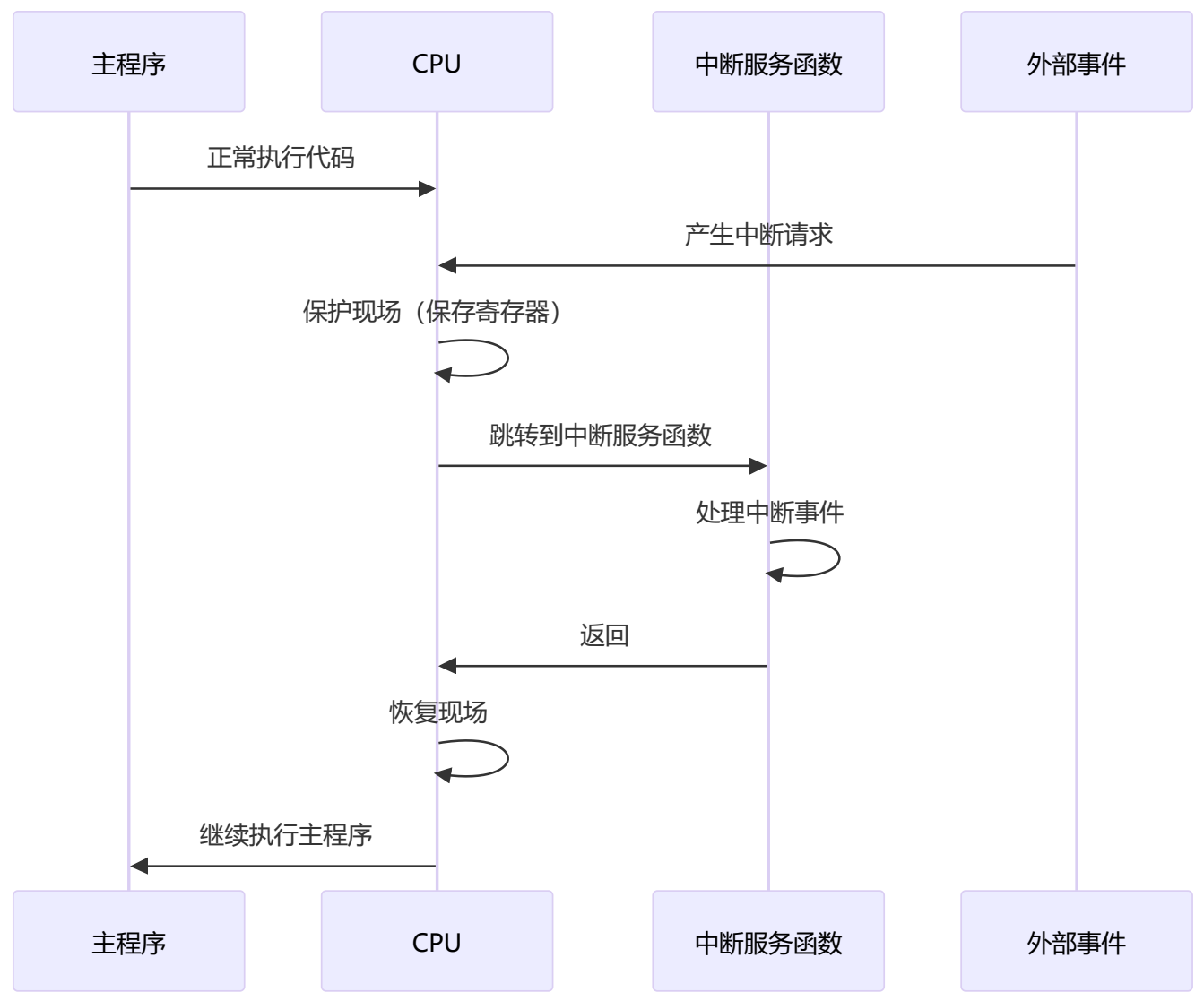
中断 = 上课时的突发情况

正常流程：
老师讲课 → 讲课 → 讲课 → 下课

有中断的情况：
老师讲课 → [有人敲门] → 暂停讲课 → 处理敲门 → 继续讲课

└─ 讲课 = 主程序
└─ 敲门 = 中断源
└─ 暂停讲课 = 保护现场
└─ 处理敲门 = 中断服务函数
└─ 继续讲课 = 恢复现场

中断流程图



14.2 中断类型

外部中断

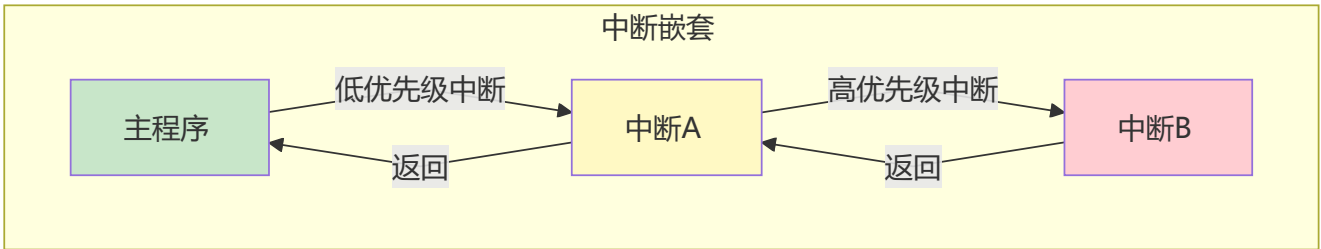
中断源	触发方式	应用场景
GPIO电平变化	上升沿/下降沿/双边沿	按键检测、传感器信号
外部引脚	电平触发	外部设备通知

内部中断

中断源	触发条件	应用场景
定时器中断	计数溢出	定时任务、PWM
串口中断	接收/发送完成	数据通信
ADC中断	转换完成	模拟信号采集
DMA中断	传输完成	高效数据传输

14.3 中断优先级

优先级概念



STM32中断优先级

```
// STM32使用NVIC管理中断优先级
// 抢占优先级：高优先级可打断低优先级
// 响应优先级：抢占优先级相同时，响应优先级高的先执行

// 优先级分组
typedef enum {
    NVIC_PriorityGroup_0 = 0, // 0位抢占，4位响应
    NVIC_PriorityGroup_1,     // 1位抢占，3位响应
    NVIC_PriorityGroup_2,     // 2位抢占，2位响应（常用）
    NVIC_PriorityGroup_3,     // 3位抢占，1位响应
    NVIC_PriorityGroup_4      // 4位抢占，0位响应
} NVIC_PriorityGroup;
```

```
// 配置中断优先级
void NVIC_Config(void) {
    NVIC_InitTypeDef NVIC_InitStruct;

    // 设置优先级分组
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

    // 配置外部中断0
    NVIC_InitStruct.NVIC_IRQChannel = EXTI0_IRQn;
    NVIC_InitStruct.NVIC_IRQChannelPreemptionPriority = 1; // 抢占优先级
    NVIC_InitStruct.NVIC_IRQChannelSubPriority = 1; // 响应优先级
    NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStruct);
}
```

14.4 外部中断实例

按键触发中断

```
#include "stm32f10x.h"

// 外部中断初始化（PA0连接按键）
void EXTI_Config(void) {
    GPIO_InitTypeDef GPIO_InitStruct;
    EXTI_InitTypeDef EXTI_InitStruct;
    NVIC_InitTypeDef NVIC_InitStruct;

    // 开启时钟
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_AFIO_CLK_ENABLE();

    // 配置GPIO为输入
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_IPU; // 上拉输入
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    // 配置外部中断线
    EXTI_InitStruct.EXTI_Line = EXTI_Line0;
    EXTI_InitStruct.EXTI_Mode = EXTI_Mode_Interrupt;
    EXTI_InitStruct.EXTI_Trigger = EXTI_Trigger_Falling; // 下降沿触发
    EXTI_InitStruct.EXTI_LineCmd = ENABLE;
    EXTI_Init(&EXTI_InitStruct);

    // 连接GPIO到中断线
    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource0);

    // 配置NVIC
    NVIC_InitStruct.NVIC_IRQChannel = EXTI0_IRQn;
    NVIC_InitStruct.NVIC_IRQChannelPreemptionPriority = 1;
    NVIC_InitStruct.NVIC_IRQChannelSubPriority = 1;
    NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
}
```

```

    NVIC_Init(&NVIC_InitStruct);
}

// 中断服务函数
volatile uint8_t keyPressed = 0;

void EXTI0_IRQHandler(void) {
    // 判断是否是中断线0触发的中断
    if (EXTI_GetITStatus(EXTI_Line0) != RESET) {
        keyPressed = 1; // 设置标志位

        // 清除中断标志
        EXTI_ClearITPendingBit(EXTI_Line0);
    }
}

// 主程序
int main(void) {
    HAL_Init();
    EXTI_Config();
    LED_Init_HAL();

    while (1) {
        if (keyPressed) {
            keyPressed = 0;
            LED_Toggle(); // 按键按下，翻转LED
        }
    }
}

```

14.5 定时器中断实例

```

#include "stm32f10x.h"

// 定时器初始化（1ms定时中断）
void TIM_Config(void) {
    TIM_TimeBaseInitTypeDef TIM_InitStruct;
    NVIC_InitTypeDef NVIC_InitStruct;

    // 开启定时器时钟
    __HAL_RCC_TIM2_CLK_ENABLE();

    // 定时器基础配置
    // 系统时钟72MHz，预分频72，则定时器频率1MHz
    // 计数1000次，则定时周期1ms
    TIM_InitStruct.TIM_Prescaler = 72 - 1;
    TIM_InitStruct.TIM_Period = 1000 - 1;
    TIM_InitStruct.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_InitStruct.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInit(TIM2, &TIM_InitStruct);
}

```

```
// 使能更新中断
TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);

// 配置NVIC
NVIC_InitStruct.NVIC_IRQChannel = TIM2_IRQn;
NVIC_InitStruct.NVIC_IRQChannelPreemptionPriority = 2;
NVIC_InitStruct.NVIC_IRQChannelSubPriority = 0;
NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStruct);

// 启动定时器
TIM_Cmd(TIM2, ENABLE);
}

// 全局变量
volatile uint32_t msCount = 0;

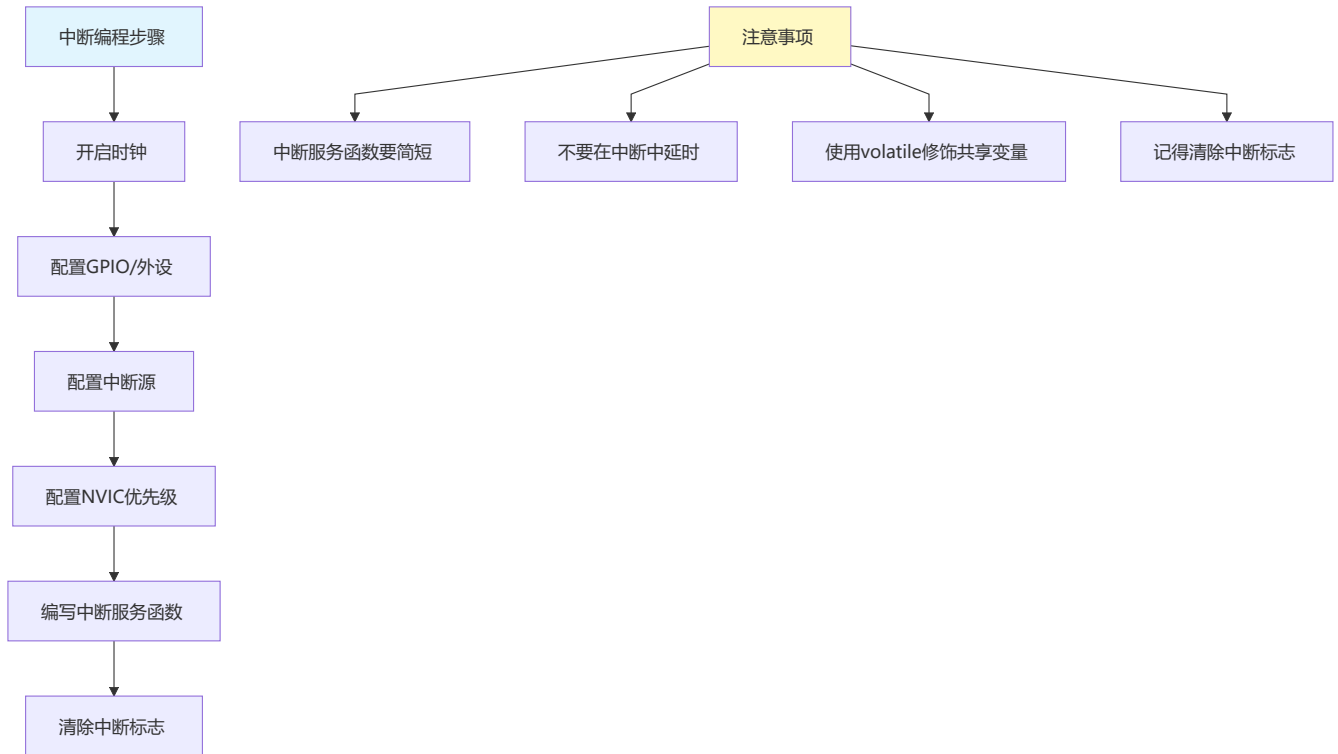
// 定时器中断服务函数
void TIM2_IRQHandler(void) {
    if (TIM_GetITStatus(TIM2, TIM_IT_Update) != RESET) {
        msCount++; // 毫秒计数

        TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    }
}

// 获取毫秒时间
uint32_t Get_Tick(void) {
    return msCount;
}

// 延时函数（非阻塞式）
void Delay_NonBlocking(uint32_t ms) {
    uint32_t start = Get_Tick();
    while ((Get_Tick() - start) < ms);
}
```

14.6 中断编程要点



中断编程注意事项

```
// 1. 共享变量使用volatile
volatile uint8_t flag = 0; // 必须用volatile

// 2. 中断服务函数要简短
void EXTI0_IRQHandler(void) {
    // ✅ 正确：只设置标志位
    flag = 1;
    EXTI_ClearITPendingBit(EXTI_Line0);

    // ❌ 错误：在中断中延时或做复杂处理
    // HAL_Delay(100);           // 不要这样做！
    // ProcessData();           // 不要这样做！
}

// 3. 主循环处理业务逻辑
int main(void) {
    while (1) {
        if (flag) {
            flag = 0;
            ProcessData(); // 在主循环中处理
        }
    }
}

// 4. 关键代码段保护
void CriticalSection(void) {
```

```
__disable_irq();    // 关闭中断
// 关键代码
__enable_irq();     // 开启中断
}
```

第十四章小结

本章要点

- 中断是实时响应外部事件的核心机制
- 中断优先级决定响应顺序和嵌套
- 中断服务函数要简短高效
- 必须清除中断标志位
- 共享变量使用volatile修饰

动手练习

练习1: 使用外部中断实现按键消抖。

练习2: 使用定时器中断实现LED闪烁。

练习3: 实现多中断嵌套处理。

第十五章：定时器

定时器是单片机的"心脏"，用于精确计时和波形生成

15.1 什么是定时器？

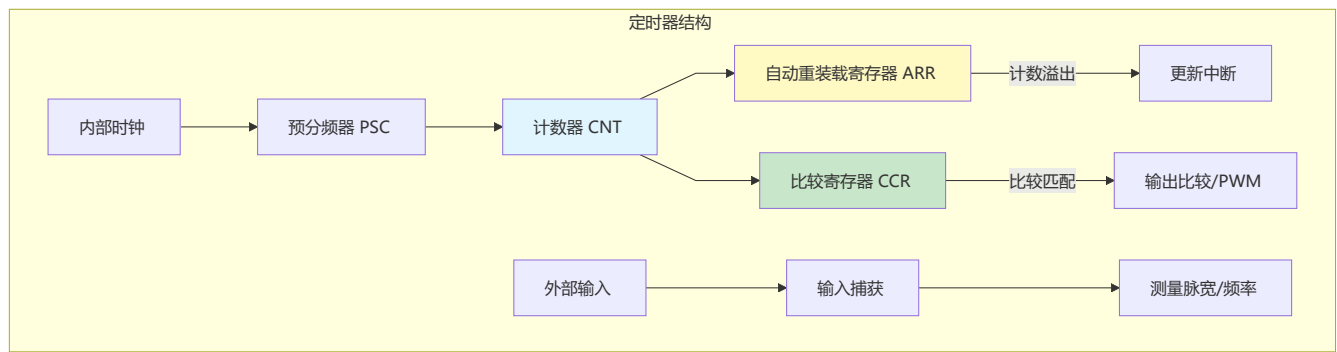
一句话理解

定时器是单片机内部的计数器，可以对时钟信号进行计数，实现定时、PWM输出、输入捕获等功能。

定时器类型

类型	说明	应用场景
基本定时器	最简单的定时功能	定时中断、DAC触发
通用定时器	功能丰富，支持PWM、捕获	电机控制、测量脉宽
高级定时器	功能最全，支持死区控制	电机驱动、电源控制

定时器结构图



15.2 定时器基本应用

定时中断

```
#include "stm32f10x.h"

// 定时器配置（定时1秒）
void TIM_Config_1s(void) {
    TIM_TimeBaseInitTypeDef TIM_InitStruct;

    __HAL_RCC_TIM2_CLK_ENABLE();

    // 计算：
    // 系统时钟72MHz
    // 预分频7200，则定时器频率10kHz（10000Hz）
    // 计数10000次，则定时周期1秒
    TIM_InitStruct.TIM_Prescaler = 7200 - 1;
    TIM_InitStruct.TIM_Period = 10000 - 1;
    TIM_InitStruct.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM2, &TIM_InitStruct);

    TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);

    // NVIC配置
    NVIC_EnableIRQ(TIM2_IRQn);

    TIM_Cmd(TIM2, ENABLE);
}

// 1秒定时中断
void TIM2_IRQHandler(void) {
    if (TIM_GetITStatus(TIM2, TIM_IT_Update)) {
        // 每秒执行一次
        LED_Toggle();

        TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    }
}
```

```
}
```

微秒级延时

```
// 使用定时器实现微秒延时
void Delay_us(uint32_t us) {
    // 配置定时器
    TIM_TimeBaseInitTypeDef TIM_InitStruct;

    __HAL_RCC_TIM3_CLK_ENABLE();

    TIM_InitStruct.TIM_Prescaler = 72 - 1; // 1MHz
    TIM_InitStruct.TIM_Period = us;        // 计数us次
    TIM_InitStruct.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM3, &TIM_InitStruct);

    TIM_Cmd(TIM3, ENABLE);

    // 等待计数完成
    while (TIM_GetFlagStatus(TIM3, TIM_FLAG_Update) == RESET);

    TIM_ClearFlag(TIM3, TIM_FLAG_Update);
    TIM_Cmd(TIM3, DISABLE);
}
```

15.3 PWM输出

PWM原理

参数

周期 $T = T_{on} + T_{off}$

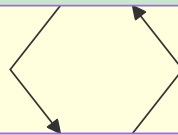
频率 $f = 1/T$

占空比 $= T_{on}/T \times 100\%$

PWM波形

高电平 (T_{on})

低电平 (T_{off})



PWM应用

占空比	应用场景
0-100%可调	LED亮度调节
固定频率可调占空比	电机速度控制
50%固定	蜂鸣器发声
可变占空比	舵机角度控制

PWM输出代码

```
#include "stm32f10x.h"

// PWM初始化 (PA6, TIM3 CH1)
void PWM_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure;
    TIM_TimeBaseInitTypeDef TIM_InitStructure;
    TIM_OCInitTypeDef TIM_OCInitStruct;

    // 开启时钟
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_TIM3_CLK_ENABLE();

    // 配置GPIO为复用推挽输出
    GPIO_InitStructure.Pin = GPIO_PIN_6;
    GPIO_InitStructure.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 定时器基础配置
    // PWM频率 = 72MHz / 72 / 1000 = 1kHz
    TIM_InitStructure.TIM_Prescaler = 72 - 1;
    TIM_InitStructure.TIM_Period = 1000 - 1;      // ARR, 决定周期
    TIM_InitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM3, &TIM_InitStructure);

    // PWM模式配置
    TIM_OCInitStruct.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStruct.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStruct.TIM_Pulse = 500;            // CCR, 决定占空比
    TIM_OCInitStruct.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OC1Init(TIM3, &TIM_OCInitStruct);

    TIM_OC1PreloadConfig(TIM3, TIM_OCPreload_Enable);
    TIM_Cmd(TIM3, ENABLE);
}

// 设置PWM占空比 (0-100%)
void PWM_SetDuty(uint8_t duty) {
```

```
uint16_t ccr = (duty * 1000) / 100; // 转换为CCR值
TIM_SetCompare1(TIM3, ccr);
}

// LED亮度调节
void LED_Brightness(void) {
    uint8_t brightness = 0;
    int8_t step = 1;

    while (1) {
        PWM_SetDuty(brightness);

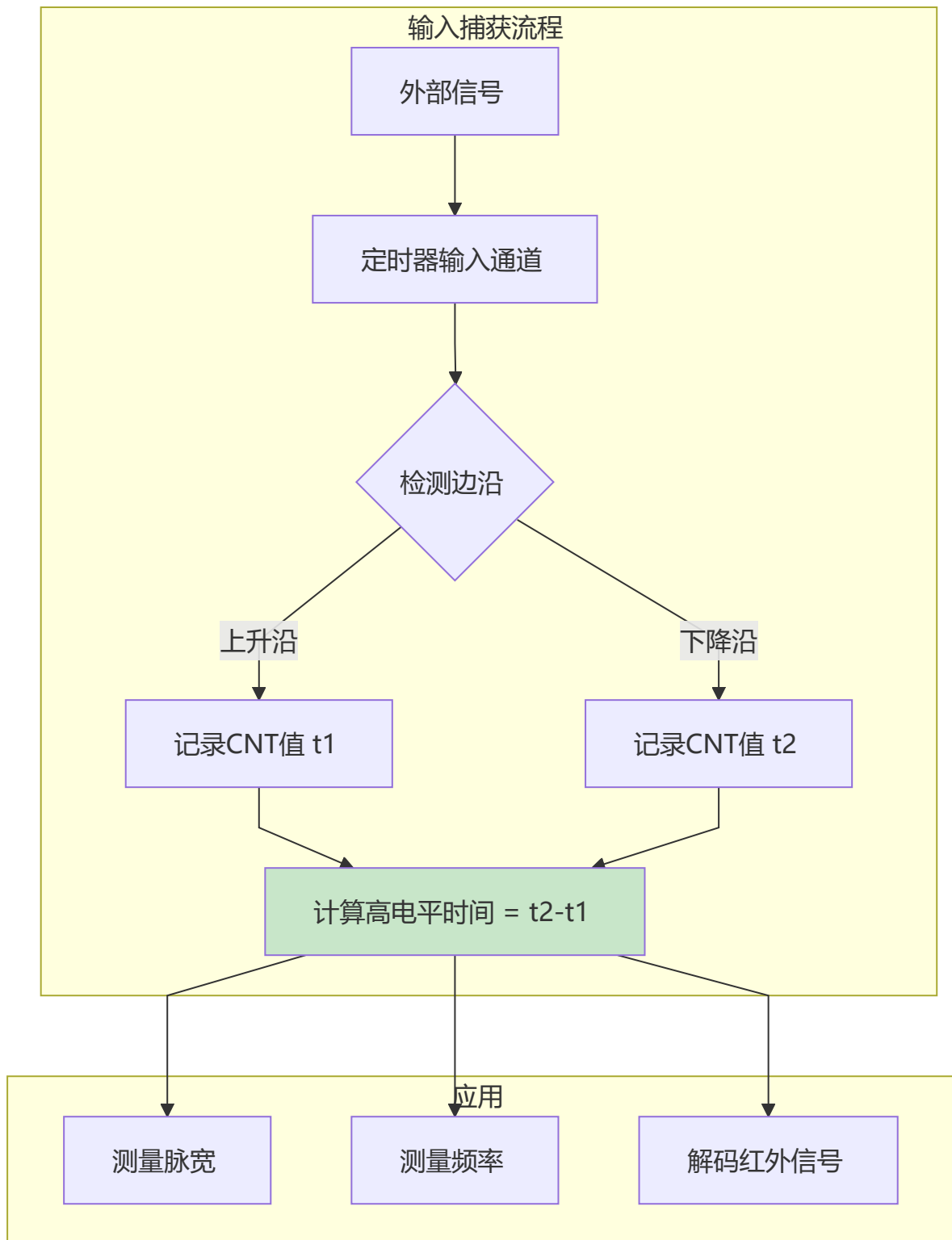
        brightness += step;
        if (brightness >= 100) step = -1;
        if (brightness <= 0) step = 1;

        HAL_Delay(10);
    }
}

// 电机速度控制
void Motor_Speed(uint8_t speed) {
    PWM_SetDuty(speed); // 0-100对应速度
}
```

15.4 输入捕获

输入捕获原理



输入捕获代码

```
#include "stm32f10x.h"

// 测量变量
volatile uint32_t captureValue1 = 0;
volatile uint32_t captureValue2 = 0;
volatile uint8_t captureState = 0;
volatile uint32_t pulsewidth = 0;
```

```

// 输入捕获初始化 (PA0, TIM2 CH1)
void InputCapture_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure;
    TIM_TimeBaseInitTypeDef TIM_InitStructure;
    TIM_ICInitTypeDef TIM_ICInitStruct;

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_TIM2_CLK_ENABLE();

    // GPIO配置为输入
    GPIO_InitStructure.Pin = GPIO_PIN_0;
    GPIO_InitStructure.Mode = GPIO_MODE_IPU;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 定时器配置
    TIM_InitStructure.TIM_Prescaler = 72 - 1; // 1MHz, 1us分辨率
    TIM_InitStructure.TIM_Period = 0xFFFF;
    TIM_InitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM2, &TIM_InitStructure);

    // 输入捕获配置
    TIM_ICInitStruct.TIM_Channel = TIM_Channel_1;
    TIM_ICInitStruct.TIM_ICPolarity = TIM_ICPolarity_Rising; // 上升沿捕获
    TIM_ICInitStruct.TIM_ICSelection = TIM_ICSelection_DirectTI;
    TIM_ICInitStruct.TIM_ICPrescaler = TIM_ICPSC_DIV1;
    TIM_ICInitStruct.TIM_ICFilter = 0;
    TIM_ICInit(TIM2, &TIM_ICInitStruct);

    TIM_ITConfig(TIM2, TIM_IT_CC1, ENABLE);
    NVIC_EnableIRQ(TIM2_IRQn);
    TIM_Cmd(TIM2, ENABLE);
}

// 输入捕获中断
void TIM2_IRQHandler(void) {
    if (TIM_GetITStatus(TIM2, TIM_IT_CC1)) {
        if (captureState == 0) {
            // 第一次捕获 (上升沿)
            captureValue1 = TIM_GetCapture1(TIM2);
            TIM_SetCompare1(TIM2, 0);

            // 切换为下降沿捕获
            TIM2->CCER |= TIM_CCER_CC1P;
            captureState = 1;
        } else {
            // 第二次捕获 (下降沿)
            captureValue2 = TIM_GetCapture1(TIM2);

            // 计算脉宽
            pulsewidth = captureValue2 - captureValue1;

            // 切换回上升沿捕获

```



```
        TIM2->CCER &= ~TIM_CCER_CC1P;
        captureState = 0;
    }

    TIM_ClearITPendingBit(TIM2, TIM_IT_CC1);
}

// 获取脉宽（微秒）
uint32_t Get_Pulsewidth(void) {
    return pulsewidth;
}
```

第十五章小结

本章要点

1. 定时器是精确计时的核心
2. PWM通过改变占空比控制功率输出
3. 输入捕获用于测量外部信号
4. ARR决定周期，CCR决定占空比
5. 预分频器决定计数频率

动手练习

练习1：使用PWM实现呼吸灯效果。

练习2：使用输入捕获测量方波频率。

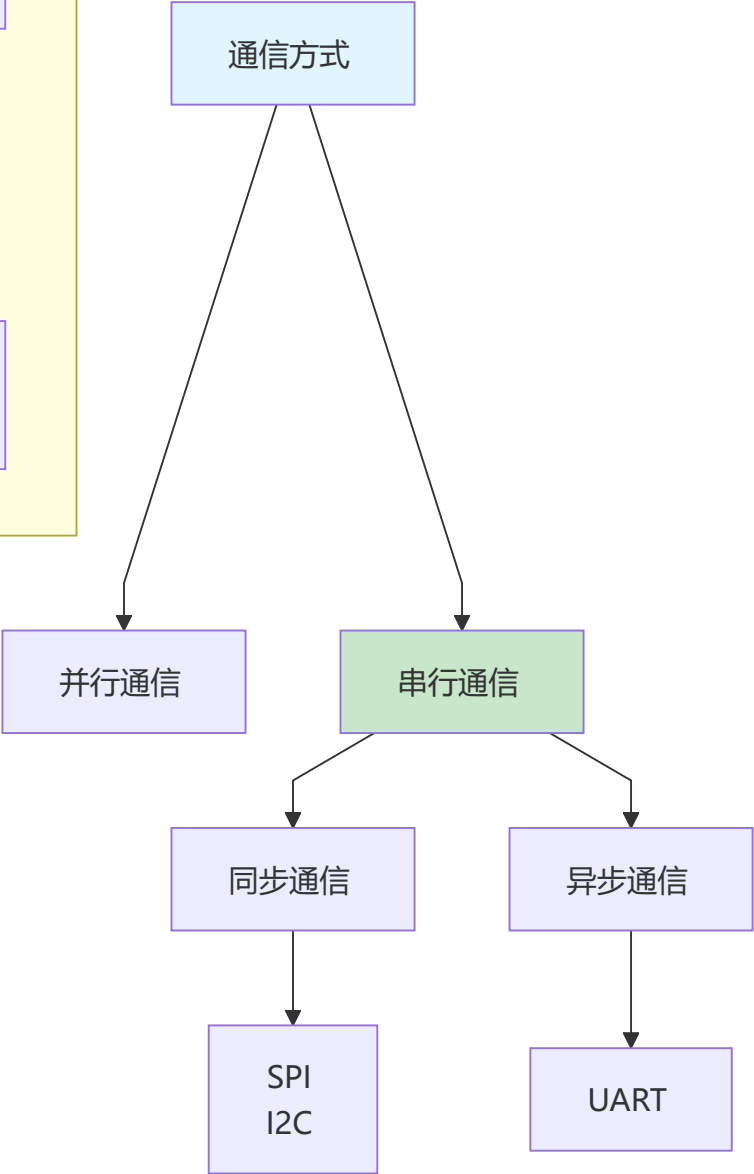
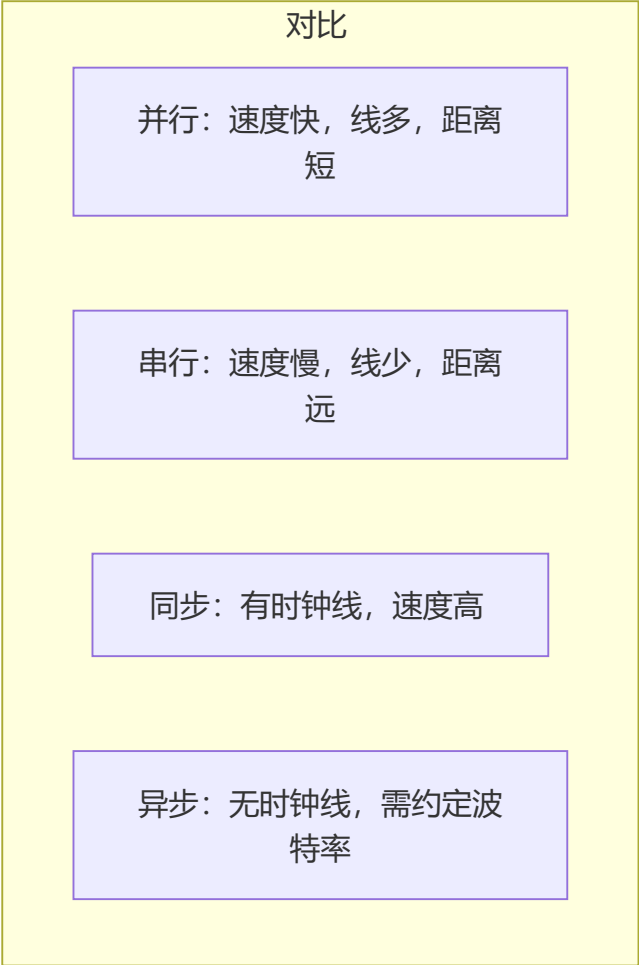
练习3：使用定时器产生多路PWM控制舵机。

第十六章：串行通信（UART/I2C/SPI）

串行通信是单片机与外部设备数据交换的桥梁

16.1 通信方式对比

通信分类



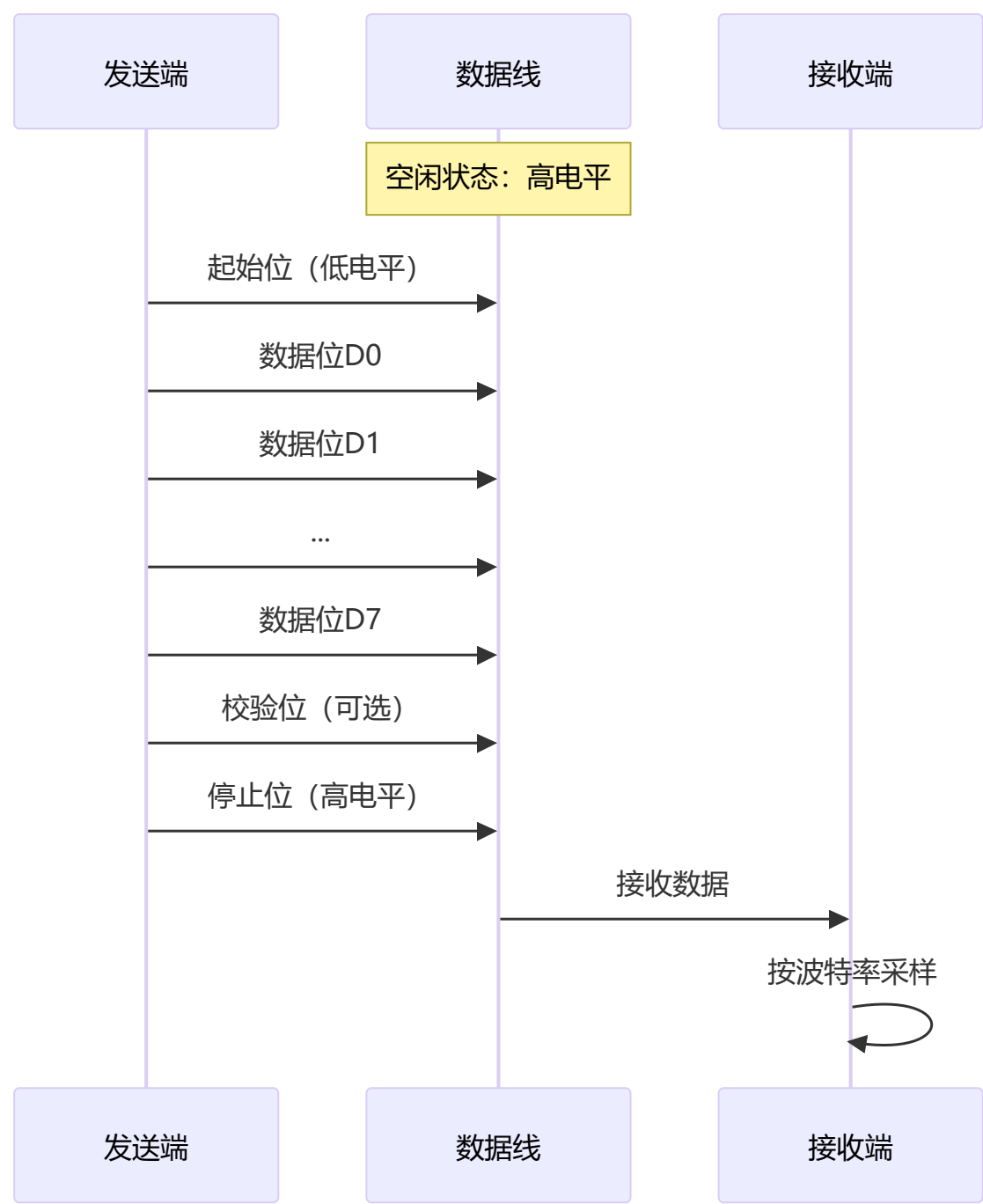
三种通信对比

特性	UART	I2C	SPI
线数	2 (TX/RX)	2 (SDA/SCL)	4 (MOSI/MISO/SCK/CS)
速度	较慢 (最高数Mbps)	中等 (最高3.4Mbps)	快 (最高数十Mbps)

特性	UART	I2C	SPI
距离	远（可达15m）	近（同一PCB）	近（同一PCB）
拓扑	点对点	多主多从	一主多从
应用	调试、蓝牙、GPS	传感器、EEPROM	SD卡、显示屏、FLASH

16.2 UART串口通信

UART原理



UART配置

```
#include "stm32f10x.h"

// UART初始化 (PA9-TX, PA10-RX)
void UART_Init(uint32_t baudrate) {
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_USART1_CLK_ENABLE();

    // TX配置为复用推挽
    GPIO_InitStructure.Pin = GPIO_PIN_9;
    GPIO_InitStructure.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    // RX配置为浮空输入
    GPIO_InitStructure.Pin = GPIO_PIN_10;
    GPIO_InitStructure.Mode = GPIO_MODE_IN_FLOATING;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    // UART配置
    USART_InitStructure.USART_BaudRate = baudrate;
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_InitStructure.USART_StopBits = USART_StopBits_1;
    USART_InitStructure.USART_Parity = USART_Parity_No;
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
    USART_Init(USART1, &USART_InitStructure);

    USART_Cmd(USART1, ENABLE);
}

// 发送单个字节
void UART_SendByte(uint8_t data) {
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);
    USART_SendData(USART1, data);
}

// 发送字符串
void UART_SendString(const char *str) {
    while (*str) {
        UART_SendByte(*str++);
    }
}

// 接收单个字节
uint8_t UART_ReceiveByte(void) {
    while (USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == RESET);
    return USART_ReceiveData(USART1);
}
```

```
}

// 重定向printf
int fputc(int ch, FILE *f) {
    UART_SendByte(ch);
    return ch;
}
```

UART中断接收

```
#include <string.h>

#define RX_BUFFER_SIZE 256
volatile uint8_t rxBuffer[RX_BUFFER_SIZE];
volatile uint16_t rxIndex = 0;
volatile uint8_t rxComplete = 0;

// 开启接收中断
void UART_EnableRxIT(void) {
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    NVIC_EnableIRQ(USART1_IRQn);
}

// 接收中断服务函数
void USART1_IRQHandler(void) {
    if (USART_GetITStatus(USART1, USART_IT_RXNE)) {
        uint8_t data = USART_ReceiveData(USART1);

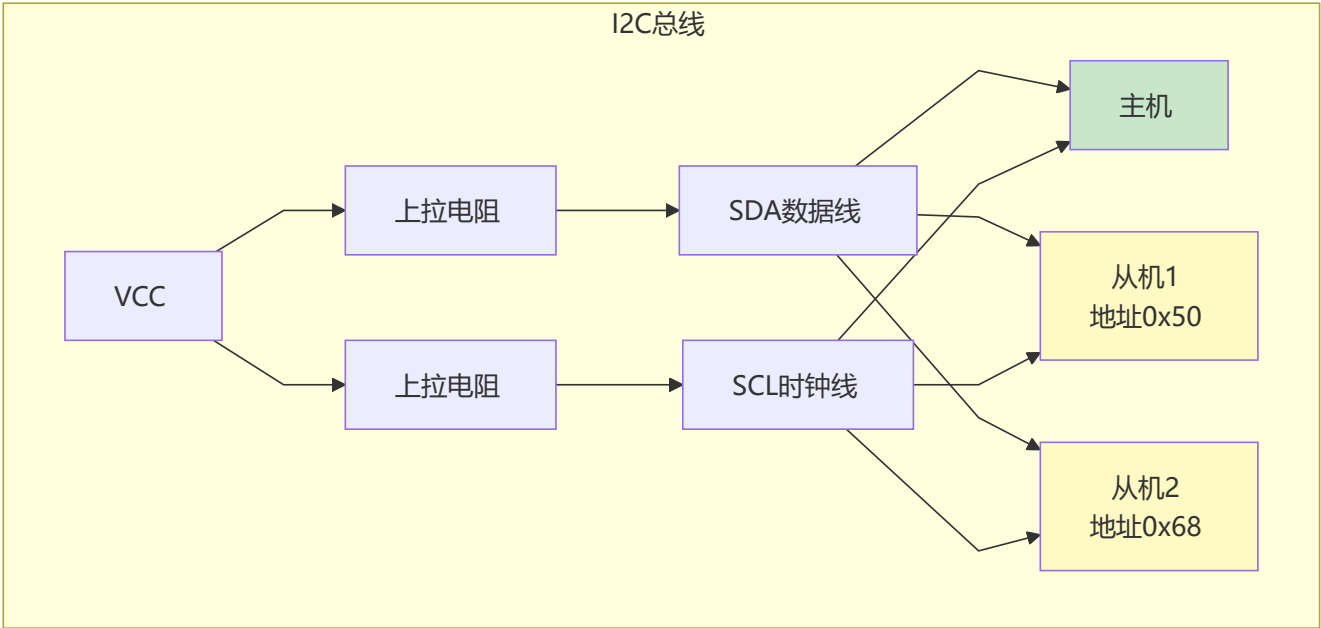
        if (data == '\n' || rxIndex >= RX_BUFFER_SIZE - 1) {
            rxBuffer[rxIndex] = '\0';
            rxComplete = 1;
            rxIndex = 0;
        } else {
            rxBuffer[rxIndex++] = data;
        }
    }
}

// 主程序处理
int main(void) {
    UART_Init(115200);
    UART_EnableRxIT();

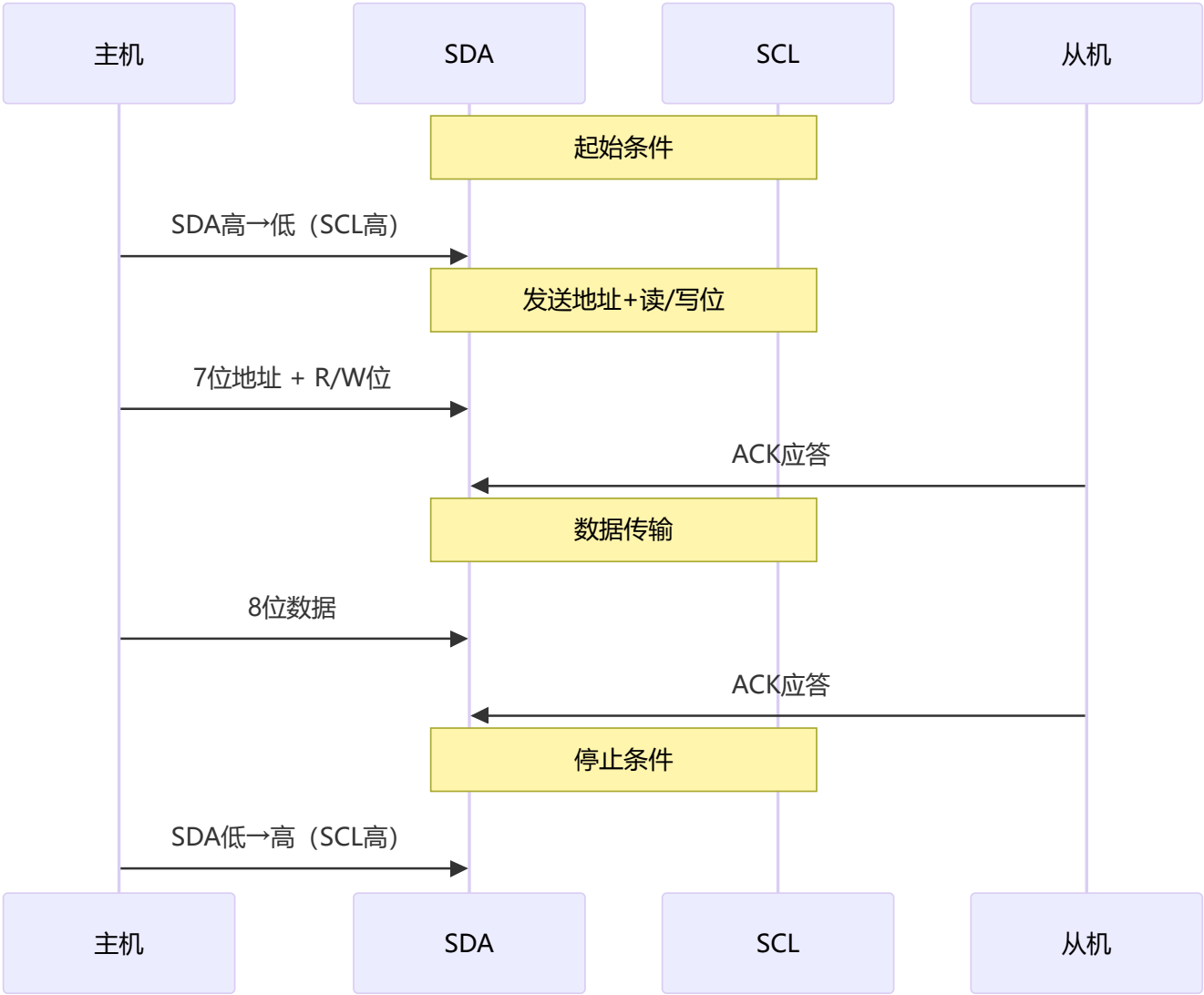
    while (1) {
        if (rxComplete) {
            rxComplete = 0;
            printf("收到: %s\n", rxBuffer);
        }
    }
}
```

16.3 I2C通信

I2C原理



I2C时序



I2C代码示例

```
#include "stm32f10x.h"

// I2C初始化
void I2C_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure;
    I2C_InitTypeDef I2C_InitStructure;

    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_I2C1_CLK_ENABLE();

    // PB6-SCL, PB7-SDA 配置为开漏输出
    GPIO_InitStructure.Pin = GPIO_PIN_6 | GPIO_PIN_7;
    GPIO_InitStructure.Mode = GPIO_MODE_AF_OD;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);

    // I2C配置
```

```

    I2C_InitStruct.I2C_Mode = I2C_Mode_I2C;
    I2C_InitStruct.I2C_DutyCycle = I2C_DutyCycle_2;
    I2C_InitStruct.I2C_OwnAddress1 = 0x00;
    I2C_InitStruct.I2C_Ack = I2C_Ack_Enable;
    I2C_InitStruct.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7bit;
    I2C_InitStruct.I2C_ClockSpeed = 100000; // 100kHz
    I2C_Init(I2C1, &I2C_InitStruct);

    I2C_Cmd(I2C1, ENABLE);
}

// I2C写入数据
void I2C_Write(uint8_t addr, uint8_t reg, uint8_t data) {
    I2C_GenerateSTART(I2C1, ENABLE);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_MODE_SELECT));

    I2C_Send7bitAddress(I2C1, addr << 1, I2C_Direction_Transmitter);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED));

    I2C_SendData(I2C1, reg);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_BYTE_TRANSMITTED));

    I2C_SendData(I2C1, data);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_BYTE_TRANSMITTED));

    I2C_GenerateSTOP(I2C1, ENABLE);
}

// I2C读取数据
uint8_t I2C_Read(uint8_t addr, uint8_t reg) {
    uint8_t data;

    I2C_GenerateSTART(I2C1, ENABLE);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_MODE_SELECT));

    I2C_Send7bitAddress(I2C1, addr << 1, I2C_Direction_Transmitter);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED));

    I2C_SendData(I2C1, reg);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_BYTE_TRANSMITTED));

    I2C_GenerateSTART(I2C1, ENABLE);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_MODE_SELECT));

    I2C_Send7bitAddress(I2C1, addr << 1, I2C_Direction_Receiver);
    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_RECEIVER_MODE_SELECTED));

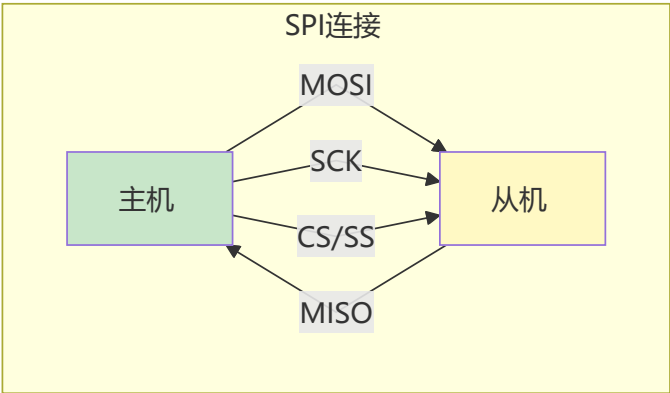
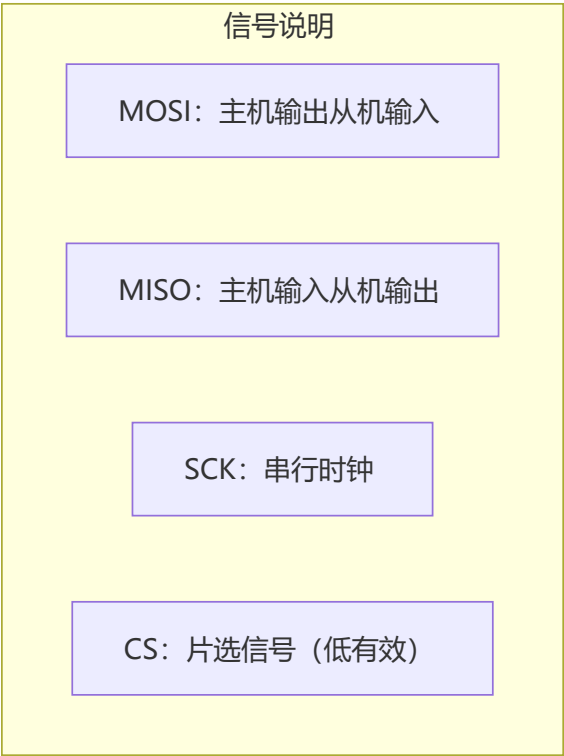
    I2C_AcknowledgeConfig(I2C1, DISABLE);
    data = I2C_ReceiveData(I2C1);
    I2C_GenerateSTOP(I2C1, ENABLE);

    return data;
}

```


16.4 SPI通信

SPI原理



SPI时序模式

模式	CPOL	CPHA	空闲电平	采样边沿
Mode 0	0	0	低	第一个边沿 (上升)
Mode 1	0	1	低	第二个边沿 (下降)
Mode 2	1	0	高	第一个边沿 (下降)
Mode 3	1	1	高	第二个边沿 (上升)

SPI代码示例

```
#include "stm32f10x.h"

// SPI初始化
void SPI_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure;
    SPI_InitTypeDef SPI_InitStructure;

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_SPI1_CLK_ENABLE();

    // PA5-SCK, PA7-MOSI 配置为复用推挽
```

```

GPIO_InitStruct.Pin = GPIO_PIN_5 | GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

// PA6-MISO 配置为浮空输入
GPIO_InitStruct.Pin = GPIO_PIN_6;
GPIO_InitStruct.Mode = GPIO_MODE_IN_FLOATING;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

// PA4-CS 配置为推挽输出
GPIO_InitStruct.Pin = GPIO_PIN_4;
GPIO_InitStruct.Mode = GPIO_MODE_OUT_PP;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET); // CS高，不选中

// SPI配置
SPI_InitStruct.SPI_Direction = SPI_Direction_2Lines_FullDuplex;
SPI_InitStruct.SPI_Mode = SPI_Mode_Master;
SPI_InitStruct.SPI_DataSize = SPI_DataSize_8b;
SPI_InitStruct.SPI_CPOL = SPI_CPOL_Low;
SPI_InitStruct.SPI_CPHA = SPI_CPHA_1Edge;
SPI_InitStruct.SPI_NSS = SPI_NSS_Soft;
SPI_InitStruct.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_8;
SPI_InitStruct.SPI_FirstBit = SPI_FirstBit_MSB;
SPI_Init(SPI1, &SPI_InitStruct);

SPI_Cmd(SPI1, ENABLE);
}

// SPI读写一个字节
uint8_t SPI_Readwrite(uint8_t data) {
    while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_TXE) == RESET);
    SPI_I2S_SendData(SPI1, data);

    while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_RXNE) == RESET);
    return SPI_I2S_ReceiveData(SPI1);
}

// 片选控制
#define CS_LOW()    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET)
#define CS_HIGH()   HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET)

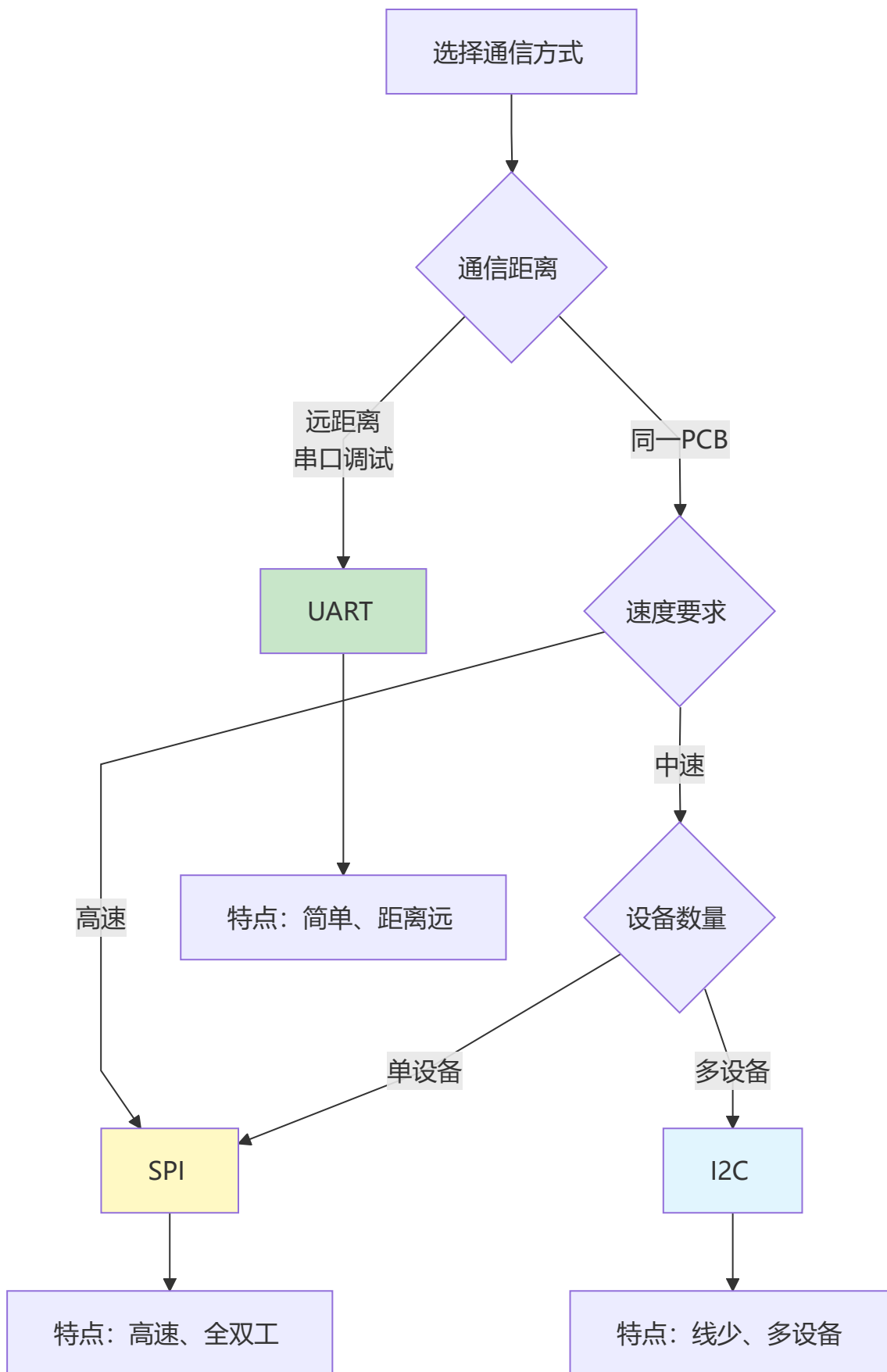
// 写入数据到设备
void SPI_WriteRegister(uint8_t reg, uint8_t value) {
    CS_LOW();
    SPI_Readwrite(reg);
    SPI_Readwrite(value);
    CS_HIGH();
}

// 从设备读取数据
uint8_t SPI_ReadRegister(uint8_t reg) {

```

```
uint8_t value;  
CS_LOW();  
SPI_ReadWrite(reg | 0x80); // 读标志  
value = SPI_ReadWrite(0xFF);  
CS_HIGH();  
return value;  
}
```

16.5 通信方式选择指南



第十六章小结

本章要点

- 1. UART是最简单的串行通信，适合调试和远距离
- 2. I2C只需要两根线，适合多设备连接
- 3. SPI速度最快，适合高速数据传输
- 4. 选择通信方式要看距离、速度、设备数量
- 5. 中断接收比轮询更高效

动手练习

- 练习1：实现UART命令解析器。
- 练习2：使用I2C读写EEPROM（AT24C02）。
- 练习3：使用SPI驱动TFT显示屏。

第十七章：ADC模数转换

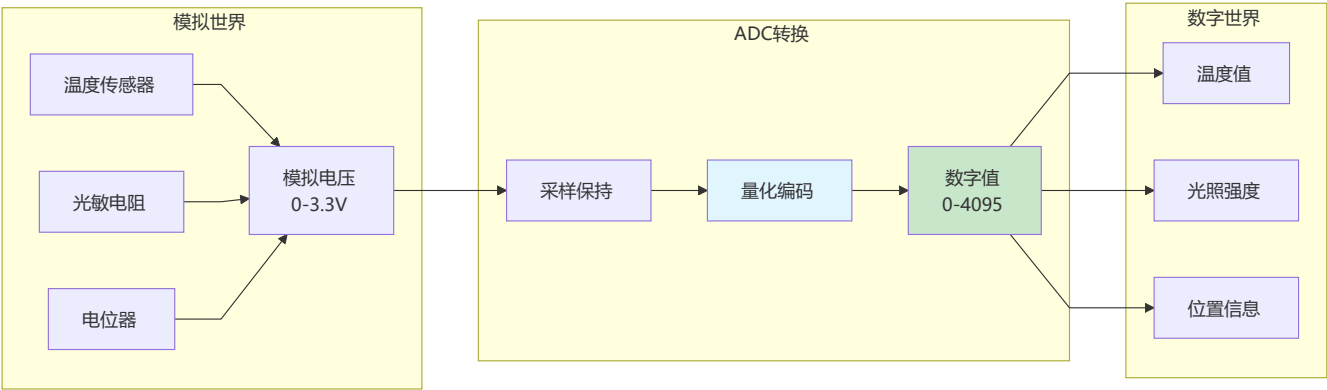
ADC让单片机能够感知模拟世界

17.1 什么是ADC？

一句话理解

ADC（Analog-to-Digital Converter）是**将模拟信号转换为数字信号的器件**，让单片机能读取温度、光照、电压等模拟量。

ADC原理



ADC关键参数

参数	说明	示例
分辨率	能区分的最小变化量	12位 = 4096级
参考电压	满量程对应的电压	3.3V
采样率	每秒采样次数	1Msps
精度	实际值与测量值差异	±1LSB

电压计算公式

数字值 = (模拟电压 / 参考电压) × (2^分辨率 - 1)

例如：12位ADC，参考电压3.3V，输入电压1.65V

数字值 = (1.65 / 3.3) × 4095 = 2048

17.2 ADC基本使用

ADC初始化

```
#include "stm32f10x.h"

// ADC初始化 (PA0, ADC1通道0)
void ADC_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure;
    ADC_InitTypeDef ADC_InitStructure;

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_ADC1_CLK_ENABLE();

    // GPIO配置为模拟输入
    GPIO_InitStructure.Pin = GPIO_PIN_0;
    GPIO_InitStructure.Mode = GPIO_MODE_AIN; // 模拟输入模式
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    // ADC配置
    ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;
    ADC_InitStructure.ADC_ScanConvMode = DISABLE;
    ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
    ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;
    ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
    ADC_InitStructure.ADC_NbrOfChannel = 1;
    ADC_Init(ADC1, &ADC_InitStructure);

    // 配置通道
    ADC_RegularChannelConfig(ADC1, ADC_Channel_0, 1, ADC_SampleTime_55Cycles5);
}
```

```

ADC_Cmd(ADC1, ENABLE);

// ADC校准
ADC_ResetCalibration(ADC1);
while (ADC_GetResetCalibrationStatus(ADC1));
ADC_StartCalibration(ADC1);
while (ADC_GetCalibrationStatus(ADC1));
}

// 读取ADC值
uint16_t ADC_Read(void) {
    ADC_SoftwareStartConvCmd(ADC1, ENABLE);
    while (ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == RESET);
    return ADC_GetConversionValue(ADC1);
}

// 读取电压值
float ADC_ReadVoltage(void) {
    uint16_t adcValue = ADC_Read();
    return (float)adcValue * 3.3f / 4095.0f;
}

```

多通道采集

```

#define ADC_CHANNELS 3
uint16_t adcValues[ADC_CHANNELS];

// 多通道ADC初始化
void ADC_MultiChannel_Init(void) {
    GPIO_InitTypeDef GPIO_InitStruct;
    ADC_InitTypeDef ADC_InitStruct;

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_ADC1_CLK_ENABLE();

    // 配置PA0, PA1, PA2为模拟输入
    GPIO_InitStruct.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2;
    GPIO_InitStruct.Mode = GPIO_MODE_AIN;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    // ADC配置（扫描模式）
    ADC_InitStruct.ADC_Mode = ADC_Mode_Independent;
    ADC_InitStruct.ADC_ScanConvMode = ENABLE;           // 扫描模式
    ADC_InitStruct.ADC_ContinuousConvMode = ENABLE;     // 连续转换
    ADC_InitStruct.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;
    ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right;
    ADC_InitStruct.ADC_NbrOfChannel = ADC_CHANNELS;
    ADC_Init(ADC1, &ADC_InitStruct);

    // 配置DMA
    // ... DMA配置代码

```

```
ADC_Cmd(ADC1, ENABLE);
ADC_SoftwareStartConvCmd(ADC1, ENABLE);
}

// 使用DMA读取多通道
void ADC_ReadAll(void) {
    for (int i = 0; i < ADC_CHANNELS; i++) {
        printf("通道%d: %.2fv\n", i, (float)adcValues[i] * 3.3f / 4095.0f);
    }
}
```

17.3 ADC应用实例

读取电位器

```
// 电位器连接PA0
void Potentiometer_Read(void) {
    float voltage = ADC_ReadVoltage();
    uint8_t percentage = (uint8_t)(voltage / 3.3f * 100);

    printf("电位器位置: %d%%\n", percentage);
}
```

读取温度传感器

```
// NTC热敏电阻温度计算
float ReadTemperature(void) {
    float voltage = ADC_ReadVoltage();

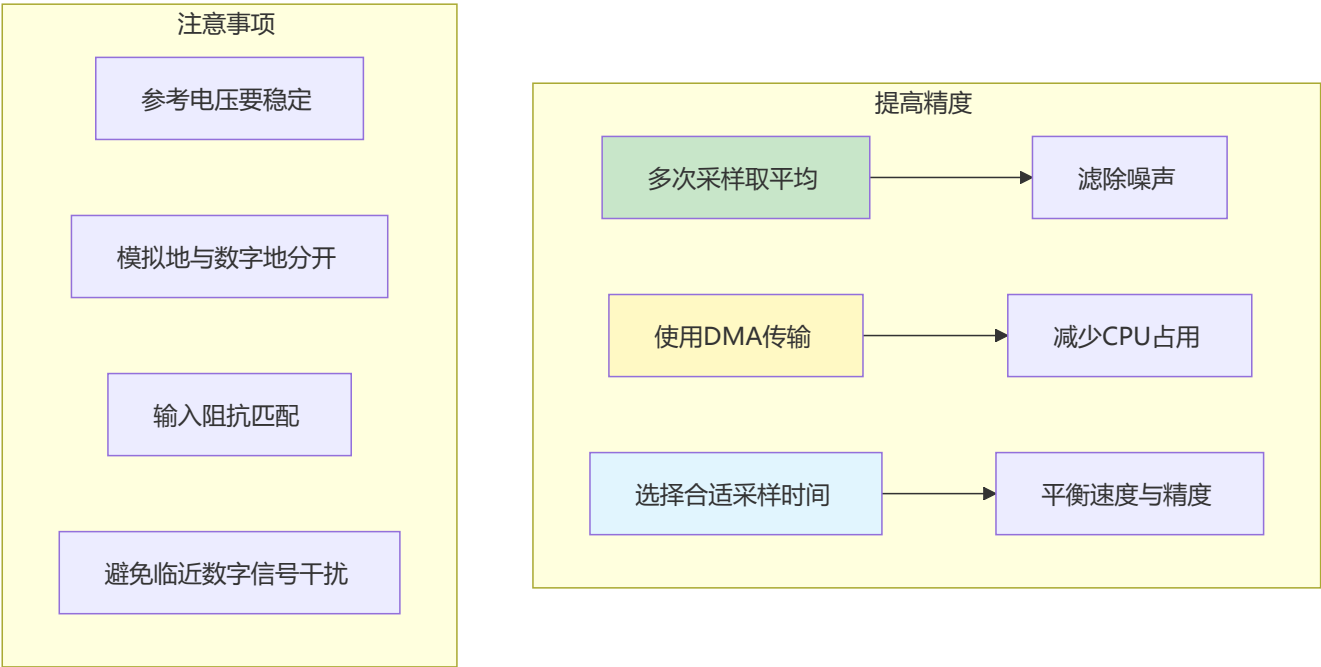
    // NTC热敏电阻计算（简化版）
    // 假设使用10K NTC, B值3950
    float R = 10000.0f * (3.3f - voltage) / voltage;
    float tempK = 1.0f / (1.0f / 298.15f + 1.0f / 3950.0f * log(R / 10000.0f));
    float tempC = tempK - 273.15f;

    return tempC;
}
```

光照检测

```
// 光敏电阻检测光照强度
uint8_t ReadLightIntensity(void) {
    float voltage = ADC_ReadVoltage();
    // 转换为0-100%光照强度
    return (uint8_t)((3.3f - voltage) / 3.3f * 100);
}
```

17.4 ADC优化技巧



滤波算法

```
// 均值滤波
uint16_t ADC_ReadAverage(uint8_t samples) {
    uint32_t sum = 0;
    for (uint8_t i = 0; i < samples; i++) {
        sum += ADC_Read();
        HAL_Delay(1);
    }
    return sum / samples;
}

// 滑动平均滤波
#define FILTER_SIZE 10
uint16_t filterBuffer[FILTER_SIZE];
uint8_t filterIndex = 0;

uint16_t ADC_ReadFiltered(uint16_t newValue) {
    filterBuffer[filterIndex] = newValue;
    filterIndex = (filterIndex + 1) % FILTER_SIZE;

    uint32_t sum = 0;
    for (uint8_t i = 0; i < FILTER_SIZE; i++) {
        sum += filterBuffer[i];
    }
    return sum / FILTER_SIZE;
}
```

第十七章小结

本章要点

- 1. ADC将模拟信号转换为数字信号
- 2. 分辨率决定测量精度
- 3. 参考电压决定测量范围
- 4. 多次采样取平均可提高精度
- 5. DMA可减少CPU占用

动手练习

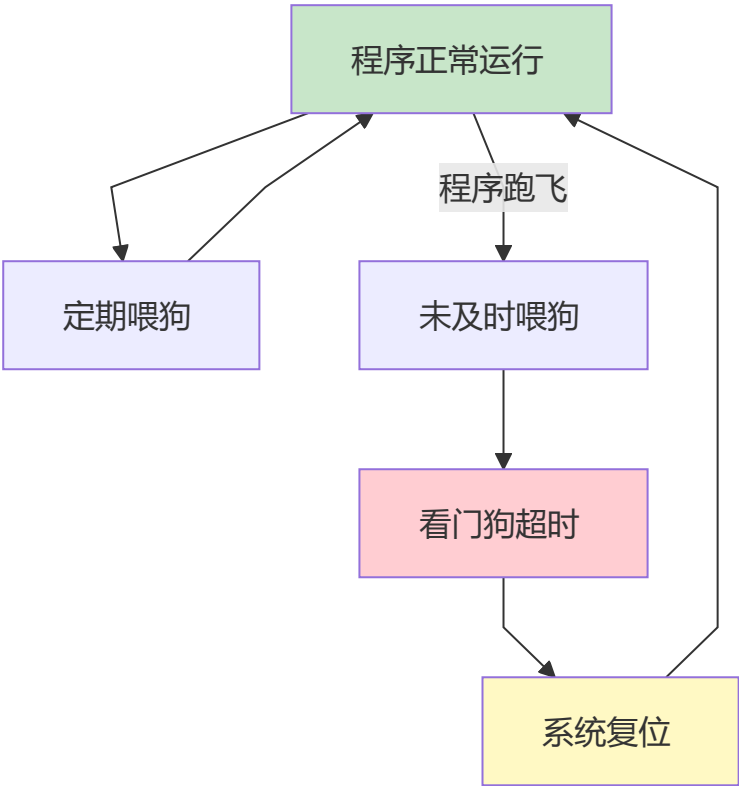
- 练习1：实现电压表功能（0-3.3V）。
- 练习2：使用ADC实现简易温度计。
- 练习3：实现光照强度自动调光LED。

第十八章：其他常用外设

这些外设让单片机系统更稳定、更高效

18.1 看门狗定时器

什么是看门狗？



独立看门狗 (IWDG)

```
#include "stm32f10x.h"

// 独立看门狗初始化
void IWDG_Init(uint32_t timeout_ms) {
    // 使能写访问
    IWDG_WriteAccessCmd(IWDG_WriteAccess_Enable);

    // 设置预分频 (40kHz LSI时钟)
    IWDG_SetPrescaler(IWDG_Prescaler_64); // 分频后约625Hz

    // 设置重载值
    uint32_t reload = timeout_ms * 625 / 1000;
    IWDG_SetReload(reload & 0xFFFF);

    // 重载计数器
    IWDG_ReloadCounter();

    // 使能看门狗
    IWDG_Enable();
}

// 喂狗
void IWDG_Feed(void) {
    IWDG_ReloadCounter();
}

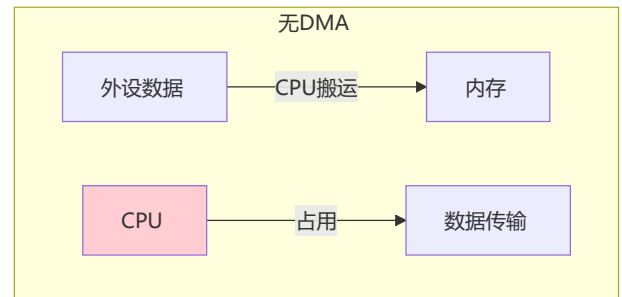
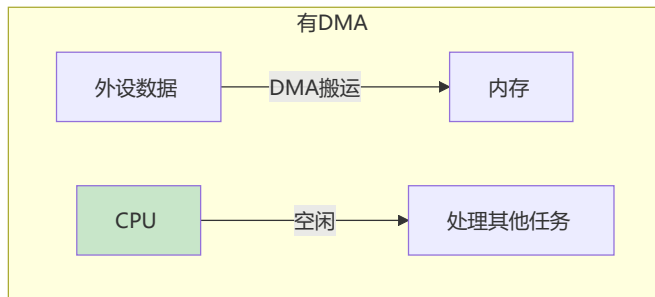
// 主程序
int main(void) {
    IWDG_Init(1000); // 1秒超时

    while (1) {
        // 主循环任务
        DoTask();

        // 定期喂狗
        IWDG_Feed();
    }
}
```

18.2 DMA直接内存访问

DMA原理



DMA配置

```
#include "stm32f10x.h"

// DMA配置（用于ADC）
void DMA_Config(void) {
    DMA_InitTypeDef DMA_InitStruct;

    __HAL_RCC_DMA1_CLK_ENABLE();

    DMA_DeInit(DMA1_Channel1);

    DMA_InitStruct.DMA_PeripheralBaseAddr = (uint32_t)&ADC1->DR;
    DMA_InitStruct.DMA_MemoryBaseAddr = (uint32_t)adcValues;
    DMA_InitStruct.DMA_DIR = DMA_DIR_PeripheralSRC;
    DMA_InitStruct.DMA_BufferSize = ADC_CHANNELS;
    DMA_InitStruct.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    DMA_InitStruct.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStruct.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Halfword;
    DMA_InitStruct.DMA_MemoryDataSize = DMA_MemoryDataSize_Halfword;
    DMA_InitStruct.DMA_Mode = DMA_Mode_Circular;
    DMA_InitStruct.DMA_Priority = DMA_Priority_High;
    DMA_InitStruct.DMA_M2M = DMA_M2M_Disable;
    DMA_Init(DMA1_Channel1, &DMA_InitStruct);

    DMA_Cmd(DMA1_Channel1, ENABLE);
    ADC_DMACmd(ADC1, ENABLE);
}
```

18.3 RTC实时时钟

RTC配置

```
#include "stm32f10x.h"

typedef struct {
    uint8_t year;
    uint8_t month;
    uint8_t day;
    uint8_t hour;
}
```

```

    uint8_t minute;
    uint8_t second;
} RTC_Time;

// RTC初始化
void RTC_Init(void) {
    // 开启电源时钟和备份域时钟
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_RCC_BKP_CLK_ENABLE();

    // 允许访问备份域
    HAL_PWR_EnableBkUpAccess();

    // 开启LSE外部低速晶振
    __HAL_RCC_LSE_CONFIG(RCC_LSE_ON);
    while (__HAL_RCC_GET_FLAG(RCC_FLAG_LSERDY) == RESET);

    // 选择RTC时钟源
    __HAL_RCC_RTC_CONFIG(RCC_RTCCLKSOURCE_LSE);

    // 开启RTC时钟
    __HAL_RCC_RTC_ENABLE();

    // 等待RTC同步
    RTC_WaitForSynchro();
    RTC_WaitForLastTask();

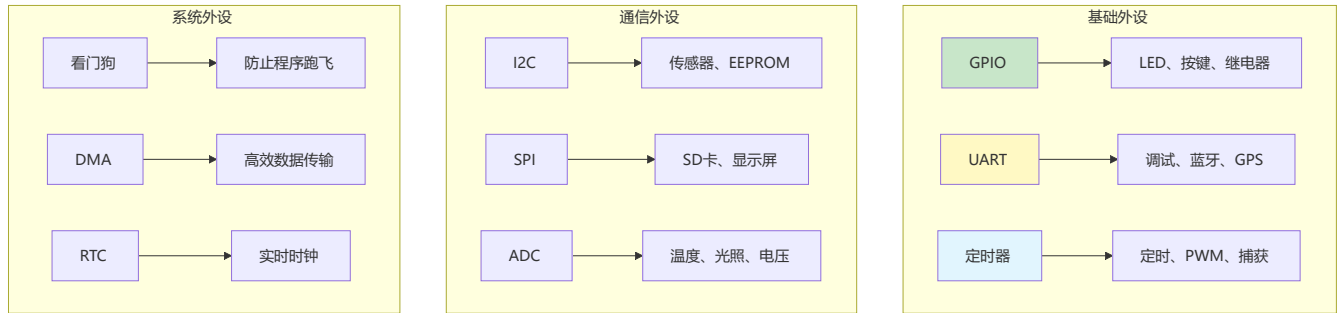
    // 设置RTC预分频 (LSE=32768Hz)
    RTC_SetPrescaler(32767);
    RTC_WaitForLastTask();
}

// 设置时间
void RTC_SetTime(RTC_Time *time) {
    uint32_t totalSeconds = 0;
    // 计算总秒数...
    RTC_SetCounter(totalSeconds);
    RTC_WaitForLastTask();
}

// 读取时间
void RTC_GetTime(RTC_Time *time) {
    uint32_t counter = RTC_GetCounter();
    // 转换为年月日时分秒...
}

```

18.4 外设使用速查表



外设	主要功能	典型应用
GPIO	数字输入输出	LED控制、按键检测
UART	异步串口通信	调试输出、蓝牙模块
I2C	两线制同步通信	温湿度传感器、EEPROM
SPI	四线制高速通信	SD卡、TFT屏幕
ADC	模数转换	温度采集、电压测量
定时器	计时/PWM/捕获	电机控制、脉宽测量
看门狗	系统保护	防止程序死机
DMA	直接内存访问	高效数据传输
RTC	实时时钟	时间记录、闹钟

第十八章小结

本章要点

- 看门狗防止程序跑飞，提高系统可靠性
- DMA减少CPU占用，提高数据传输效率
- RTC提供实时时钟功能，低功耗运行
- 合理使用外设组合，发挥最大效能

动手练习

- 练习1:** 实现看门狗保护的LED闪烁程序。
- 练习2:** 使用DMA实现高效的多通道ADC采集。
- 练习3:** 实现带RTC的电子时钟。

📌 单片机外设速查总表

章节	外设	核心功能	关键配置
第十三章	GPIO	数字输入输出	模式、上下拉、速度
第十四章	中断	事件响应	优先级、触发方式
第十五章	定时器	定时/PWM/捕获	预分频、ARR、CCR
第十六章	UART	异步通信	波特率、数据位、校验
第十六章	I2C	两线制通信	地址、速度、应答
第十六章	SPI	四线制通信	模式、时钟极性
第十七章	ADC	模数转换	分辨率、采样时间
第十八章	看门狗	系统保护	超时时间、喂狗
第十八章	DMA	高效传输	源地址、目标地址
第十八章	RTC	实时时钟	时钟源、预分频

学习建议：单片机外设的学习需要理论结合实践，建议准备一块开发板（如STM32），边学边做，通过实际项目加深理解。

进阶方向：RTOS实时操作系统、USB协议、以太网、无线通信（WiFi/蓝牙/LoRa）

第十九章：FreeRTOS简介与核心概念

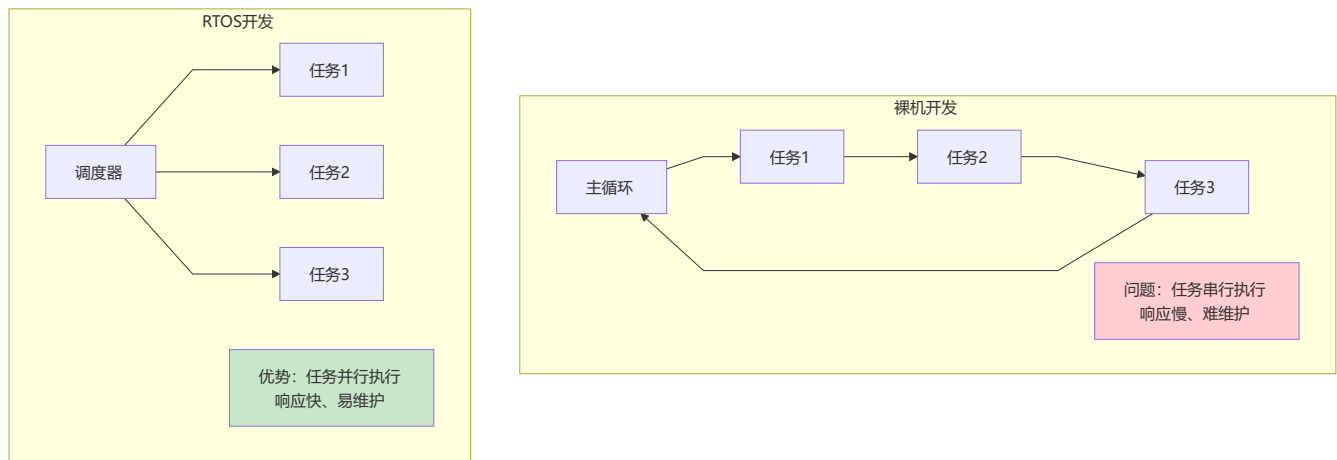
FreeRTOS是嵌入式领域最流行的实时操作系统，让多任务开发变得简单

19.1 什么是FreeRTOS?

一句话理解

FreeRTOS是一个**开源的实时操作系统（RTOS）**，提供多任务管理、任务调度、同步通信等功能，让嵌入式开发更高效。

为什么需要RTOS?



裸机 vs RTOS对比

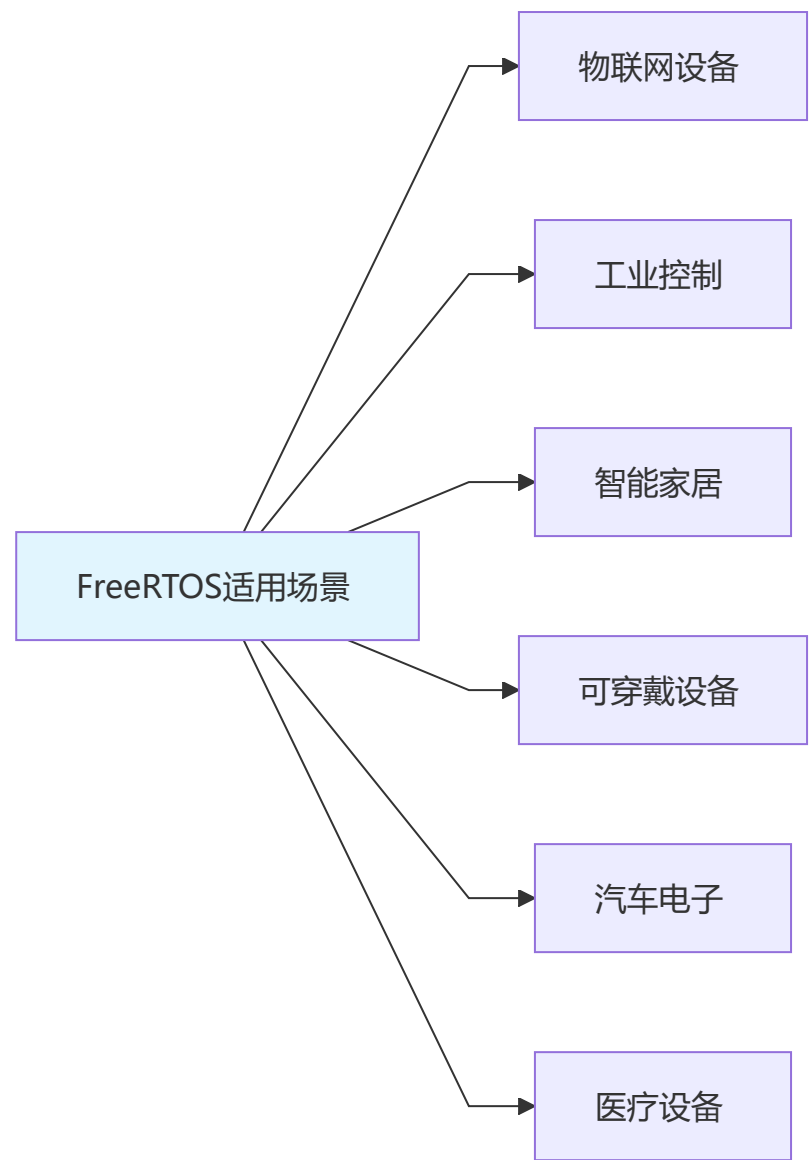
特性	裸机开发	RTOS开发
任务执行	串行（轮询）	并行（调度）
响应时间	不确定	可预测
代码结构	超级循环	模块化任务
实时性	差	好
开发难度	简单项目容易	需要学习概念
维护性	复杂后难维护	模块化易维护

19.2 FreeRTOS特点

核心特点

特点	说明
开源免费	MIT许可证，商业免费使用
内核小	核心9KB左右
可裁剪	按需配置功能
支持广泛	30+架构（ARM、RISC-V等）
社区活跃	文档丰富，生态完善

适用场景

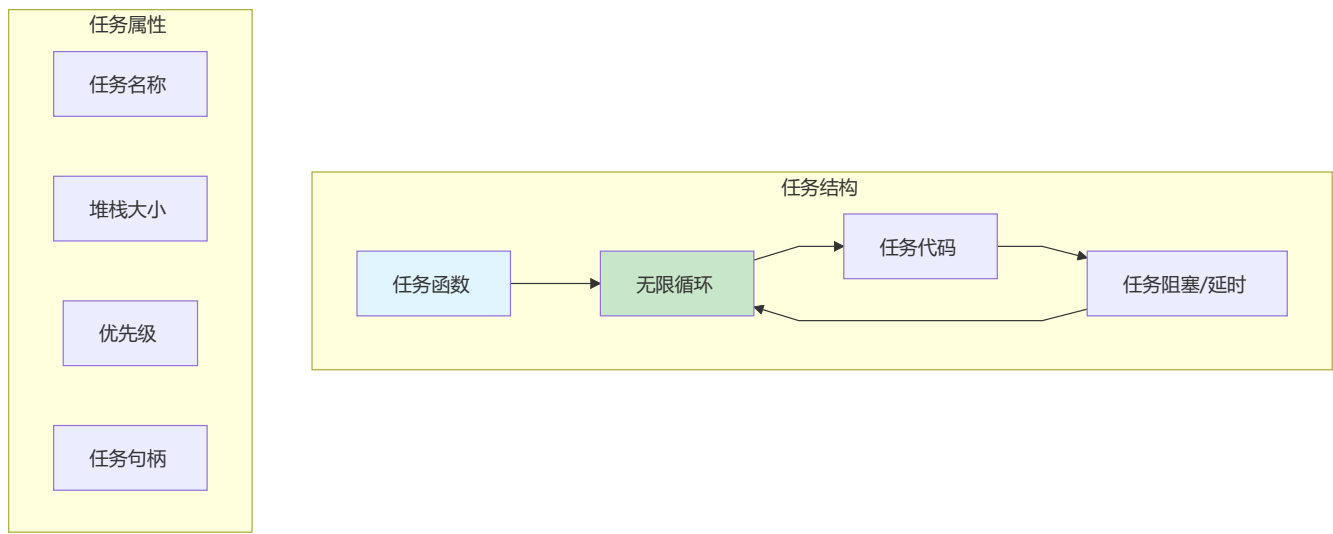


场景	说明
物联网设备	需要同时处理通信、传感器、控制
工业控制	实时响应要求高
智能家居	多设备协同工作
可穿戴设备	低功耗、多功能
汽车电子	安全性要求高

19.3 核心概念

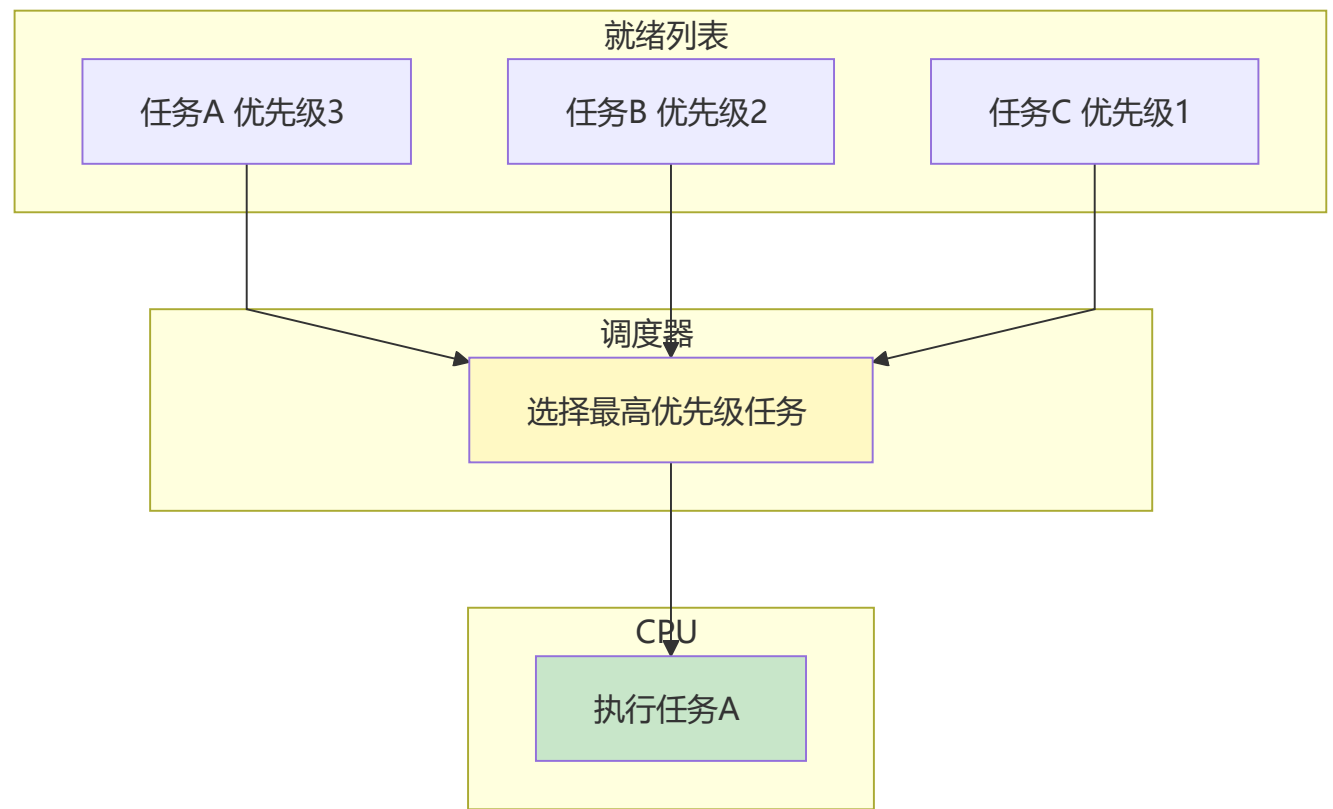
任务 (Task)

任务是FreeRTOS中最基本的执行单元，每个任务都是一个独立的函数。

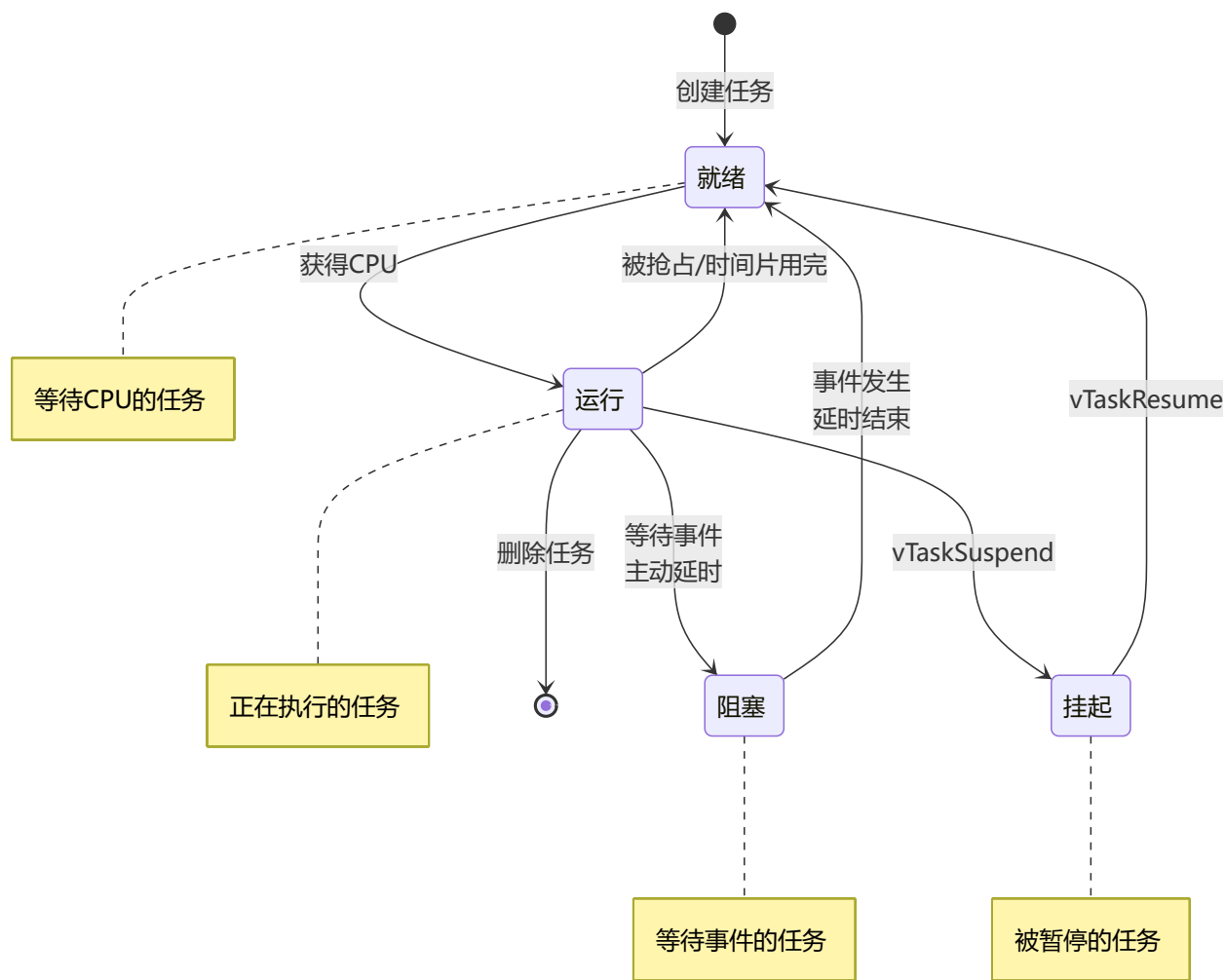


调度器 (Scheduler)

调度器决定哪个任务获得CPU使用权。



任务状态



状态转换详解

状态	说明	进入条件	退出条件
运行	正在执行	调度器选中	抢占/阻塞/挂起
就绪	等待CPU	创建/阻塞结束/恢复	被调度器选中
阻塞	等待事件	延时/等待信号量	超时/事件发生
挂起	暂停执行	vTaskSuspend()	vTaskResume()

19.4 FreeRTOS文件结构

```
FreeRTOS/
├── source/           # 源码目录
│   ├── tasks.c      # 任务管理
│   ├── queue.c      # 队列和信号量
│   └── timers.c     # 软件定时器
```

```

|   |─ list.c           # 链表操作
|   |─ croutine.c       # 协程（旧版）
|   |─ event_groups.c   # 事件组
|   |─ stream_buffer.c  # 流缓冲区
|   |─ include/         # 头文件
|   |   └─ portable/    # 移植层
|   |       └─ MemMang/  # 内存管理
|   |           └─ heap_1.c
|   |           └─ heap_2.c
|   |           └─ heap_3.c
|   |           └─ heap_4.c
|   |           └─ heap_5.c
|   └─ [架构名]/        # 架构相关代码
|       └─ port.c
└─ Demo/                # 示例工程
└─ License/             # 许可证

```

19.5 配置文件FreeRTOSConfig.h

// FreeRTOSConfig.h - 核心配置文件

```

#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

```

// ===== 基础配置 =====

```

#define configUSE_PREEMPTION    1    // 抢占式调度
#define configUSE_IDLE_HOOK     0    // 空闲任务钩子
#define configUSE_TICK_HOOK     0    // Tick钩子
#define configCPU_CLOCK_HZ      (72000000) // CPU频率
#define configTICK_RATE_HZ      ((TickType_t)1000) // Tick频率

```

// ===== 内存配置 =====

```

#define configTOTAL_HEAP_SIZE   ((size_t)(10 * 1024)) // 堆大小
#define configAPPLICATION_ALLOCATED_HEAP 0 // 应用分配堆

```

// ===== 任务配置 =====

```

#define configMAX_PRIORITIES    (5) // 最大优先级数
#define configMINIMAL_STACK_SIZE ((uint16_t)128) // 空闲任务堆栈
#define configMAX_TASK_NAME_LEN (16) // 任务名最大长度
#define configUSE_16_BIT_TICKS 0 // 32位Tick

```

// ===== 同步机制配置 =====

```

#define configUSE_MUTEXES        1 // 使能互斥量
#define configUSE_COUNTING_SEMAPHORES 1 // 使能计数信号量
#define configUSE_RECURSIVE_MUTEXES 1 // 使能递归互斥量
#define configUSE_QUEUE_SETS    1 // 使能队列集

```

// ===== 定时器配置 =====

```

#define configUSE_TIMERS        1 // 使能软件定时器
#define configTIMER_TASK_PRIORITY (2) // 定时器任务优先级
#define configTIMER_QUEUE_LENGTH (10) // 定时器队列长度

```

```

#define configTIMER_TASK_STACK_DEPTH          (128)    // 定时器任务堆栈

// ===== API函数包含配置 =====
#define INCLUDE_vTaskPrioritySet                1
#define INCLUDE_uxTaskPriorityGet              1
#define INCLUDE_vTaskDelete                   1
#define INCLUDE_vTaskSuspend                   1
#define INCLUDE_vTaskDelayUntil                1
#define INCLUDE_vTaskDelay                     1

// ===== 中断配置 =====
#ifdef __NVIC_PRIO_BITS
    #define configPRIO_BITS                    __NVIC_PRIO_BITS
#else
    #define configPRIO_BITS                    4
#endif

#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY    15
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY    5
#define configKERNEL_INTERRUPT_PRIORITY
(configLIBRARY_LOWEST_INTERRUPT_PRIORITY << (8 - configPRIO_BITS))
#define configMAX_SYSCALL_INTERRUPT_PRIORITY
(configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 - configPRIO_BITS))

#endif /* FREERTOS_CONFIG_H */

```

第十九章小结

本章要点

1. FreeRTOS是开源免费的实时操作系统
2. 解决裸机开发任务串行、响应慢的问题
3. 核心概念：任务、调度器、状态机
4. 任务有四种状态：运行、就绪、阻塞、挂起
5. 通过FreeRTOSConfig.h配置系统参数

动手练习

练习1：下载FreeRTOS源码，了解目录结构。

练习2：阅读FreeRTOSConfig.h配置文件，理解各项含义。

练习3：在开发板上移植FreeRTOS。

第二十章：任务管理

任务是FreeRTOS的核心，学会任务管理就掌握了RTOS的一半

20.1 任务结构

任务函数模板

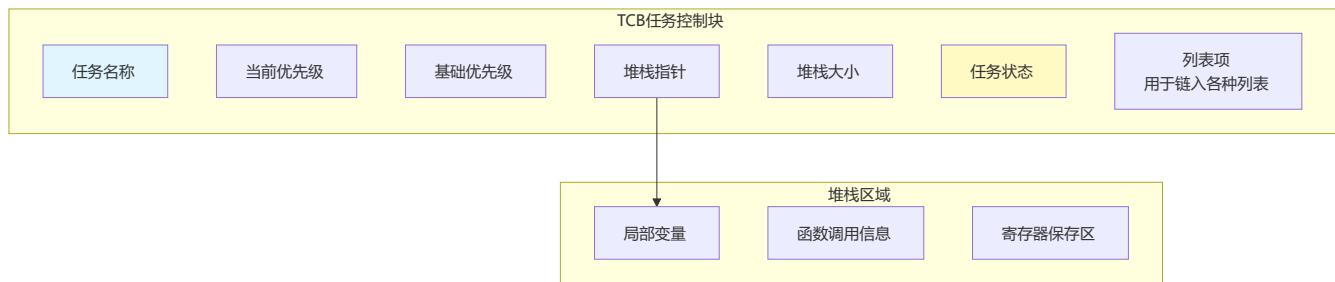
```
// 任务函数必须遵循这个格式
void TaskFunction(void *pvParameters) {
    // 可选：参数处理
    int param = *(int *)pvParameters;

    // 任务主体：无限循环
    while (1) {
        // 任务代码

        // 阻塞/延时（重要！让出CPU）
        vTaskDelay(pdMS_TO_TICKS(100));
    }

    // 可选：任务删除（通常不会执行到这里）
    vTaskDelete(NULL);
}
```

任务控制块（TCB）



20.2 任务创建与删除

任务创建

```
#include "FreeRTOS.h"
#include "task.h"

// 任务句柄（用于后续操作）
TaskHandle_t ledTaskHandle;
TaskHandle_t sensorTaskHandle;

// LED任务
void LED_Task(void *pvParameters) {
    while (1) {
        LED_Toggle();
        vTaskDelay(pdMS_TO_TICKS(500)); // 500ms延时
    }
}
```

```

}

// 传感器任务
void Sensor_Task(void *pvParameters) {
    while (1) {
        float temp = ReadTemperature();
        printf("温度: %.2f\n", temp);
        vTaskDelay(pdMS_TO_TICKS(1000)); // 1秒延时
    }
}

// 创建任务
void Create_Tasks(void) {
    BaseType_t result;

    // 创建LED任务
    result = xTaskCreate(
        LED_Task,           // 任务函数
        "LED Task",         // 任务名称
        128,                // 堆栈大小（字）
        NULL,               // 参数
        1,                  // 优先级
        &ledTaskHandle      // 任务句柄
    );

    if (result != pdPASS) {
        printf("LED任务创建失败\n");
    }

    // 创建传感器任务
    result = xTaskCreate(
        Sensor_Task,
        "Sensor Task",
        256,                // 需要更大堆栈（使用printf）
        NULL,
        2,                  // 更高优先级
        &sensorTaskHandle
    );

    if (result != pdPASS) {
        printf("传感器任务创建失败\n");
    }
}

// 主函数
int main(void) {
    // 硬件初始化
    HAL_Init();
    LED_Init();
    UART_Init();

    // 创建任务
    Create_Tasks();
}

```

```
// 启动调度器
vTaskStartScheduler();

// 正常情况下不会执行到这里
while (1);
}
```

任务删除

```
// 删除任务
void Delete_Task_Example(void) {
    // 删除自己
    vTaskDelete(NULL);

    // 删除其他任务
    vTaskDelete(lcdTaskHandle);
}

// 动态创建和删除任务
void Task_Manager(void *pvParameters) {
    TaskHandle_t tempHandle;

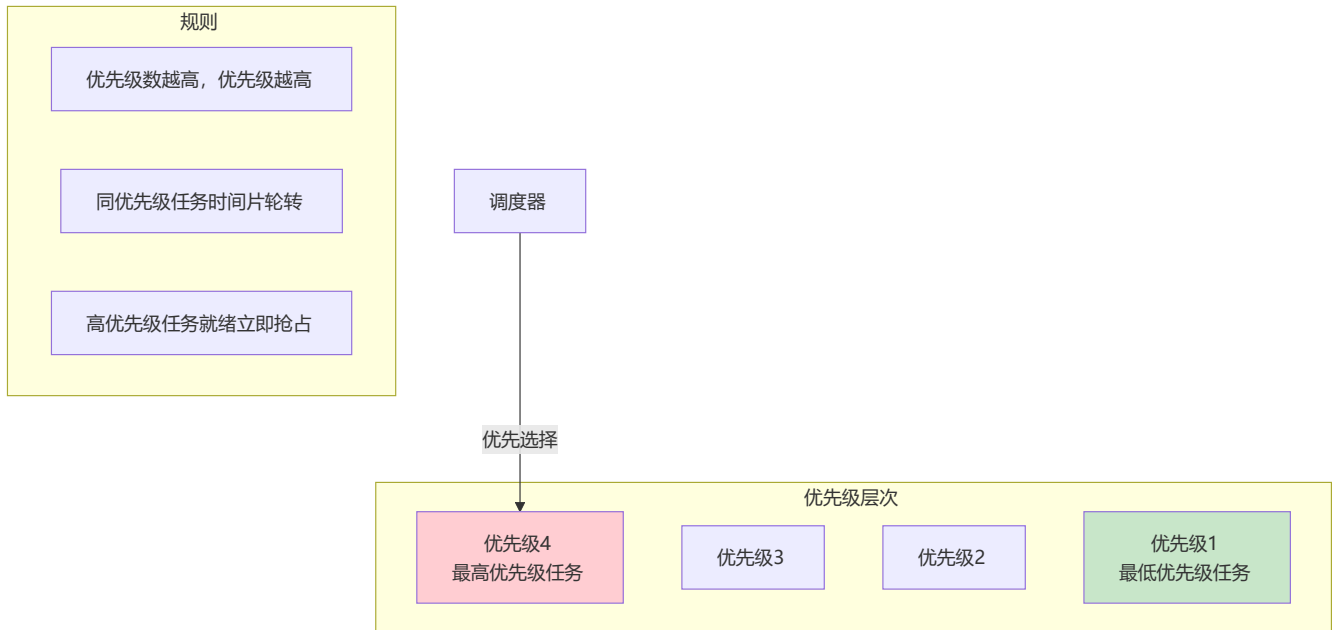
    while (1) {
        // 等待事件
        if (NeedCreateTask()) {
            xTaskCreate(Temp_Task, "Temp", 128, NULL, 1, &tempHandle);
        }

        if (NeedDeleteTask() && tempHandle != NULL) {
            vTaskDelete(tempHandle);
            tempHandle = NULL;
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

20.3 任务优先级

优先级规则



优先级操作

```
#include "FreeRTOS.h"
#include "task.h"

// 获取当前任务优先级
UBaseType_t GetCurrentPriority(void) {
    return uxTaskPriorityGet(NULL); // NULL表示当前任务
}

// 设置任务优先级
void ChangePriority(TaskHandle_t task, UBaseType_t newPriority) {
    vTaskPrioritySet(task, newPriority);
}

// 动态调整优先级示例
void Priority_Task(void *pvParameters) {
    UBaseType_t currentPriority;

    while (1) {
        currentPriority = uxTaskPriorityGet(NULL);
        printf("当前优先级: %d\n", currentPriority);

        // 某些条件下提升优先级
        if (UrgentCondition()) {
            vTaskPrioritySet(NULL, currentPriority + 1);
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

20.4 任务延时

相对延时 vs 绝对延时



延时函数

```
#include "FreeRTOS.h"
#include "task.h"

// 相对延时（常用）
void Relative_Delay_Example(void) {
    while (1) {
        DoTask(); // 执行任务（假设耗时20ms）
        vTaskDelay(pdMS_TO_TICKS(100)); // 延时100ms
        // 实际周期 = 20ms + 100ms = 120ms
    }
}

// 绝对延时（精确周期）
void Absolute_Delay_Example(void) {
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(100); // 100ms周期

    while (1) {
        DoTask(); // 执行任务（假设耗时20ms）
        vTaskDelayUntil(&xLastWakeTime, xFrequency);
        // 实际周期始终 = 100ms（精确）
    }
}

// 时间转换宏
// pdMS_TO_TICKS(ms) - 毫秒转Tick
// pdTICKS_TO_MS(ticks) - Tick转毫秒
```

20.5 任务状态查询

```
#include "FreeRTOS.h"
#include "task.h"

// 任务状态查询函数
void Task_Status_Query(void) {
    TaskHandle_t taskHandle = xTaskGetHandle("LED Task");

    if (taskHandle != NULL) {
        // 获取任务名称
        const char *name = pcTaskGetName(taskHandle);
    }
}
```

```

// 获取任务优先级
UBaseType_t priority = uxTaskPriorityGet(taskHandle);

// 获取任务堆栈剩余空间
UBaseType_t stackWatermark = uxTaskGetStackHighWaterMark(taskHandle);

// 获取任务状态
eTaskState state = eTaskGetState(taskHandle);

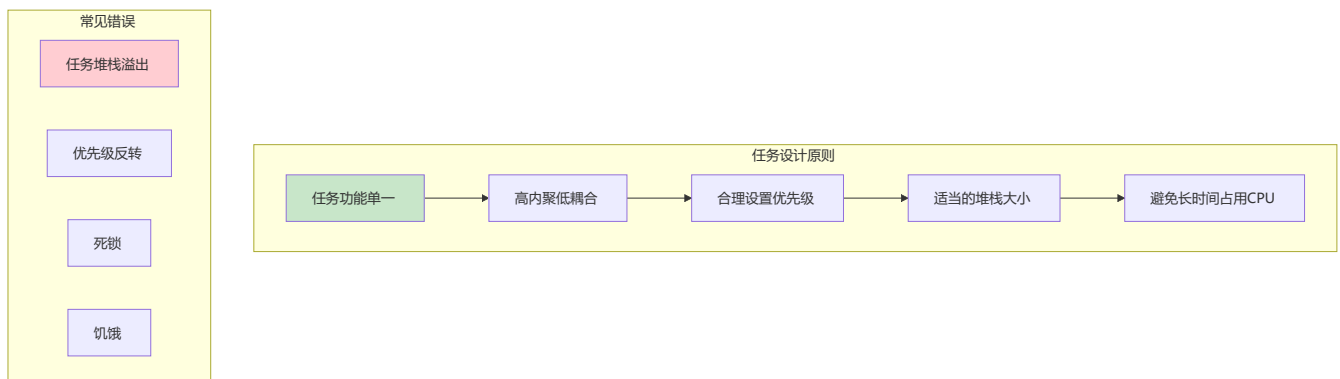
printf("任务: %s\n", name);
printf("优先级: %d\n", priority);
printf("堆栈剩余: %d 字\n", stackWatermark);
printf("状态: %d\n", state); // 0=运行, 1=就绪, 2=阻塞, 3=挂起, 4=删除
}
}

// 获取所有任务信息
void List_All_Tasks(void) {
    char *taskList = pvPortMalloc(1024);
    if (taskList != NULL) {
        vTaskList(taskList);
        printf("任务列表:\n%s\n", taskList);
        vPortFree(taskList);
    }
}

// 任务运行时间统计
void Task_Runtime_Stats(void) {
    char *stats = pvPortMalloc(512);
    if (stats != NULL) {
        vTaskGetRunTimeStats(stats);
        printf("任务运行时间:\n%s\n", stats);
        vPortFree(stats);
    }
}
}

```

20.6 任务设计最佳实践



任务设计建议

建议	说明
功能单一	每个任务只做一件事
合理延时	任务中必须有阻塞点
堆栈估算	根据局部变量和调用深度估算
优先级分配	实时性要求高的任务优先级高
避免忙等	使用信号量代替轮询

堆栈大小估算

```
// 堆栈大小估算方法
// 1. 局部变量占用
// 2. 函数调用深度（每层约需几十字节）
// 3. 中断嵌套（可能使用任务堆栈）
// 4. 预留安全裕量（通常加50%）

void Stack_Estimation_Task(void *pvParameters) {
    // 局部变量：约100字节
    uint8_t buffer[64];
    char message[32];
    float values[8];

    // 函数调用深度：假设3层，每层约32字节
    ProcessData(buffer); // 调用深度1

    // 建议堆栈大小：100 + 3*32 + 中断预留 + 安全裕量
    // = 100 + 96 + 100 + 200 = 496，取整为512字（2048字节）
}
```

第二十章小结

本章要点

- 1. 任务函数必须是无限循环，且有阻塞点
- 2. xTaskCreate创建任务，vTaskDelete删除任务
- 3. 优先级越高，越优先执行
- 4. vTaskDelay相对延时，vTaskDelayUntil绝对延时
- 5. 合理估算堆栈大小，避免溢出

动手练习

练习1：创建两个不同优先级的任务，观察执行顺序。

练习2：使用vTaskDelayUntil实现精确周期任务。

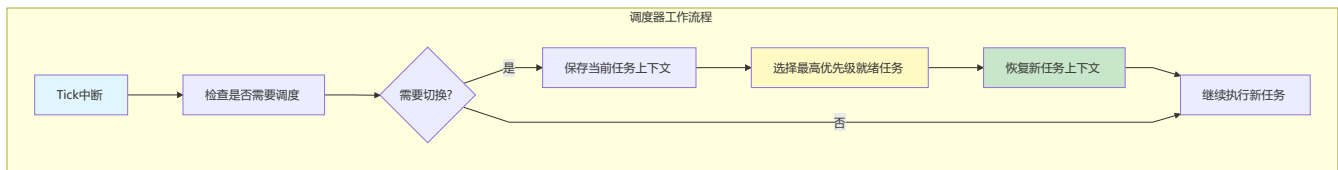
练习3：使用vTaskList查看所有任务状态。

第二十一章：调度机制

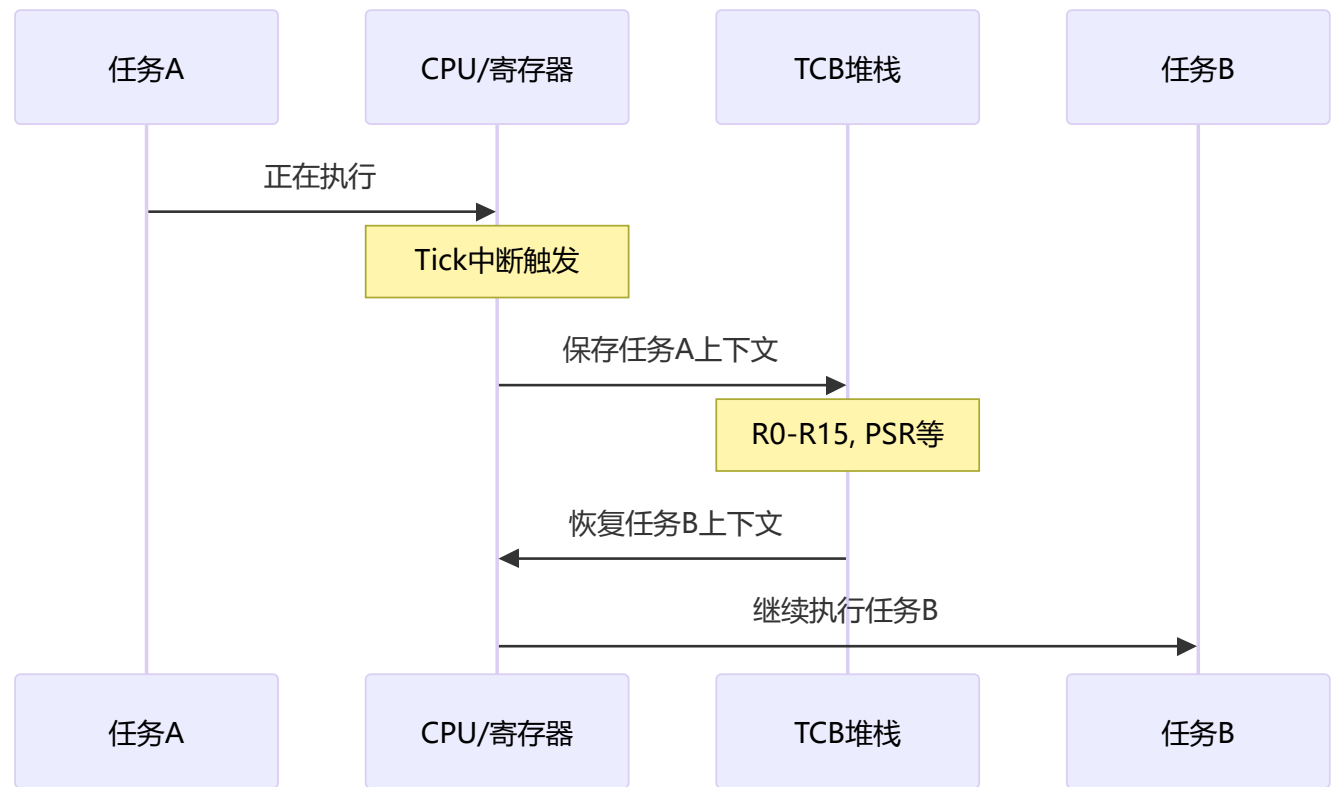
理解调度机制是掌握FreeRTOS的关键

21.1 调度器工作原理

调度器核心功能

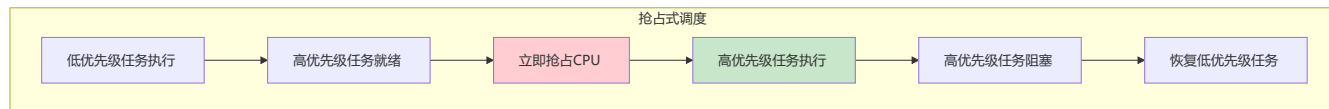


上下文切换



21.2 抢占式调度

抢占式调度原理



抢占式调度示例

```
#include "FreeRTOS.h"
#include "task.h"

TaskHandle_t lowPriorityHandle;
TaskHandle_t highPriorityHandle;

// 低优先级任务
void Low_Priority_Task(void *pvParameters) {
    while (1) {
        printf("低优先级任务运行\n");
        vTaskDelay(pdMS_TO_TICKS(500));
        // 此处可能被高优先级任务抢占
    }
}

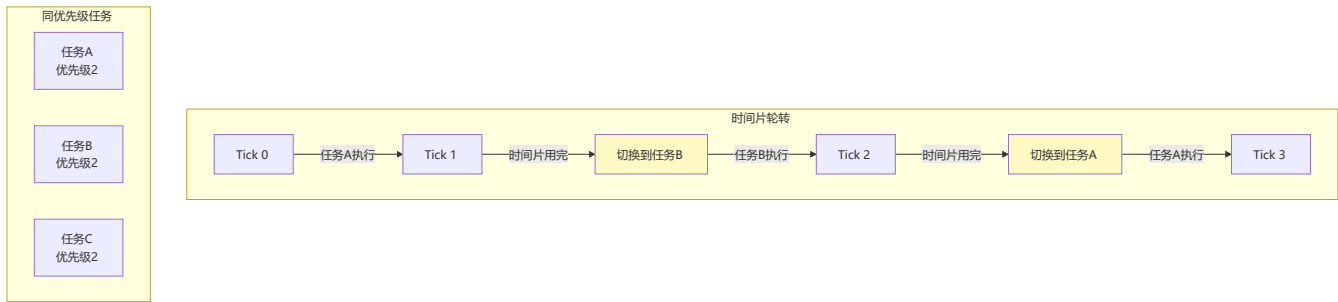
// 高优先级任务
void High_Priority_Task(void *pvParameters) {
    while (1) {
        printf("高优先级任务运行\n");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// 配置
void Setup_Tasks(void) {
    xTaskCreate(Low_Priority_Task, "Low", 128, NULL, 1, &lowPriorityHandle);
    xTaskCreate(High_Priority_Task, "High", 128, NULL, 3, &highPriorityHandle); // 更高优先级
}

// FreeRTOSConfig.h中配置
// #define configUSE_PREEMPTION 1 // 使能抢占式调度
```

21.3 时间片轮转

时间片轮转原理



时间片轮转示例

```
#include "FreeRTOS.h"
#include "task.h"

// 三个同优先级任务
void Task_A(void *pvParameters) {
    while (1) {
        printf("Task A\n");
        // 不主动延时，让时间片轮转
        for (volatile int i = 0; i < 100000; i++); // 模拟工作
    }
}

void Task_B(void *pvParameters) {
    while (1) {
        printf("Task B\n");
        for (volatile int i = 0; i < 100000; i++);
    }
}

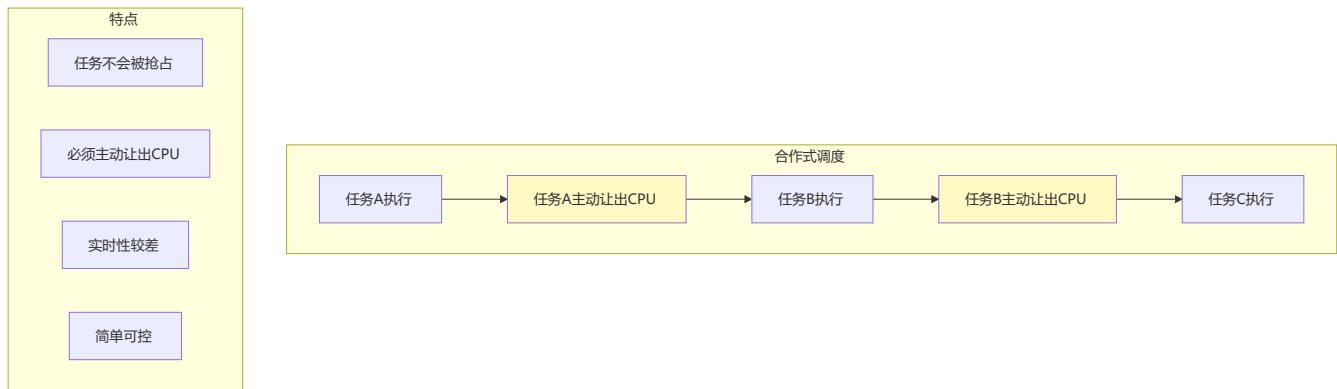
void Task_C(void *pvParameters) {
    while (1) {
        printf("Task C\n");
        for (volatile int i = 0; i < 100000; i++);
    }
}

void Setup_Timeslice(void) {
    // 三个任务优先级相同
    xTaskCreate(Task_A, "TaskA", 128, NULL, 2, NULL);
    xTaskCreate(Task_B, "TaskB", 128, NULL, 2, NULL);
    xTaskCreate(Task_C, "TaskC", 128, NULL, 2, NULL);
}

// FreeRTOSConfig.h中配置
// #define configUSE_TIME_SLICING 1 // 使能时间片轮转
// #define configTICK_RATE_HZ 1000 // 1ms一个时间片
```

21.4 合作式调度

合作式调度原理



合作式调度示例

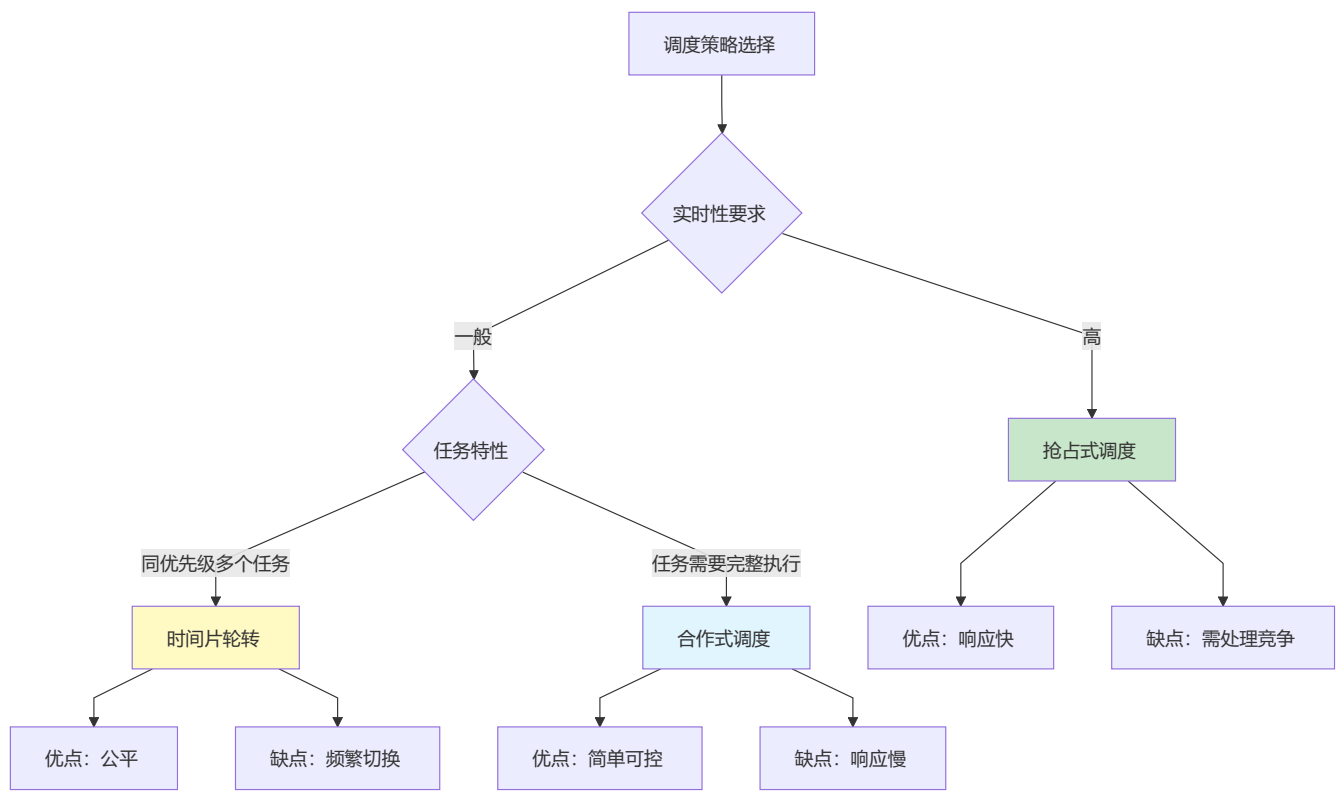
```
#include "FreeRTOS.h"
#include "task.h"

// 合作式调度：任务必须主动让出CPU
void Cooperative_Task_A(void *pvParameters) {
    while (1) {
        printf("Task A working...\n");
        // 做一些工作
        taskYIELD(); // 主动让出CPU
    }
}

void Cooperative_Task_B(void *pvParameters) {
    while (1) {
        printf("Task B working...\n");
        // 做一些工作
        taskYIELD(); // 主动让出CPU
    }
}

// FreeRTOSConfig.h中配置
// #define configUSE_PREEMPTION 0 // 关闭抢占式调度
```

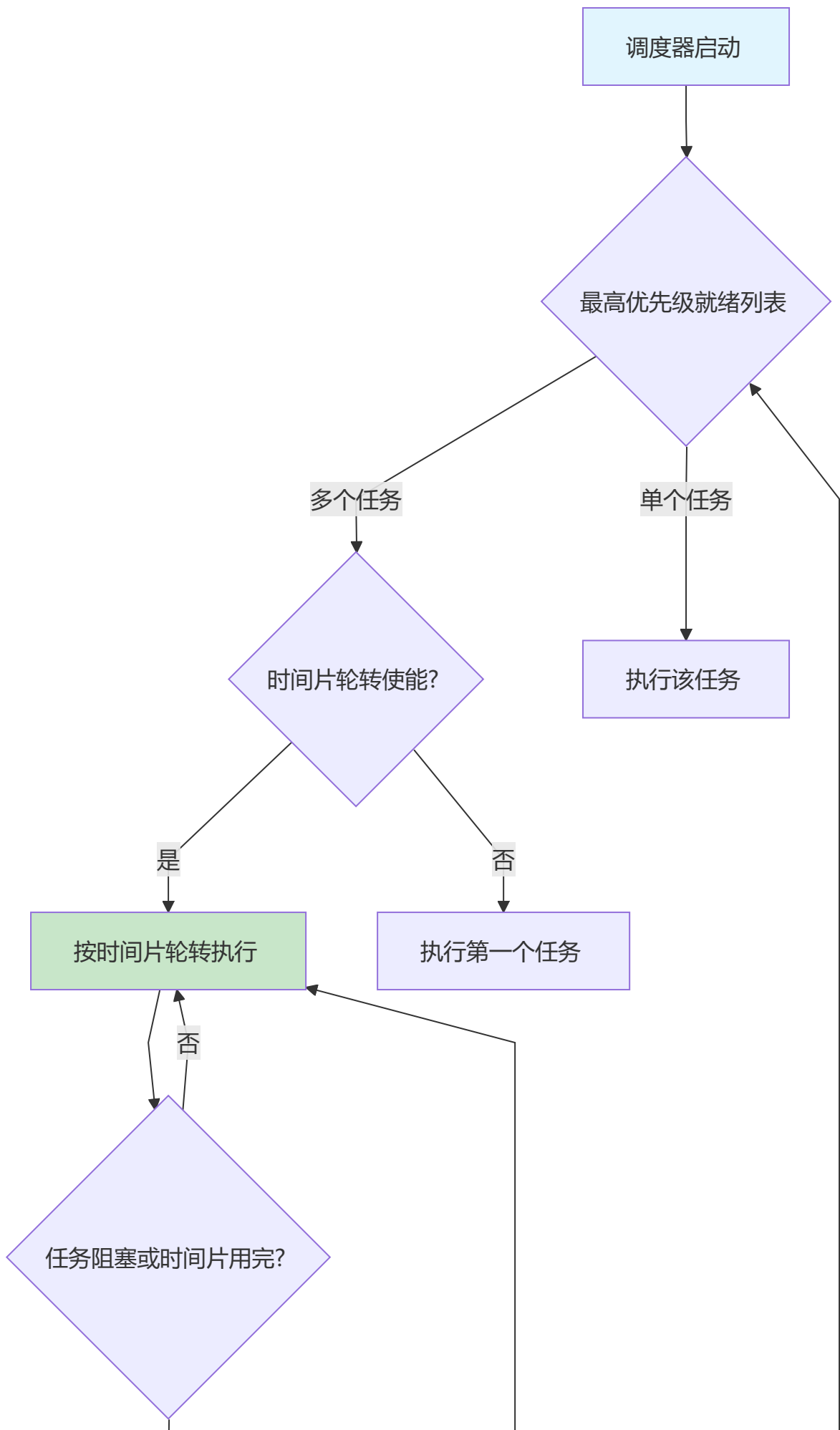

21.5 调度策略对比

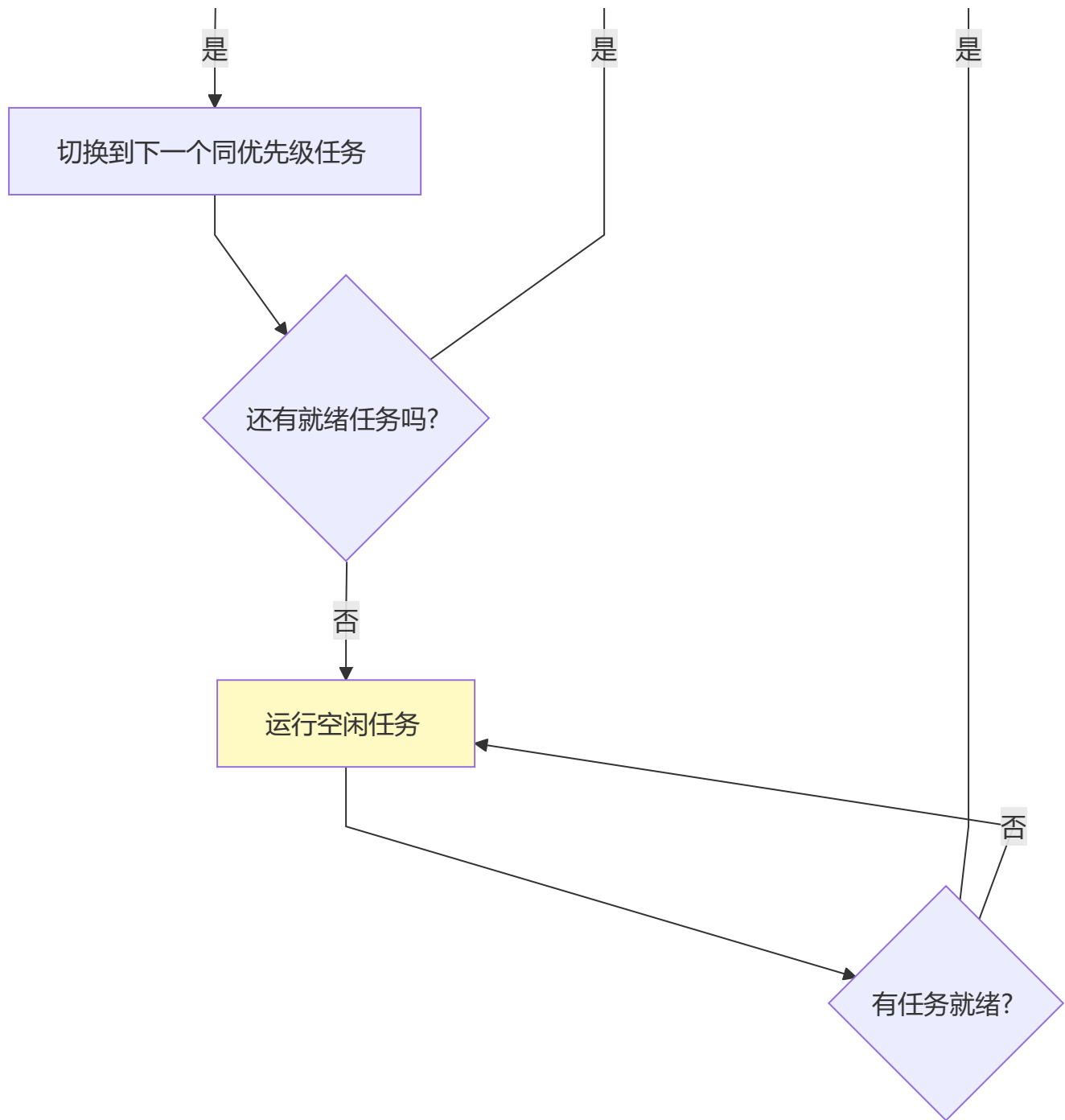


调度策略对比表

策略	响应时间	公平性	复杂度	适用场景
抢占式	快	一般	高	实时系统
时间片轮转	中等	好	中	多任务公平执行
合作式	慢	差	低	简单系统

21.6 任务调度流程图





第二十一章小结

本章要点

1. 抢占式调度：高优先级任务立即抢占CPU
2. 时间片轮转：同优先级任务轮流执行
3. 合作式调度：任务主动让出CPU
4. 大多数情况使用抢占式+时间片轮转
5. 通过配置项选择调度策略

动手练习

练习1：观察抢占式调度下高优先级任务的抢占行为。

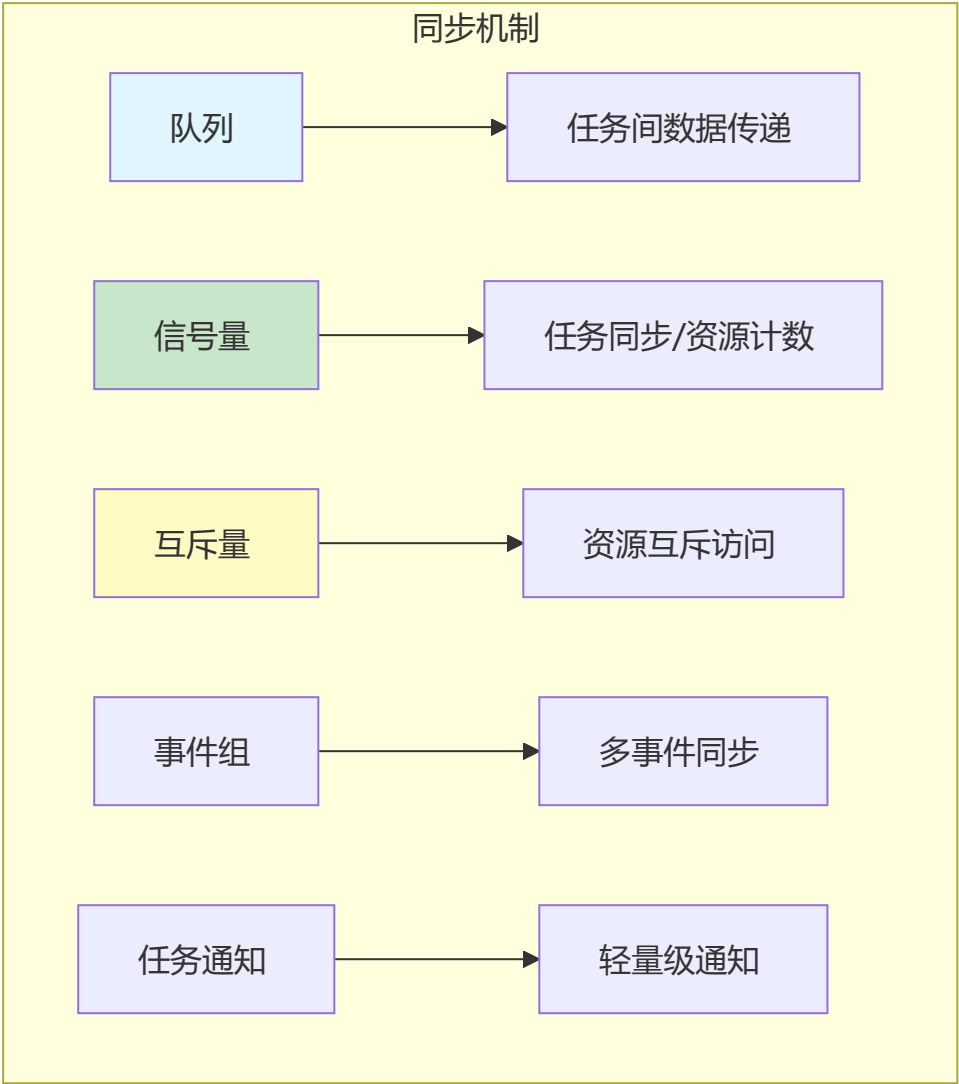
练习2：创建多个同优先级任务，观察时间片轮转。

练习3：尝试合作式调度模式，理解其特点。

第二十二章：同步与通信机制

任务间的协作与数据交换是RTOS的核心能力

22.1 同步机制概述

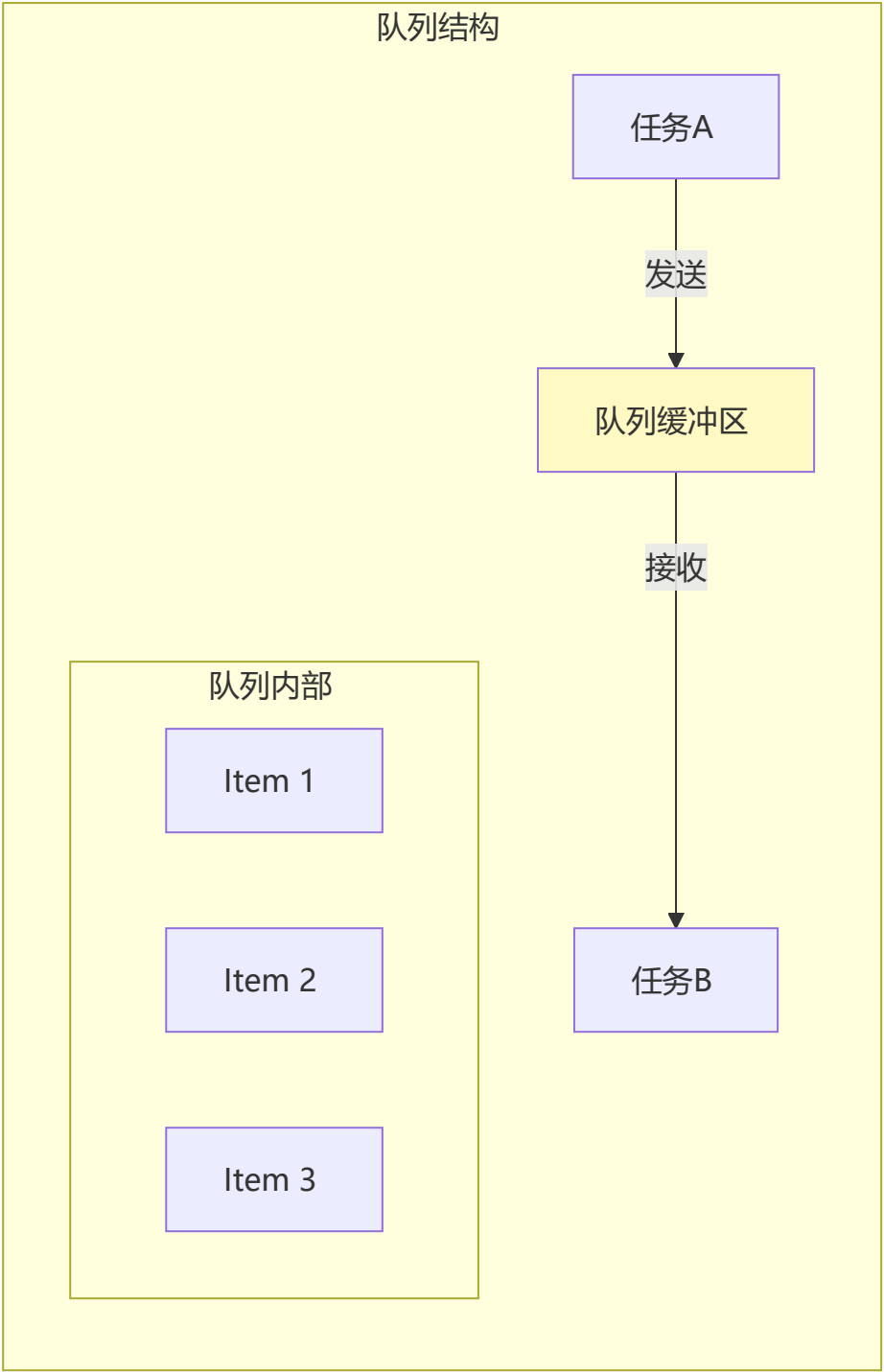
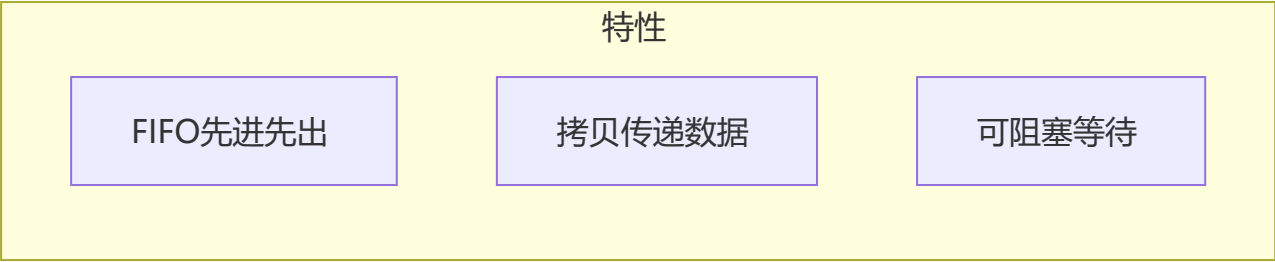


机制选择指南

机制	用途	特点
队列	数据传递	可传递数据，有缓冲
二值信号量	同步	只能传递事件
计数信号量	资源计数	可计数多次事件
互斥量	互斥访问	有优先级继承
事件组	多事件同步	可等待多个事件
任务通知	轻量通知	速度最快

22.2 队列（Queue）

队列原理



队列基本操作

```
#include "FreeRTOS.h"
#include "queue.h"

// 定义队列句柄
QueueHandle_t xQueue;

// 定义数据类型
typedef struct {
    uint8_t id;
    float value;
    uint32_t timestamp;
} SensorData_t;

// 创建队列
void Queue_Create(void) {
    // 创建能存10个SensorData_t的队列
    xQueue = xQueueCreate(10, sizeof(SensorData_t));

    if (xQueue == NULL) {
        printf("队列创建失败\n");
    }
}

// 发送任务
void Sender_Task(void *pvParameters) {
    SensorData_t data;
    BaseType_t result;

    while (1) {
        // 采集数据
        data.id = 1;
        data.value = ReadSensor();
        data.timestamp = xTaskGetTickCount();

        // 发送到队列（等待最多100ms）
        result = xQueueSend(xQueue, &data, pdMS_TO_TICKS(100));

        if (result != pdPASS) {
            printf("队列发送失败\n");
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// 接收任务
void Receiver_Task(void *pvParameters) {
    SensorData_t receivedData;
    BaseType_t result;
```



```

while (1) {
    // 从队列接收（无限等待）
    result = xQueueReceive(xQueue, &receivedData, portMAX_DELAY);

    if (result == pdPASS) {
        printf("收到数据: ID=%d, 值=%.2f\n",
            receivedData.id, receivedData.value);
    }
}
}

```

队列操作函数

```

// 创建队列
QueueHandle_t xQueueCreate(UBaseType_t uxQueueLength, UBaseType_t uxItemSize);

// 发送（队尾）
BaseType_t xQueueSend(QueueHandle_t xQueue, const void *pvItemToQueue, TickType_t xTicksToWait);
BaseType_t xQueueSendToBack(QueueHandle_t xQueue, const void *pvItemToQueue, TickType_t xTicksToWait);

// 发送到队首
BaseType_t xQueueSendToFront(QueueHandle_t xQueue, const void *pvItemToQueue, TickType_t xTicksToWait);

// 接收（队首）
BaseType_t xQueueReceive(QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);

// 查看队首（不移除）
BaseType_t xQueuePeek(QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);

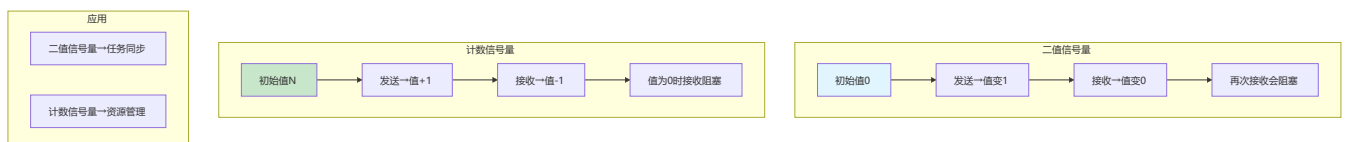
// 查询队列状态
UBaseType_t uxQueueMessagesWaiting(QueueHandle_t xQueue); // 消息数
UBaseType_t uxQueueSpacesAvailable(QueueHandle_t xQueue); // 空闲空间

// 删除队列
void vQueueDelete(QueueHandle_t xQueue);

```

22.3 信号量（Semaphore）

信号量类型



二值信号量示例

```
#include "FreeRTOS.h"
#include "semphr.h"

SemaphoreHandle_t xBinarySemaphore;

// 创建二值信号量
void BinarySemaphore_Create(void) {
    xBinarySemaphore = xSemaphoreCreateBinary();

    if (xBinarySemaphore == NULL) {
        printf("二值信号量创建失败\n");
    }
}

// 中断中释放信号量
void EXTI0_IRQHandler(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // 在中断中释放信号量
    xSemaphoreGiveFromISR(xBinarySemaphore, &xHigherPriorityTaskWoken);

    // 如果唤醒了更高优先级任务，触发上下文切换
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);

    // 清除中断标志
    EXTI_ClearITPendingBit(EXTI_Line0);
}

// 任务中等待信号量
void ISR_Handler_Task(void *pvParameters) {
    while (1) {
        // 等待信号量（无限等待）
        if (xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdTRUE) {
            printf("中断事件发生\n");
            // 处理中断事件
        }
    }
}
```

计数信号量示例

```
#include "FreeRTOS.h"
#include "semphr.h"

SemaphoreHandle_t xCountingSemaphore;

// 创建计数信号量
void CountingSemaphore_Create(void) {
    // 最大计数10，初始计数0
```

```
xCountingSemaphore = xSemaphoreCreateCounting(10, 0);
}

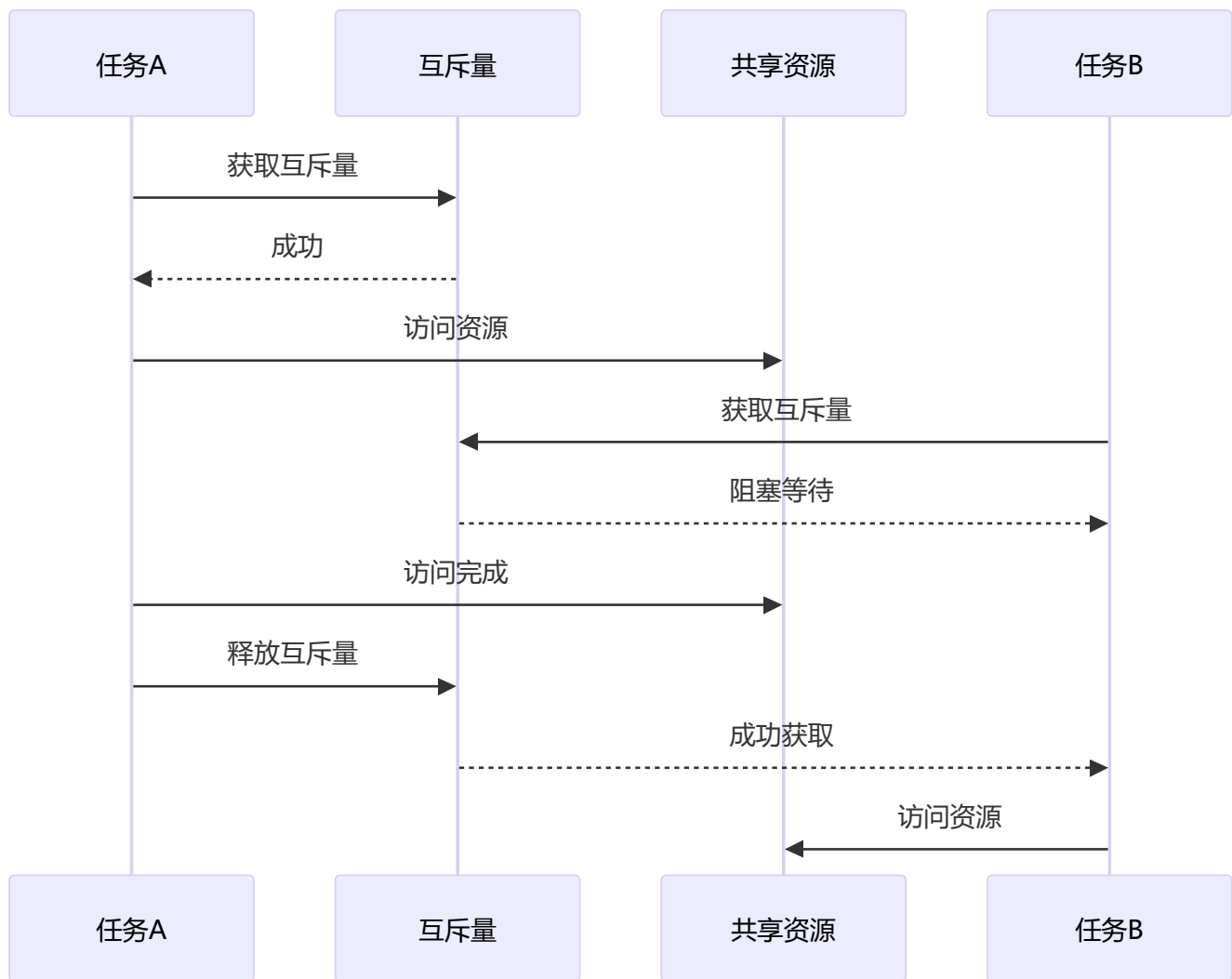
// 生产者任务
void Producer_Task(void *pvParameters) {
    while (1) {
        // 生产事件
        printf("生产事件\n");
        xSemaphoreGive(xCountingSemaphore); // 计数+1

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

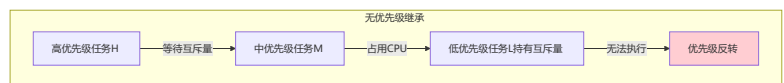
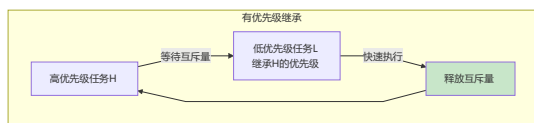
// 消费者任务
void Consumer_Task(void *pvParameters) {
    while (1) {
        // 等待事件
        if (xSemaphoreTake(xCountingSemaphore, portMAX_DELAY) == pdTRUE) {
            printf("消费事件\n"); // 计数-1
        }
    }
}
```

22.4 互斥量 (Mutex)

互斥量原理



优先级继承



互斥量示例

```

#include "FreeRTOS.h"
#include "semphr.h"

SemaphoreHandle_t xMutex;

// 创建互斥量
void Mutex_Create(void) {
    xMutex = xSemaphoreCreateMutex();

    if (xMutex == NULL) {
        printf("互斥量创建失败\n");
    }
}
  
```

```
// 共享资源（如串口）
void Print_Safe(const char *message) {
    // 获取互斥量
    if (xSemaphoreTake(xMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
        // 访问共享资源
        printf("%s", message);

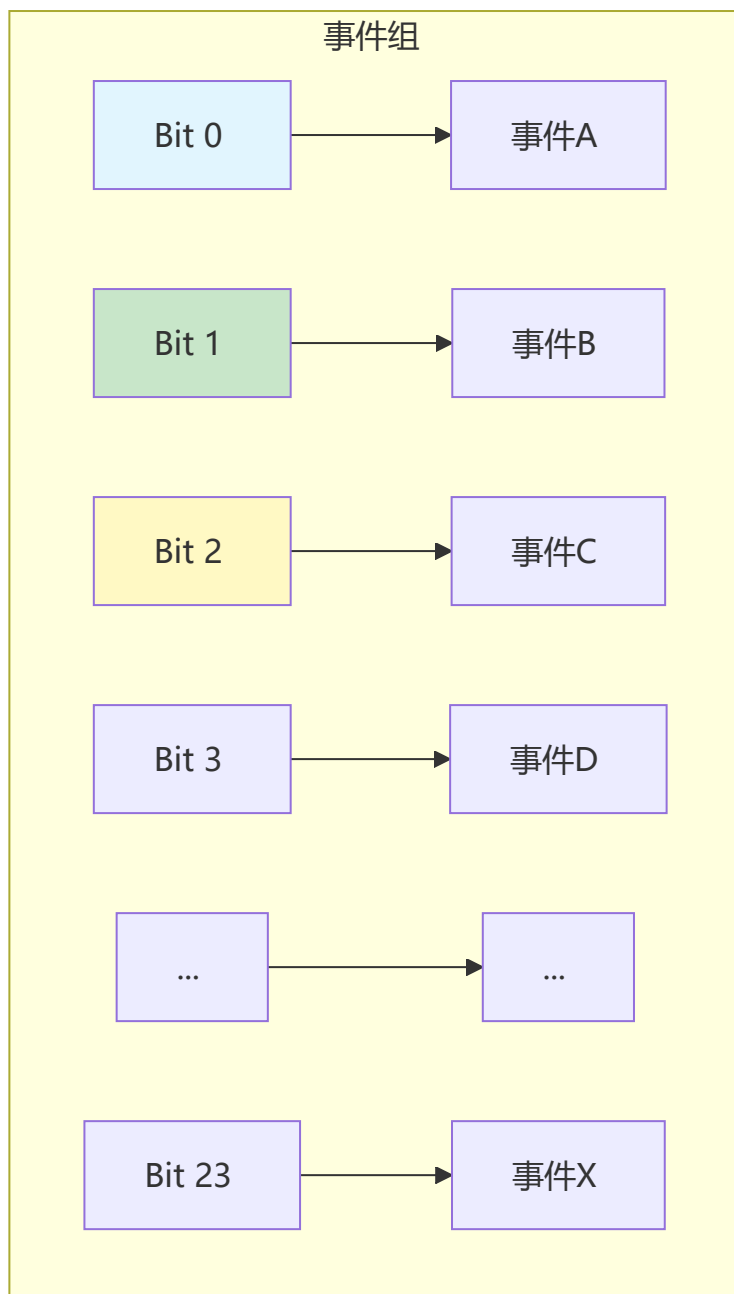
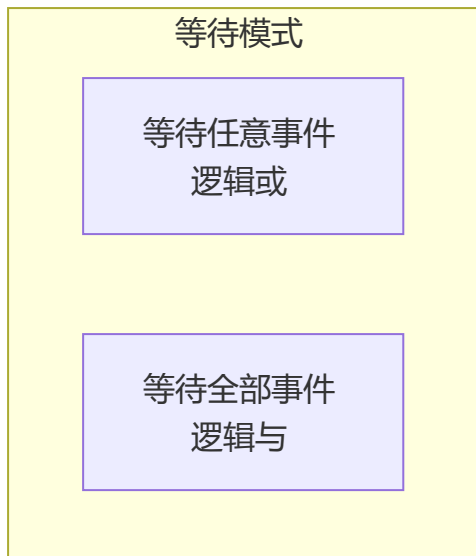
        // 释放互斥量
        xSemaphoreGive(xMutex);
    }
}

// 任务1
void Task1(void *pvParameters) {
    while (1) {
        Print_Safe("Task1 正在运行\n");
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

// 任务2
void Task2(void *pvParameters) {
    while (1) {
        Print_Safe("Task2 正在运行\n");
        vTaskDelay(pdMS_TO_TICKS(300));
    }
}
```

22.5 事件组 (Event Group)

事件组原理



事件组示例

```
#include "FreeRTOS.h"
#include "event_groups.h"

EventGroupHandle_t xEventGroup;

// 定义事件位
#define EVENT_WIFI_CONNECTED    (1 << 0)
#define EVENT_SERVER_READY     (1 << 1)
#define EVENT_DATA_READY       (1 << 2)

// 创建事件组
void EventGroup_Create(void) {
    xEventGroup = xEventGroupCreate();
}
```

```

    if (xEventGroup == NULL) {
        printf("事件组创建失败\n");
    }
}

// WiFi任务: 设置WiFi连接事件
void WiFi_Task(void *pvParameters) {
    while (1) {
        // 连接WiFi
        ConnectWiFi();

        // 设置事件位
        xEventGroupSetBits(xEventGroup, EVENT_WIFI_CONNECTED);

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// 服务器任务: 设置服务器就绪事件
void Server_Task(void *pvParameters) {
    while (1) {
        // 连接服务器
        ConnectServer();

        // 设置事件位
        xEventGroupSetBits(xEventGroup, EVENT_SERVER_READY);

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// 主任务: 等待所有条件满足
void Main_Task(void *pvParameters) {
    EventBits_t uxBits;

    while (1) {
        // 等待WiFi和服务器都就绪
        uxBits = xEventGroupWaitBits(
            xEventGroup,                                // 事件组句柄
            EVENT_WIFI_CONNECTED | EVENT_SERVER_READY,  // 等待的位
            pdTRUE,                                       // 退出时清除位
            pdTRUE,                                       // 等待所有位 (AND)
            portMAX_DELAY                                 // 无限等待
        );

        if ((uxBits & (EVENT_WIFI_CONNECTED | EVENT_SERVER_READY)) ==
            (EVENT_WIFI_CONNECTED | EVENT_SERVER_READY)) {
            printf("WiFi和服务器都已就绪, 开始通信\n");

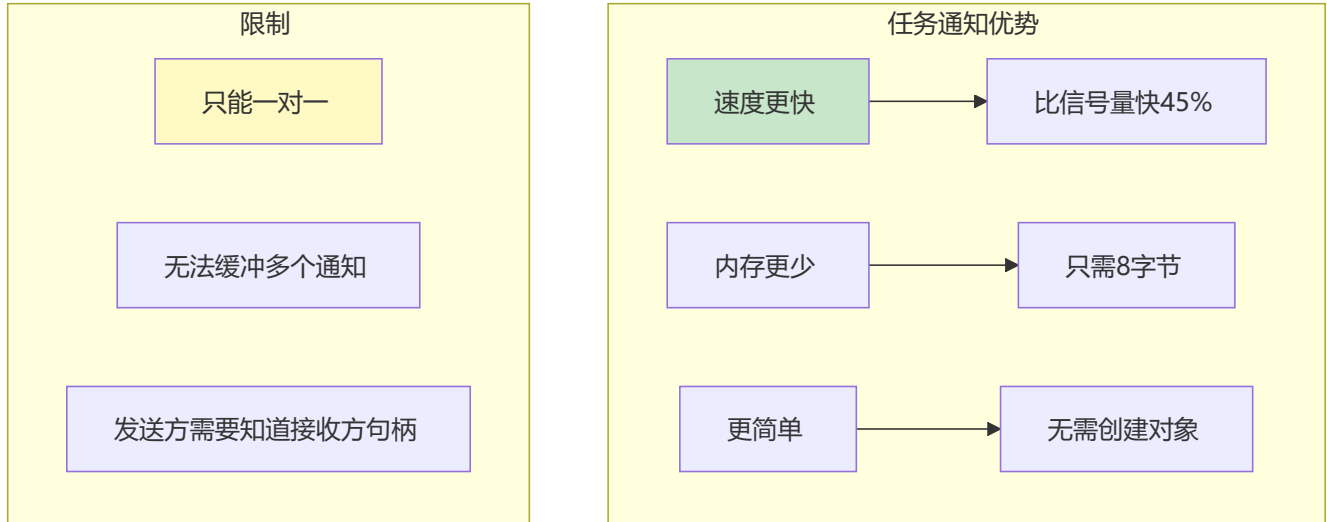
            // 开始数据通信
            // ...
        }
    }
}

```

```
}
```

22.6 任务通知 (Task Notification)

任务通知特点



任务通知示例

```
#include "FreeRTOS.h"
#include "task.h"

TaskHandle_t xHandlerTaskHandle;

// 发送通知的任务
void Notifier_Task(void *pvParameters) {
    while (1) {
        // 等待某个事件
        vTaskDelay(pdMS_TO_TICKS(1000));

        // 发送通知给Handler任务
        // 参数: 任务句柄, 通知值, 增值方式
        xTaskNotifyGive(xHandlerTaskHandle);

        // 或者发送带值的通知
        // xTaskNotify(xHandlerTaskHandle, 0x01, eSetBits);
    }
}

// 接收通知的任务
void Handler_Task(void *pvParameters) {
    uint32_t ulNotificationValue;

    while (1) {
        // 等待通知 (类似二值信号量)
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
```

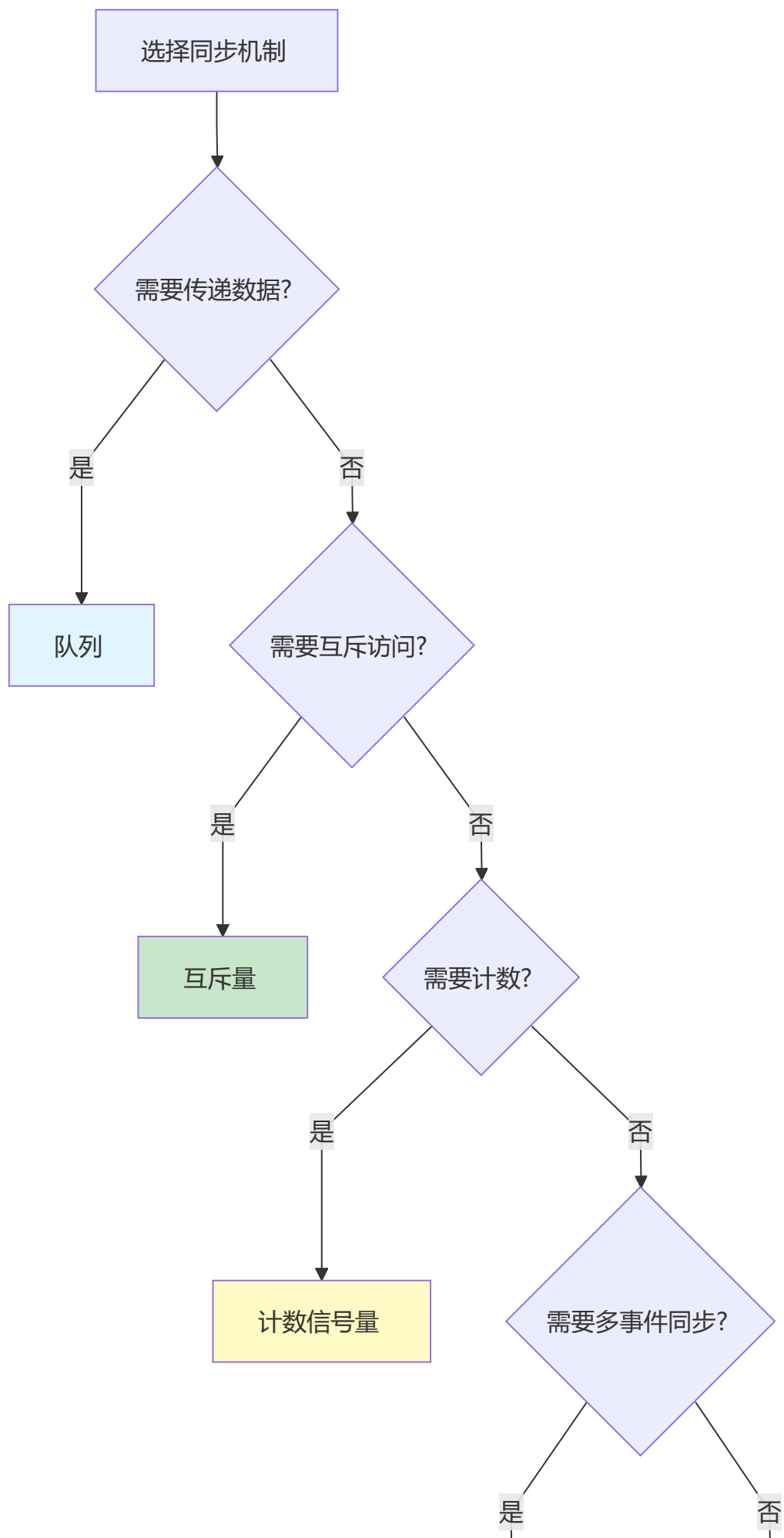


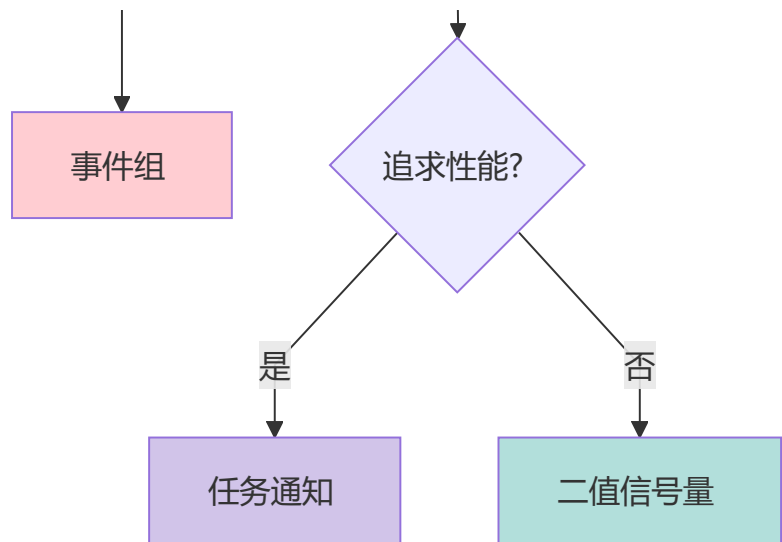
```
printf("收到通知, 开始处理\n");

// 或者等待带值的通知
// xTaskNotifyWait(0x00, 0xFFFFFFFF, &ulNotificationValue, portMAX_DELAY);
// printf("收到通知值: 0x%02X\n", ulNotificationValue);
}
}

// 创建任务
void Create_Notification_Tasks(void) {
    // 必须先创建接收任务以获取句柄
    xTaskCreate(Handler_Task, "Handler", 128, NULL, 2, &xHandlerTaskHandle);
    xTaskCreate(Notifier_Task, "Notifier", 128, NULL, 1, NULL);
}
```

22.7 同步机制对比总结





机制选择速查表

需求	推荐机制	原因
任务间传递数据	队列	支持数据拷贝传递
中断通知任务	二值信号量/任务通知	简单高效
保护共享资源	互斥量	有优先级继承
资源计数	计数信号量	可计数
等待多个事件	事件组	支持位操作
高性能通知	任务通知	速度最快

第二十二章小结

本章要点

1. 队列用于任务间数据传递，FIFO机制
2. 二值信号量用于同步，计数信号量用于资源计数
3. 互斥量保护共享资源，有优先级继承
4. 事件组可等待多个事件
5. 任务通知是最轻量级的同步方式

动手练习

练习1：使用队列实现生产者-消费者模型。

练习2：使用互斥量保护串口打印。

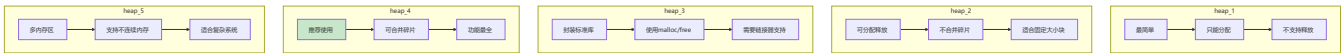
练习3：使用事件组实现多条件等待。

第二十三章：内存管理

合理的内存管理是系统稳定运行的基础

23.1 FreeRTOS内存管理方案

五种内存方案对比



方案对比表

方案	分配	释放	合并碎片	复杂度	推荐场景
heap_1	✓	✗	N/A	最低	只创建不删除
heap_2	✓	✓	✗	低	固定大小块
heap_3	✓	✓	取决于库	中	使用标准库
heap_4	✓	✓	✓	中	推荐使用
heap_5	✓	✓	✓	高	多内存区域

23.2 内存分配函数

基本API

```
#include "FreeRTOS.h"
#include "task.h"

// 动态内存分配
void *pvPortMalloc(size_t xSize);

// 释放内存
void vPortFree(void *pv);

// 获取剩余堆大小
size_t xPortGetFreeHeapSize(void);

// 获取历史最小剩余堆
size_t xPortGetMinimumEverFreeHeapSize(void);

// 获取堆状态 (heap_5专用)
void vPortGetHeapStats(HeapStats_t *pxHeapStats);
```

内存分配示例

```
#include "FreeRTOS.h"
#include "task.h"

void Memory_Example(void) {
    // 分配内存
    uint8_t *buffer = (uint8_t *)pvPortMalloc(256);

    if (buffer != NULL) {
        // 使用内存
        memset(buffer, 0, 256);

        // 释放内存
        vPortFree(buffer);
        buffer = NULL; // 防止悬空指针
    } else {
        printf("内存分配失败\n");
        printf("剩余堆: %d 字节\n", xPortGetFreeHeapSize());
    }
}

// 内存使用统计任务
void Memory_Monitor_Task(void *pvParameters) {
    size_t freeHeap, minFreeHeap;

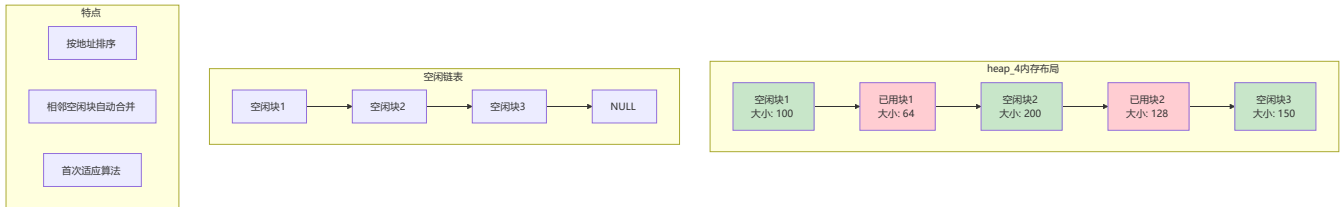
    while (1) {
        freeHeap = xPortGetFreeHeapSize();
        minFreeHeap = xPortGetMinimumEverFreeHeapSize();

        printf("当前剩余堆: %d 字节\n", freeHeap);
        printf("历史最小剩余: %d 字节\n", minFreeHeap);

        vTaskDelay(pdMS_TO_TICKS(5000));
    }
}
```

23.3 heap_4详解（推荐方案）

heap_4内存结构



heap_4配置

```
// FreeRTOSConfig.h

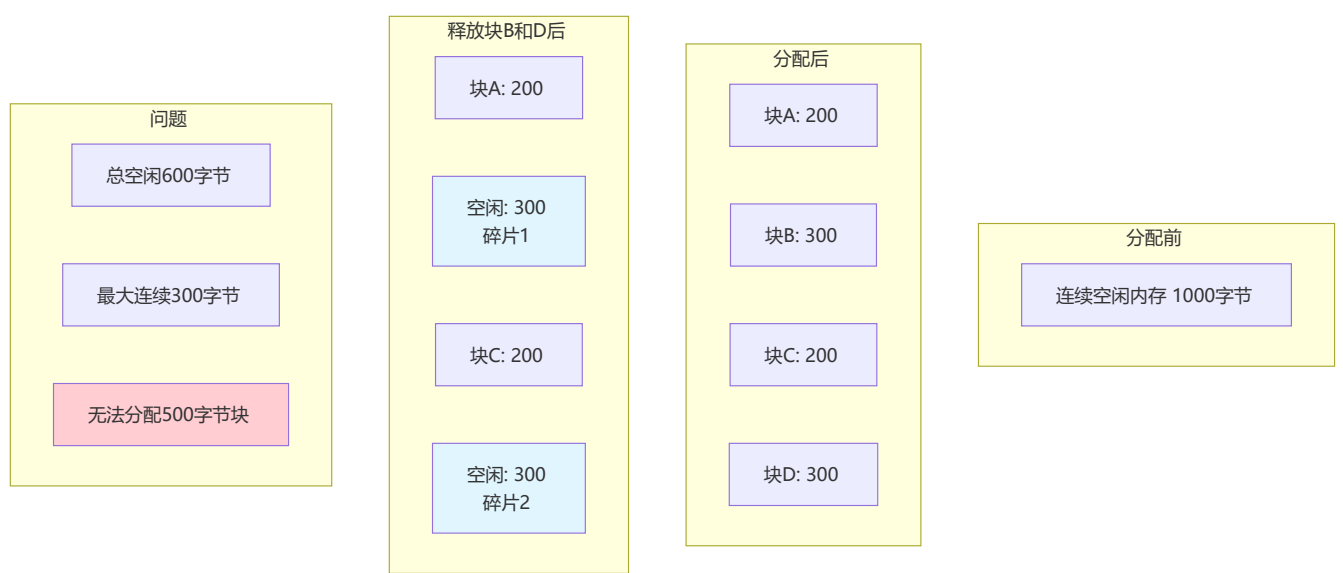
// 总堆大小
#define configTOTAL_HEAP_SIZE ((size_t)(20 * 1024)) // 20KB

// 应用程序自己分配堆空间（可选）
#define configAPPLICATION_ALLOCATED_HEAP 0

// 如果使能，需要自己定义堆数组
// uint8_t ucHeap[configTOTAL_HEAP_SIZE];
```

23.4 内存碎片处理

碎片问题



减少碎片的方法

```
// 1. 使用固定大小内存块
#define BUFFER_SIZE 256
typedef struct {
    uint8_t data[BUFFER_SIZE];
} Buffer_t;

// 2. 预分配内存池
#define POOL_SIZE 10
Buffer_t bufferPool[POOL_SIZE];
uint8_t bufferUsed[POOL_SIZE] = {0};

Buffer_t* GetBuffer(void) {
    for (int i = 0; i < POOL_SIZE; i++) {
        if (!bufferUsed[i]) {
```

```

        bufferUsed[i] = 1;
        return &bufferPool[i];
    }
}
return NULL;
}

void ReleaseBuffer(Buffer_t *buf) {
    int index = buf - bufferPool;
    if (index >= 0 && index < POOL_SIZE) {
        bufferUsed[index] = 0;
    }
}

// 3. 长时间存在的对象尽早分配
void System_Init(void) {
    // 启动时分配所有长期对象
    // 这样碎片只会出现在堆的尾部
}

```

23.5 堆栈溢出检测

堆栈溢出检测配置

```

// FreeRTOSConfig.h

// 使能堆栈溢出检测
#define configCHECK_FOR_STACK_OVERFLOW 2 // 0=禁用, 1=方法1, 2=方法2

// 堆栈溢出钩子函数
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
    printf("堆栈溢出! 任务: %s\n", pcTaskName);

    // 可以在这里:
    // 1. 记录日志
    // 2. 复位系统
    // 3. 进入安全模式

    while (1); // 死循环或复位
}

```

堆栈监控

```

// 监控任务堆栈使用情况
void Stack_Monitor_Task(void *pvParameters) {
    UBaseType_t stackWatermark;
    TaskStatus_t *taskList;
    UBaseType_t taskCount, i;

    while (1) {

```



```

// 获取任务数量
taskCount = uxTaskGetNumberOfTasks();

// 分配内存存储任务状态
taskList = pvPortMalloc(taskCount * sizeof(TaskStatus_t));

if (taskList != NULL) {
    // 获取所有任务状态
    uxTaskGetSystemState(taskList, taskCount, NULL);

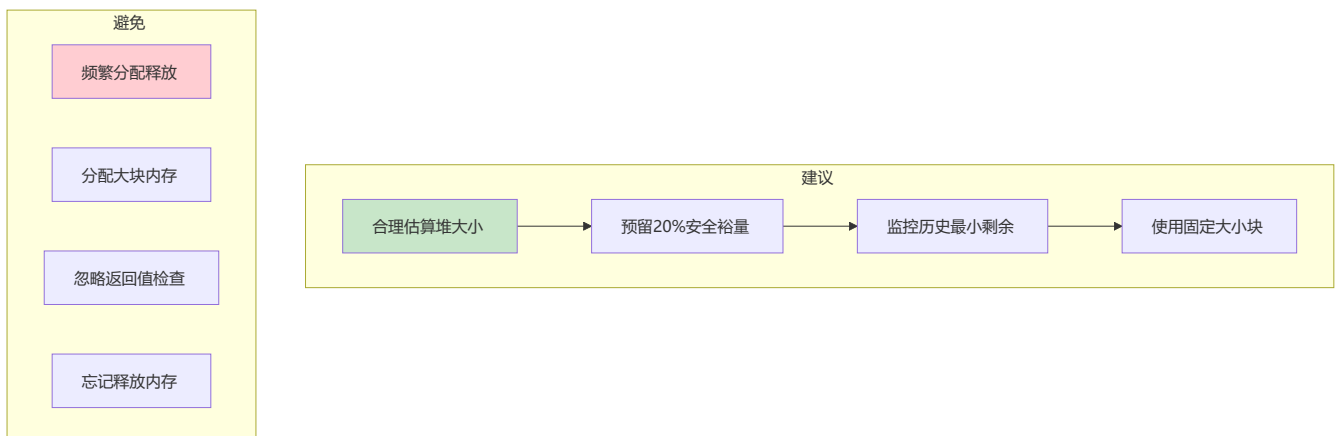
    printf("\n=== 堆栈使用报告 ===\n");
    for (i = 0; i < taskCount; i++) {
        stackwatermark = taskList[i].usStackHighWaterMark;
        printf("%-16s 优先级:%d 堆栈剩余:%d 字\n",
            taskList[i].pcTaskName,
            taskList[i].uxCurrentPriority,
            stackwatermark);
    }

    vPortFree(taskList);
}

vTaskDelay(pdMS_TO_TICKS(10000)); // 10秒检查一次
}
}

```

23.6 内存管理最佳实践



最佳实践代码

```

// 1. 检查分配结果
void Safe_Allocation(void) {
    uint8_t *ptr = (uint8_t *)pvPortMalloc(1024);

    if (ptr == NULL) {
        // 处理分配失败
        printf("内存不足!\n");
        // 可以释放一些非必要内存或重启系统
    }
}

```

```

        return;
    }

    // 使用内存...

    // 2. 使用后释放
    vPortFree(ptr);
    ptr = NULL;    // 避免悬空指针
}

// 3. 堆栈大小估算
// 局部变量 + 函数调用深度(每层约50字节) + 中断上下文 + 安全裕量
#define TASK_STACK_SIZE ((128 + 200 + 100) * 1.5)    // 约640字

// 4. 启动时检查内存充足
void Check_Memory_Available(void) {
    size_t required = 1024;    // 需要的内存
    size_t available = xPortGetFreeHeapSize();

    if (available < required) {
        printf("内存不足！需要：%d，可用：%d\n", required, available);
        // 处理错误
    }
}

```

第二十三章小结

本章要点

1. heap_4是最推荐的内存方案，支持碎片合并
2. 堆大小在FreeRTOSConfig.h中配置
3. 使用pvPortMalloc分配，vPortFree释放
4. 监控历史最小剩余堆，预防内存不足
5. 开启堆栈溢出检测，及时发现问题

动手练习

练习1：观察内存分配前后堆的变化。

练习2：故意触发堆栈溢出，观察检测效果。

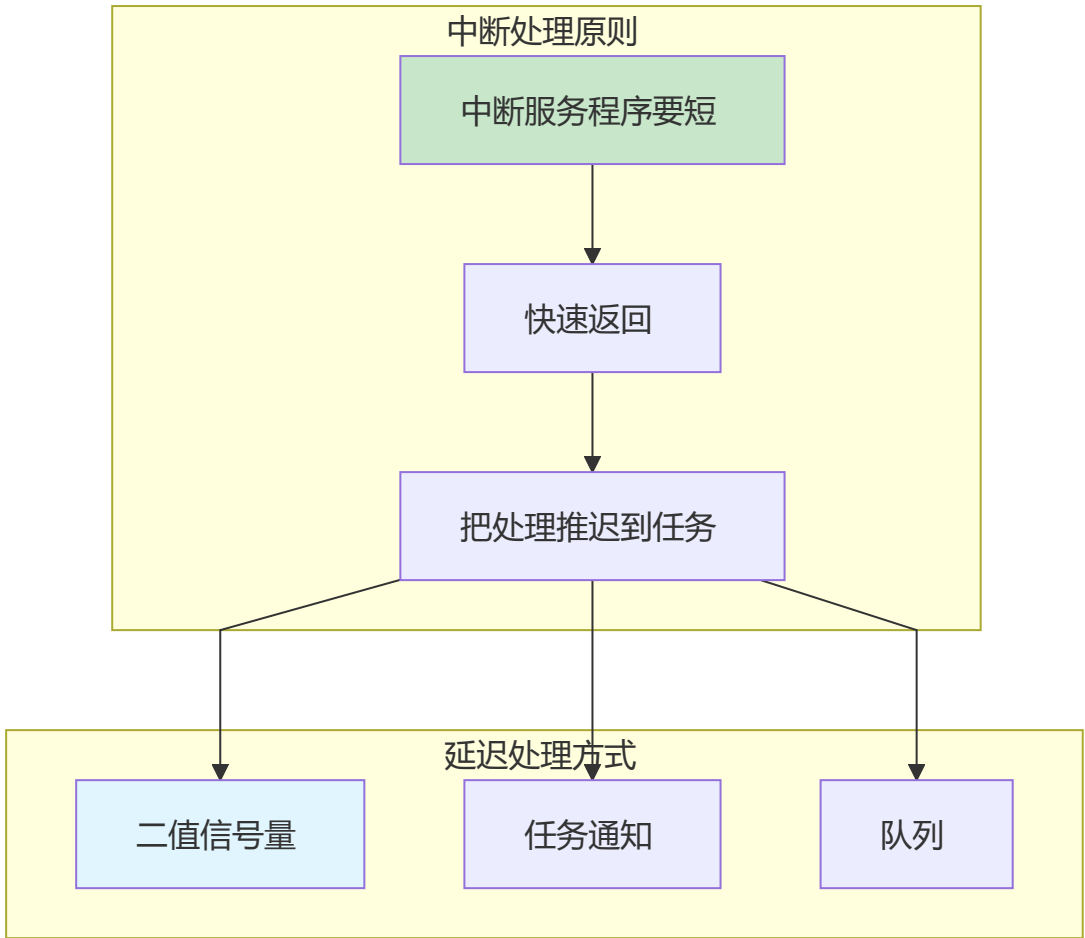
练习3：实现一个固定大小的内存池。

第二十四章：中断管理与定时器

中断和定时器是实时系统的时间基础

24.1 中断管理原则

中断处理原则



中断优先级配置

```
// FreeRTOSConfig.h中的中断优先级配置

// Cortex-M架构有4位优先级（0-15）
// 数值越大，优先级越低

// FreeRTOS可管理的最高中断优先级
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5

// 实际使用
// 优先级0-4： 不受FreeRTOS管理（可打断临界区）
// 优先级5-15： 受FreeRTOS管理（可调用FromISR函数）
```

24.2 中断安全API

带FromISR后缀的函数

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"

// 中断中使用的变量
volatile uint8_t flag = 0;

// 正确示例：使用FromISR函数
void EXTI0_IRQHandler(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // ✅ 使用FromISR版本的函数
    xSemaphoreGiveFromISR(xBinarySemaphore, &xHigherPriorityTaskWoken);

    // 如果唤醒了更高优先级任务，请求上下文切换
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);

    // 清除中断标志
    EXTI_ClearITPendingBit(EXTI_Line0);
}

// 错误示例
void EXTI0_IRQHandler_Bad(void) {
    // ❌ 不能在中断中调用普通API
    // xSemaphoreGive(xBinarySemaphore); // 错误！

    // ❌ 不能在中断中延时
    // vTaskDelay(10); // 错误！
}
```

常用FromISR函数

普通函数	中断函数
xQueueSend()	xQueueSendFromISR()
xQueueReceive()	xQueueReceiveFromISR()
xSemaphoreGive()	xSemaphoreGiveFromISR()
xSemaphoreTake()	xSemaphoreTakeFromISR()
xTaskNotifyGive()	vTaskNotifyGiveFromISR()
xTaskNotify()	vTaskNotifyFromISR()

24.3 延迟中断处理

使用二值信号量

```
#include "FreeRTOS.h"
#include "semphr.h"
#include "task.h"

SemaphoreHandle_t xISRSemaphore;
TaskHandle_t xISRHandlerTaskHandle;

// 中断处理任务
void ISR_Handler_Task(void *pvParameters) {
    while (1) {
        // 等待中断通知
        if (xSemaphoreTake(xISRSemaphore, portMAX_DELAY) == pdTRUE) {
            // 在任务中处理中断事件（可以延时、调用任何API）
            ProcessInterruptEvent();
        }
    }
}

// 外部中断
void EXTI0_IRQHandler(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // 只做最少的工作
    // 通知任务处理
    xSemaphoreGiveFromISR(xISRSemaphore, &xHigherPriorityTaskWoken);

    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    EXTI_ClearITPendingBit(EXTI_Line0);
}

// 初始化
void ISR_Handler_Init(void) {
    xISRSemaphore = xSemaphoreCreateBinary();
    xTaskCreate(ISR_Handler_Task, "ISR_Handler", 256, NULL, 3, &xISRHandlerTaskHandle);
}
```

使用任务通知（更高效）

```
#include "FreeRTOS.h"
#include "task.h"

TaskHandle_t xHandlerTaskHandle;

// 处理任务
void Notification_Handler_Task(void *pvParameters) {
    uint32_t ulNotificationValue;
```

```

while (1) {
    // 等待通知
    xTaskNotifyWait(0x00, 0xFFFFFFFF, &ulNotificationValue, portMAX_DELAY);

    // 处理不同类型的通知
    if (ulNotificationValue & 0x01) {
        // 处理事件1
    }
    if (ulNotificationValue & 0x02) {
        // 处理事件2
    }
}

// 中断
void EXTI0_IRQHandler(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

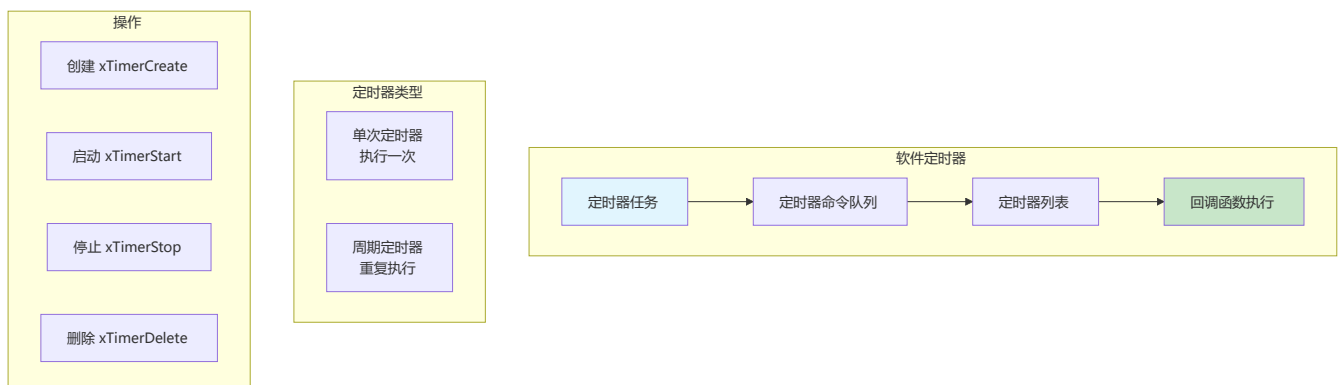
    // 发送通知（比信号量更快）
    vTaskNotifyGiveFromISR(xHandlerTaskHandle, &xHigherPriorityTaskWoken);

    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    EXTI_ClearITPendingBit(EXTI_Line0);
}

```

24.4 软件定时器

软件定时器原理



软件定时器示例

```

#include "FreeRTOS.h"
#include "timers.h"

// 定时器句柄
TimerHandle_t xOneShotTimer;
TimerHandle_t xPeriodicTimer;

// 定时器回调函数

```

```

void OneShotTimerCallback(TimerHandle_t xTimer) {
    printf("单次定时器触发\n");
    // 只会执行一次
}

void PeriodicTimerCallback(TimerHandle_t xTimer) {
    static uint32_t count = 0;
    count++;
    printf("周期定时器触发, 次数: %d\n", count);
    // 会周期性执行
}

// 创建定时器
void Create_Timers(void) {
    // 单次定时器: 1秒后触发
    xOneShotTimer = xTimerCreate(
        "OneShot", // 名称
        pdMS_TO_TICKS(1000), // 周期
        pdFALSE, // 自动重载 (pdFALSE=单次)
        (void *)0, // ID
        OneShotTimerCallback // 回调函数
    );

    // 周期定时器: 每500ms触发
    xPeriodicTimer = xTimerCreate(
        "Periodic",
        pdMS_TO_TICKS(500),
        pdTRUE, // 自动重载 (pdTRUE=周期)
        (void *)1,
        PeriodicTimerCallback
    );

    // 启动定时器
    if (xOneShotTimer != NULL) {
        xTimerStart(xOneShotTimer, 0);
    }

    if (xPeriodicTimer != NULL) {
        xTimerStart(xPeriodicTimer, 0);
    }
}

// 定时器控制任务
void Timer_Control_Task(void *pvParameters) {
    while (1) {
        if (GetSomeCondition()) {
            // 停止定时器
            xTimerStop(xPeriodicTimer, pdMS_TO_TICKS(100));
        }

        if (GetAnotherCondition()) {
            // 重启定时器 (改变周期)
            xTimerChangePeriod(xPeriodicTimer, pdMS_TO_TICKS(1000), pdMS_TO_TICKS(100));
        }
    }
}

```

```
        xTimerReset(xPeriodicTimer, pdMS_TO_TICKS(100));
    }

    vTaskDelay(pdMS_TO_TICKS(100));
}
}
```

定时器API函数

```
// 创建定时器
TimerHandle_t xTimerCreate(const char *pcTimerName,
                           TickType_t xTimerPeriod,
                           UBaseType_t uxAutoReload,
                           void *pvTimerID,
                           TimerCallbackFunction_t pxCallbackFunction);

// 启动定时器
BaseType_t xTimerStart(TimerHandle_t xTimer, TickType_t xTicksToWait);

// 停止定时器
BaseType_t xTimerStop(TimerHandle_t xTimer, TickType_t xTicksToWait);

// 重置定时器（重新开始计时）
BaseType_t xTimerReset(TimerHandle_t xTimer, TickType_t xTicksToWait);

// 改变定时器周期
BaseType_t xTimerChangePeriod(TimerHandle_t xTimer, TickType_t xNewPeriod, TickType_t
xTicksToWait);

// 删除定时器
BaseType_t xTimerDelete(TimerHandle_t xTimer, TickType_t xTicksToWait);

// 获取定时器ID
void *pvTimerGetTimerID(TimerHandle_t xTimer);

// 设置定时器ID
void vTimerSetTimerID(TimerHandle_t xTimer, void *pvNewID);

// 检查定时器是否运行
BaseType_t xTimerIsTimerActive(TimerHandle_t xTimer);

// 从ISR调用的版本
BaseType_t xTimerStartFromISR(TimerHandle_t xTimer, BaseType_t *pxHigherPriorityTaskWoken);
BaseType_t xTimerStopFromISR(TimerHandle_t xTimer, BaseType_t *pxHigherPriorityTaskWoken);
```


24.5 临界区保护

临界区API

```
#include "FreeRTOS.h"
#include "task.h"

// 任务级临界区
void Critical_Section_Example(void) {
    // 进入临界区（关闭中断）
    taskENTER_CRITICAL();

    // 这段代码不会被中断打断
    // 访问共享资源
    sharedResource++;

    // 退出临界区（恢复中断）
    taskEXIT_CRITICAL();
}

// 嵌套临界区
void Nested_Critical_Section(void) {
    taskENTER_CRITICAL();
    {
        // 嵌套深度 = 1

        taskENTER_CRITICAL();
        {
            // 嵌套深度 = 2
        }
        taskEXIT_CRITICAL();

        // 嵌套深度 = 1
    }
    taskEXIT_CRITICAL();

    // 嵌套深度 = 0，中断恢复
}

// ISR级临界区（用于中断中）
void ISR_Critical_Section(void) {
    UBaseType_t uxSavedStatus;

    uxSavedStatus = taskENTER_CRITICAL_FROM_ISR();

    // 临界区代码

    taskEXIT_CRITICAL_FROM_ISR(uxSavedStatus);
}

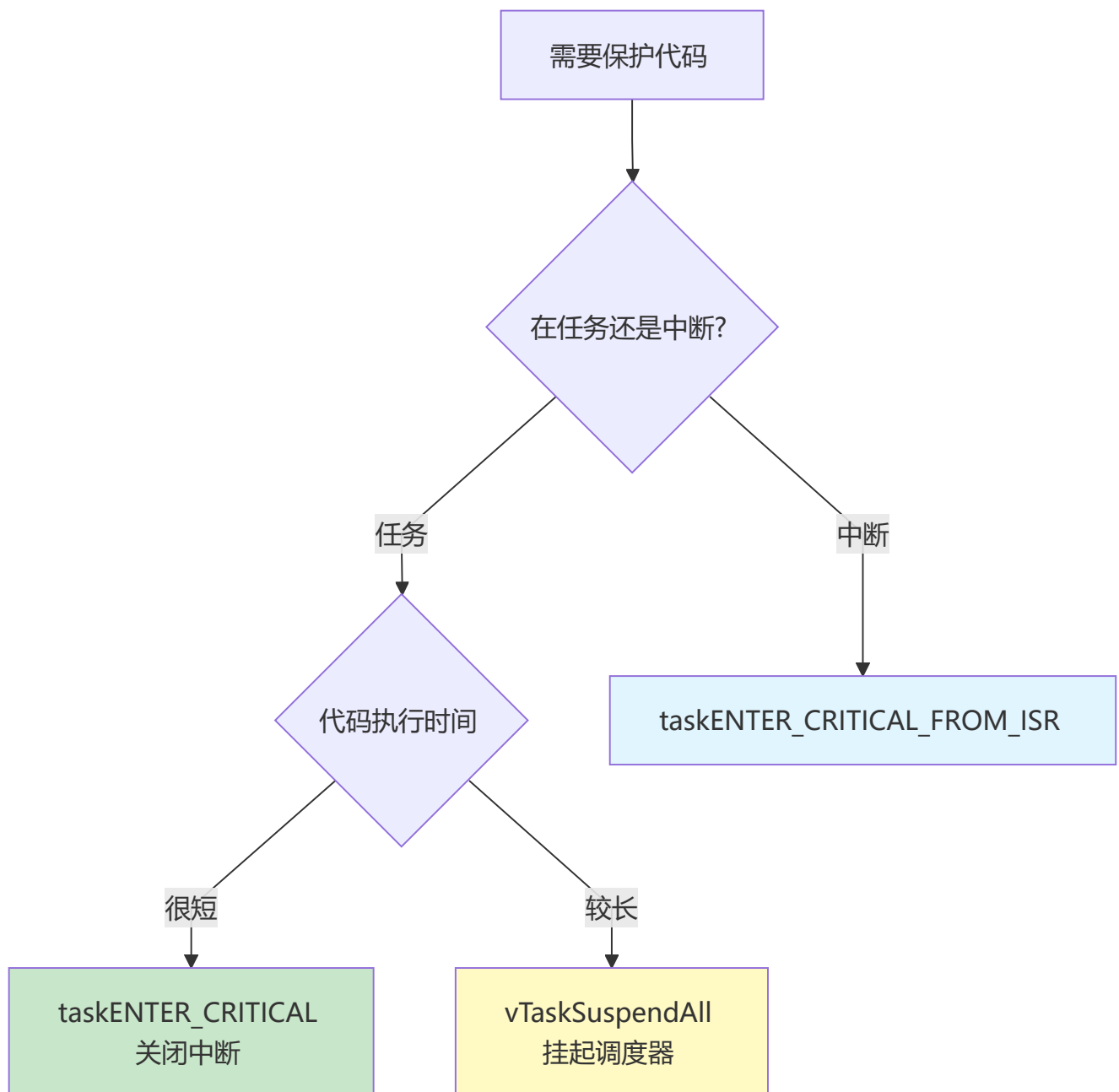
// 挂起调度器（不关闭中断，只是暂停调度）
void Scheduler_Suspend_Example(void) {
```

```
vTaskSuspendAll();

// 这段代码不会被其他任务打断
// 但可以被中断打断

xTaskResumeAll();
}
```

临界区选择



24.6 中断与定时器综合示例

```
#include "FreeRTOS.h"
#include "task.h"
```

```

#include "timers.h"
#include "semphr.h"

// 句柄定义
TaskHandle_t xMainTaskHandle;
TimerHandle_t xHeartbeatTimer;
SemaphoreHandle_t xButtonSemaphore;

// 按键中断处理
void EXTI0_IRQHandler(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // 通知按键任务
    xSemaphoreGiveFromISR(xButtonSemaphore, &xHigherPriorityTaskWoken);

    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    EXTI_ClearITPendingBit(EXTI_Line0);
}

// 心跳定时器回调
void HeartbeatTimerCallback(TimerHandle_t xTimer) {
    LED_Toggle(); // 翻转LED
}

// 按键处理任务
void Button_Task(void *pvParameters) {
    while (1) {
        if (xSemaphoreTake(xButtonSemaphore, portMAX_DELAY) == pdTRUE) {
            // 按键处理（消抖、功能执行等）
            vTaskDelay(pdMS_TO_TICKS(20)); // 消抖延时

            if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_0) == 0) {
                printf("按键按下\n");

                // 可以在这里启停定时器
                if (xTimerIsTimerActive(xHeartbeatTimer)) {
                    xTimerStop(xHeartbeatTimer, 0);
                } else {
                    xTimerStart(xHeartbeatTimer, 0);
                }
            }
        }
    }
}

// 系统初始化
void System_Init(void) {
    // 创建信号量
    xButtonSemaphore = xSemaphoreCreateBinary();

    // 创建心跳定时器（500ms周期）
    xHeartbeatTimer = xTimerCreate(
        "Heartbeat",

```

```
    pdMS_TO_TICKS(500),
    pdTRUE,
    (void *)0,
    HeartbeatTimerCallback
);

// 启动定时器
xTimerStart(xHeartbeatTimer, 0);

// 创建按键任务
xTaskCreate(Button_Task, "Button", 128, NULL, 2, NULL);

// 启动调度器
vTaskStartScheduler();
}

int main(void) {
    // 硬件初始化
    HAL_Init();
    LED_Init();
    Button_Init();
    UART_Init();

    // 创建信号量和任务
    System_Init();

    while (1);
}
```

第二十四章小结

本章要点

1. 中断服务程序要短，把处理推迟到任务
2. 中断中必须使用FromISR版本的API
3. 软件定时器在定时器任务中执行回调
4. 临界区用于保护短代码段
5. 合理配置中断优先级

动手练习

练习1：实现按键中断通知任务处理。

练习2：创建周期定时器实现LED闪烁。

练习3：使用临界区保护共享变量。



FreeRTOS速查总表

类别	API	说明
任务	xTaskCreate	创建任务
任务	vTaskDelete	删除任务
任务	vTaskDelay	相对延时
任务	vTaskDelayUntil	绝对延时
队列	xQueueCreate	创建队列
队列	xQueueSend	发送消息
队列	xQueueReceive	接收消息
信号量	xSemaphoreCreateBinary	创建二值信号量
信号量	xSemaphoreCreateCounting	创建计数信号量
信号量	xSemaphoreGive	释放信号量
信号量	xSemaphoreTake	获取信号量
互斥量	xSemaphoreCreateMutex	创建互斥量
事件组	xEventGroupCreate	创建事件组
事件组	xEventGroupSetBits	设置事件位
事件组	xEventGroupWaitBits	等待事件
任务通知	xTaskNotifyGive	发送通知
任务通知	ulTaskNotifyTake	接收通知
定时器	xTimerCreate	创建定时器
定时器	xTimerStart	启动定时器
内存	pvPortMalloc	分配内存
内存	vPortFree	释放内存
临界区	taskENTER_CRITICAL	进入临界区
临界区	taskEXIT_CRITICAL	退出临界区

学习建议：FreeRTOS是实践性很强的技术，建议结合STM32等开发板进行实践，从简单的多任务创建开始，逐步掌握各种同步机制。

进阶方向：RTOS原理深入、内存管理优化、实时性分析、多核RTOS